



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

Trabajo fin de Grado

Grado en Ingeniería Informática – Ingeniería del Software

**Linceus Assistant: chatbot universitario con RAG para el alumnado de la
ETSII**

**Realizado por
Santia Bregu**

**Dirigido por
José Antonio Parejo Maestre**

**Departamento
Lenguajes y Sistemas Informáticos**

Sevilla, 2 de mayo de 2026

*“La IA es una herramienta para aumentar nuestras capacidades, no
para reemplazarnos.”*

— Yann LeCun

Agradecimientos

A mi familia, por el apoyo constante a lo largo de toda la carrera y por hacer posible que llegase hasta este punto.

A mi tutor, José Antonio Parejo Maestre, por la dirección, la disponibilidad y el criterio con el que ha acompañado este proyecto desde el primer día.

A mi equipo de People1 — Joaquín, Miguel y Fran —, sin los que ni siquiera sabría qué es RAG. Lo que en estas páginas aparece como una pieza técnica más, en mi caso fue un punto de inflexión, y se lo debo a ellos.

Resumen

El avance de las herramientas digitales ha generado una expectativa clara en los usuarios: que la información relevante esté disponible de forma rápida y sea fácil de consultar. Esta expectativa, sin embargo, contrasta con la realidad de las instituciones de educación superior. Una universidad maneja un volumen enorme de información heterogénea (planes de estudio, normativas, calendarios, directorios de profesorado, sistemas de gestión académica) distribuida en múltiples fuentes y orientada a perfiles muy distintos: estudiantes, docentes, personal administrativo y órganos de gobierno. Unificar todo ese contenido en un único sistema es, por su propia naturaleza, una tarea de gran complejidad, y es razonable asumir que esta fragmentación seguirá siendo la norma en el futuro. La información existe y es accesible; el problema es que recuperarla exige conocer qué herramienta contiene cada dato y navegar entre ellas. La brecha entre lo que el usuario espera de un sistema de información moderno y lo que la institución puede ofrecer no es un fallo puntual, sino una limitación estructural conocida.

Este Trabajo Fin de Grado presenta **Linceus Assistant**, un asistente conversacional que cierra esa distancia consolidando el acceso a asignaturas, horarios y profesorado en un único punto de entrada. Desarrollado inicialmente para el alumnado de la ETSII de la Universidad de Sevilla, el sistema está concebido para extenderse al resto de centros de la universidad y, por su diseño, a cualquier institución de educación superior. Frente a los chatbots universitarios desplegados en el ecosistema español, la propuesta se diferencia en dos aspectos. Primero, está enfocada al **estudiante ya matriculado**, no solo al ciclo de captación y matrícula que cubren los asistentes institucionales existentes. Segundo, adopta una arquitectura de **recuperación aumentada por generación** (RAG) sobre los documentos oficiales, lo que le permite responder a preguntas no anticipadas en un guion cerrado, frente al modelo de menús predefinidos habitual en las soluciones comerciales.

El sistema se ha construido sobre Rasa 3.6, integrado con la API de Gemini para clasificación de intenciones, generación de SQL y renderizado de respuestas. La base de datos reside en Supabase con la extensión `pgvector`, lo que permite combinar consultas estructuradas con recuperación semántica sobre los planes docentes. El despliegue se realiza con Docker Compose en cuatro contenedores. El producto cubre los tres dominios funcionales del prototipo (**asignaturas, horarios y profesorado**), incorpora cambio dinámico de titulación y dispone de un panel de administración para garantizar la mantenibilidad del sistema.

La validación se ha organizado en cinco capas independientes y todas superan su umbral académico de referencia: **94,3 %** en la suite E2E (159 casos), **91,9 %** en RAG sobre 74 preguntas extraídas de planes docentes reales, **100 %** en política Rasa, **98,3 %** en el piloto con 59 sesiones de alumnos reales y una suite específica para el **panel de administración** que cubre los flujos críticos de gestión de datos. Las métricas no funcionales acompañan: latencia mediana de 3,4 segundos por turno y coste extrapolado entre 8 y 50 euros anuales para 700 alumnos, dos órdenes de magnitud por debajo de cualquier suscripción comercial equivalente.

La arquitectura es **extensible y escalable** por construcción: ampliar el sistema a nuevas titulaciones u otros centros de la Universidad de Sevilla, o incorporar dominios adicionales, descansa principalmente en la carga de datos a través del panel de administración, sin tocar el núcleo conversacional. El proyecto demuestra que la tecnología disponible permite cerrar esa distancia entre expectativa y realidad de forma sencilla, validada cuantitativamente y económicamente sostenible empleando herramientas accesibles a un trabajo de fin de grado.

Abstract

The progress of digital tools has set a clear expectation among users: that relevant information should be quickly available and easy to consult. This expectation, however, stands in contrast to the reality of higher education institutions. A university handles a large volume of heterogeneous information (study plans, regulations, calendars, faculty directories, academic management systems) distributed across multiple sources and aimed at very different audiences: students, faculty, administrative staff, and governing bodies. Bringing all that content together into a single system is, by its very nature, a highly complex task, and it is reasonable to assume that this fragmentation will remain the norm going forward. The information exists and is accessible; the problem is that retrieving it requires knowing which tool holds each piece of data and navigating between them. The gap between what a user expects from a modern information system and what the institution can offer is not an isolated shortcoming, but a well-known structural limitation.

This Bachelor's Thesis introduces **Linceus Assistant**, a conversational assistant that closes this gap by consolidating access to courses, timetables, and faculty into a single entry point. Initially developed for students of the ETSII at the University of Seville, the system is designed to extend to other schools within the university and, by its architecture, to any higher education institution. Compared to university chatbots deployed across the Spanish ecosystem, the proposal differs in two key aspects. First, it targets the **enrolled student**, not only the admissions and registration cycle covered by existing institutional assistants. Second, it adopts a **retrieval-augmented generation** (RAG) architecture over the official documents, which allows it to answer questions not anticipated by a closed script, in contrast to the menu-driven model typical of commercial solutions.

The system has been built on Rasa 3.6, integrated with the Gemini API for intent classification, SQL generation, and response rendering. The database resides in Supabase with the `pgvector` extension, which enables combining structured queries with semantic retrieval over teaching plans. Deployment is carried out with Docker Compose across four containers. The product covers the three functional domains of the prototype (**courses, timetables, and faculty**), supports dynamic switching between degrees, and provides an administration panel to ensure long-term maintainability.

Validation has been organised into five independent layers, and all of them exceed their academic reference threshold: **94.3 %** on the E2E suite (159 cases), **91.9 %** on RAG over 74 questions extracted from real syllabi, **100 %** on Rasa policy accuracy, **98.3 %** on the pilot with 59 sessions of real students, and a dedicated test suite for the **administration panel** covering the critical data-management flows. Non-functional metrics keep pace: a median latency of 3.4 seconds per turn and an extrapolated cost between 8 and 50 euros per year for 700 students, two orders of magnitude below any equivalent commercial subscription.

The architecture is **extensible and scalable** by construction: extending the system to new degrees, to other schools at the University of Seville, or incorporating additional domains relies mostly on loading data through the administration panel, without touching the conversational core. The project shows that available technology makes it feasible to close the gap between expectation and reality in a simple, quantitatively validated, and economically sustainable way, using tools accessible to a final-year project.

Índice general

Agradecimientos	2
Resumen	3
Abstract	4
I Introducción y Conceptos	
1 Introducción	16
1.1 Contexto	16
1.2 Motivación	16
1.3 Objetivos	17
1.3.1 Objetivo general	17
1.3.2 Objetivos técnicos	17
1.3.3 Objetivos académicos y personales	18
1.4 Soluciones existentes	18
1.4.1 Asistentes en universidades españolas	18
1.4.2 Plataformas internacionales	19
1.4.3 Trabajos académicos comparables	19
1.4.4 Comparativa y diferenciación	20
1.5 Estructura de la memoria	21
2 Preparación del proyecto	23
2.1 Estudio de las tecnologías y toma de decisiones	23
2.1.1 Decisión de frameworks conversacionales	23
2.1.2 Decisión de base de datos	26
2.1.3 Decisión de API de LLM	29
2.2 Infraestructura del sistema	31
2.2.1 Arquitectura del sistema	32
2.2.2 Componentes tecnológicos del sistema	32
2.3 Casos de uso principales	32
2.3.1 Gestión del contexto académico	33
2.3.2 Consulta sobre información de asignaturas	33
2.3.3 Consulta sobre horarios	34
2.3.4 Consulta sobre datos de contacto del profesorado	35
3 Conceptos iniciales	36
3.1 Sistema conversacional	36
3.2 Componentes del sistema conversacional con Rasa	36
3.2.1 Comprensión del lenguaje natural (NLU)	36
3.2.2 Gestión del diálogo	37
3.2.3 Mecanismos de respuesta: actions	37
3.2.4 Formularios	37
3.2.5 Fallback	37

3.3	Archivos de configuración de Rasa	37
3.4	Flujo de procesamiento de un mensaje	38
3.5	Generación aumentada por recuperación (RAG)	38
3.6	Backend y persistencia de datos	38
3.6.1	Supabase y PostgreSQL	38
3.6.2	pgvector	39
3.6.3	Embeddings	39

II Metodología y Planificación

4	Metodología	41
4.1	Metodología de desarrollo: Scrum adaptado	41
4.1.1	Scrum: fundamentos	41
4.1.2	Adaptación al proyecto individual	41
4.1.3	Sprints y duración	42
4.2	Gestión del código fuente	42
4.2.1	Política de ramas	42
4.2.2	Convenciones de commits	42
4.3	Estándares de codificación	42
4.4	Gestión del trabajo	43
4.5	Versionado del sistema	43
5	Planificación	45
5.1	Estructura de Descomposición del Trabajo (EDT)	45
5.2	Diccionario EDT	45
5.2.1	Fase 1 — Inicio	45
5.2.2	Fase 2 — Desarrollo	47
5.2.3	Fase 3 — Cierre	48
5.3	Resumen cuantitativo de la EDT	51
5.4	Backlog del producto	51
5.5	Planificación de sprints	52
5.6	Cronograma	52

III Diseño e Implementación

6	Análisis de requisitos	54
6.1	Identificación de actores	54
6.2	Requisitos funcionales	55
6.2.1	Dominio de asignaturas	55
6.2.2	Dominio de horarios	55
6.2.3	Dominio de profesorado	57
6.2.4	Comportamientos transversales	57
6.2.5	Funcionalidades del panel de administración	57
6.3	Requisitos de información	57
6.3.1	Entidades de dominio	57
6.3.2	Entidades del módulo RAG	57
6.3.3	Entidades de operación	58
6.4	Requisitos no funcionales	58
6.5	Prototipos de interfaz	59
6.6	Evolución del alcance respecto a la propuesta inicial	61
7	Arquitectura del sistema y decisiones de diseño	63
7.1	Arquitectura general	63

7.1.1	Capa de presentación	63
7.1.2	Capa de procesamiento conversacional	63
7.1.3	Capa de datos	64
7.2	Diagrama de componentes	64
7.3	Diagrama de despliegue	66
7.3.1	Nodos de ejecución	66
7.3.2	Protocolos de comunicación	67
7.4	Decisiones de diseño	67
7.4.1	Text-to-SQL con LLM en lugar de reglas estáticas	67
7.4.2	Gemini como servicio principal con Ollama como alternativa local	68
7.4.3	Embeddings de 2000 dimensiones con HNSW	68
7.4.4	Chunking con solapamiento para preservar contexto	68
7.4.5	Búsqueda vectorial con fallback a búsqueda por palabras clave	69
7.4.6	Fuzzy matching para resolución de nombres	69
7.4.7	Slots conversacionales para memoria de contexto	69
7.4.8	Pipeline NLU de 9 etapas con doble featurización	70
7.4.9	Hash SHA-256 para evitar reprocesamiento de documentos	70
7.5	Patrones arquitectónicos aplicados	71
7.5.1	Arquitectura por capas (<i>Layered Architecture</i>)	71
7.5.2	Patrón <i>Action Server</i> (microservicio de lógica)	71
7.5.3	Patrón <i>Strategy</i> para clientes LLM	71
7.5.4	Patrón <i>Pipeline</i> para procesamiento RAG	71
7.5.5	Patrón <i>Repository</i> para acceso a datos	71
8	Marco tecnológico	73
8.1	Tecnologías de procesamiento conversacional	73
8.1.1	Rasa Framework	73
8.1.2	Procesamiento de Lenguaje Natural	74
8.2	Tecnologías de Inteligencia Artificial	74
8.2.1	Ollama y modelos de lenguaje grandes (LLMs)	74
8.2.2	Embeddings y búsqueda semántica	75
8.3	Tecnologías de base de datos	75
8.3.1	PostgreSQL	75
8.3.2	Supabase	75
8.3.3	pgvector	76
8.4	Tecnologías de frontend	76
8.4.1	Node.js	76
8.4.2	Framework web	76
8.5	Herramientas de desarrollo	76
8.5.1	Control de versiones	76
8.5.2	Gestión de dependencias	76
8.5.3	Variables de entorno	77
8.6	Consideraciones de seguridad	77
8.6.1	Protección de datos	77
8.6.2	Validación de entrada	77
8.6.3	Ejecución local de modelos	77
8.7	Stack tecnológico completo	77
9	Estructura del proyecto	79
9.1	Visión general del repositorio	79
9.2	Directorio raíz	80
9.3	Directorio <code>actions/</code>	80

9.3.1	Organización de los módulos	80
9.3.2	Flujo de ejecución de una acción	81
9.4	Directorio <code>data/</code>	81
9.4.1	Subdirectorio <code>nlu/</code>	81
9.4.2	Ficheros de flujo conversacional	82
9.5	Directorio <code>frontend/</code>	82
9.6	Directorio <code>data/</code> de Rasa vs. docs/	82
9.7	Directorio <code>tests/</code>	82
9.8	Separación de responsabilidades	83
10	Estructura de la base de datos	84
10.1	Diseño conceptual	84
10.1.1	Principios de diseño	84
10.2	Modelo entidad-relación	84
10.3	Entidades principales	84
10.3.1	Universidades	84
10.3.2	Centros	85
10.3.3	Departamentos	85
10.3.4	Titulaciones	85
10.3.5	Asignaturas	86
10.3.6	Profesores	86
10.3.7	Horarios	86
10.4	Tablas de vectorización (RAG)	88
10.4.1	Planes docentes	88
10.4.2	Chunks de planes docentes	88
10.5	Relaciones entre entidades	89
10.6	Índices y optimización	89
10.7	Integridad referencial	89
10.8	Estrategia de datos de prueba	90
11	Implementación	91
11.1	Anatomía común de una <i>custom action</i>	91
11.2	Épica de asignaturas	92
11.2.1	Consulta específica de asignatura	92
11.2.2	Listado y conteo de asignaturas	92
11.2.3	Horario por asignatura	94
11.3	Épica de profesores	94
11.4	Épica de horarios	96
11.5	Épica de contexto académico	96
11.6	Épica de fallback inteligente	98
11.7	Épica de <i>multi-intent</i>	98
11.8	Módulos compartidos	99
11.8.1	Cliente de Gemini	99
11.8.2	Conexión a base de datos	99
11.8.3	Matching difuso de nombres	99
IV	Validación y Cierre	
12	Pruebas y Validación	101
12.1	Estrategia general	101
12.1.1	Marco metodológico	101
12.1.2	Política de exclusión por trabajo futuro	102

12.1.3	Ejecución y registro	102
12.2	Ejecución funcional extremo a extremo	102
12.2.1	Alcance	102
12.2.2	Lectura frente a la literatura	103
12.3	Recuperación aumentada por generación	103
12.3.1	Capa baseline	103
12.3.2	Capa de profundidad	103
12.4	Clasificación NLU y política de diálogo	104
12.5	Reproducción de tráfico real tras el despliegue	105
12.5.1	Metodología de revisión	105
12.5.2	Resultados	105
12.6	Métricas no funcionales: latencia y coste	105
12.6.1	Latencia	106
12.6.2	Coste por turno	106
12.7	Visión integrada y discusión	106
12.7.1	Cumplimiento del criterio de cierre	107
12.7.2	Limitaciones y trabajo futuro	107
12.8	Pruebas automatizadas del panel de administración	107
12.8.1	Resultados	108
13	Manuales	109
13.1	Manual de usuario	109
13.1.1	Acceso a la aplicación	109
13.1.2	Modelo conversacional	111
13.1.3	Funcionalidades disponibles	111
13.1.4	Recomendaciones de uso	111
13.1.5	Panel de administración	112
13.2	Manual de despliegue	113
13.2.1	Requisitos previos	115
13.2.2	Instalación	115
13.2.3	Arquitectura del despliegue	115
13.2.4	Verificación del despliegue	116
13.2.5	Operaciones habituales	116
13.3	Manual de desarrollo	117
13.3.1	Estructura del repositorio	117
13.3.2	Entorno de desarrollo	117
13.3.3	Ciclo de incorporación de cambios	118
13.3.4	Cómo añadir una nueva intención	118
13.3.5	Buenas prácticas y convenciones	119
14	Conclusiones y desarrollos futuros	121
14.1	Cumplimiento de objetivos	121
14.1.1	Objetivo general	121
14.1.2	Objetivos técnicos	121
14.1.3	Objetivos académicos y personales	122
14.2	Lecciones aprendidas	122
14.3	Análisis de desviación	123
14.4	Desarrollos futuros	123
14.4.1	Optimización del sistema actual	123
14.4.2	Ampliación funcional	124
14.4.3	Validación reforzada	124
14.5	Reflexión final	124

15 Uso de Inteligencia Artificial	126
15.1 Herramientas utilizadas	126
15.2 Tareas asistidas y verificación	126
15.3 Limitaciones y responsabilidad del autor	126
16 Vídeo demostración y presentación	127
16.1 Vídeo demostración	127
16.2 Estructura de la presentación	127
Apéndices	128
.1 Actas de reuniones con el tutor	128
.2 Listado completo de casos de prueba	131
.3 Esquema SQL de la base de datos	131
.4 Configuración de Rasa	131
.5 Informe de tiempo invertido	131
Bibliografía	132

Índice de figuras

5.1	Estructura de Descomposición del Trabajo (EDT) — fases y paquetes de trabajo.	46
6.1	Widget conversacional accesible para el estudiante.	59
6.2	Panel de administración: gestión de asignaturas.	60
6.3	Panel de administración: auditoría de conversaciones.	60
6.4	Panel de administración: gestión de horarios.	60
6.5	Panel de administración: gestión de profesorado.	61
7.1	Diagrama de componentes del sistema Linceus.	65
7.2	Diagrama de despliegue del sistema Linceus en DigitalOcean (Ubuntu 24.04 LTS) con el pipeline de despliegue automatizado vía GitHub Actions.	72
11.1	Pipeline de <code>ActionConsultaEspecifica</code>	93
11.2	Pipeline de listado y conteo (<i>text-to-SQL</i>).	94
11.3	Pipeline de <code>ActionConsultaProfesor</code>	95
11.4	Pipeline de <code>ActionConsultaHorario</code>	97
11.5	Pipeline de <code>ActionSmartFallback</code>	98
13.1	Página principal del prototipo con el <i>widget</i> de chat cerrado.	110
13.2	<i>Widget</i> desplegado con mensaje de bienvenida y selector de titulación.	110
13.3	Ejemplo de sesión con seguimiento conversacional y cambio de contexto.	112
13.4	Panel de administración: vista de centros y titulaciones.	112
13.5	Panel de administración: gestión del profesorado.	113
13.6	Panel de administración: vista de horarios.	113
13.7	Panel de administración: registro de conversaciones anonimizadas.	114
13.8	Panel de administración: estadísticas de uso del sistema.	114

Índice de tablas

1.1	Comparativa de soluciones existentes frente a Linceus Assistant.	20
2.1	Comparación de frameworks conversacionales	25
2.2	Comparación de bases de datos	28
2.3	Comparación de APIs de LLM	31
2.4	Componentes tecnológicos del sistema en producción.	32
3.1	Archivos de configuración principales del sistema Rasa	38
5.1	Diccionario EDT — 1.1: Gestión del proyecto.	46
5.2	Diccionario EDT — 1.2: Investigación tecnológica.	46
5.3	Diccionario EDT — 1.3: Diseño de la arquitectura.	47
5.4	Diccionario EDT — 2.1: Épica Asignaturas.	47
5.5	Diccionario EDT — 2.2: Épica Profesores.	48
5.6	Diccionario EDT — 2.3: Épica Horarios.	48
5.7	Diccionario EDT — 2.4: Pipeline RAG.	49
5.8	Diccionario EDT — 2.5: Panel admin y frontend.	49
5.9	Diccionario EDT — 3.1: Validación y testing.	50
5.10	Diccionario EDT — 3.2: Piloto con usuarios.	50
5.11	Diccionario EDT — 3.3: Despliegue en producción.	50
5.12	Diccionario EDT — 3.4: Memoria y defensa.	51
5.13	Resumen cuantitativo de la EDT por fase y paquete de trabajo.	51
5.14	Priorización del backlog dentro de la fase de Desarrollo.	52
6.1	Requisitos funcionales del dominio de asignaturas.	55
6.2	Requisitos funcionales del dominio de horarios.	55
6.3	Requisitos funcionales del dominio de profesorado.	56
6.4	Requisitos funcionales transversales.	56
6.5	Requisitos funcionales del panel de administración.	57
6.6	Entidades de información del dominio académico.	58
6.7	Requisitos no funcionales y evidencia.	59
6.8	Evolución del alcance respecto a la propuesta inicial.	61
7.1	Protocolos y puertos de comunicación entre nodos del sistema.	67
8.1	Stack tecnológico completo del proyecto LinceUS Assistant.	78
10.1	Estructura de la tabla UNIVERSIDADES	85
10.2	Estructura de la tabla CENTROS	85
10.3	Estructura de la tabla DEPARTAMENTOS	85
10.4	Estructura de la tabla TITULACIONES	86
10.5	Estructura de la tabla ASIGNATURAS	86
10.6	Estructura de la tabla PROFESORES	87
10.7	Estructura de la tabla HORARIOS	87
10.8	Estructura de la tabla PLANES_DOCENTES	88

10.9 Estructura de la tabla PLANES_DOCENTES_CHUNKS	88
12.1 Resultados por categoría de la suite E2E (revisión manual, 26/04/2026).	103
12.2 Cobertura de la capa de profundidad RAG por curso.	104
12.3 Resultados de la capa NLU y Policy.	104
12.4 Resultados de la reproducción de tráfico real.	105
12.5 Latencia extremo a extremo (segundos).	106
12.6 Extrapolación del coste anual a 700 alumnos según intensidad de uso.	106
12.7 Visión integrada de las cinco capas de validación funcional.	107
12.8 Resultados de la suite <code>test_admin.py</code> (pytest, 26/04/2026).	108
13.1 Tipos de consulta soportados por el prototipo.	111
13.2 Requisitos mínimos del entorno de despliegue.	115
13.3 Contenedores del despliegue y responsabilidad de cada uno.	116
13.4 Estructura del repositorio Linceus Assistant.	117
13.5 Etapas del ciclo de incorporación de cambios.	118
14.1 Cruce entre objetivos técnicos y evidencia obtenida.	122

Índice de código

13.1	Levantar la pila completa con Docker Compose.	115
13.2	Consultar registros de un contenedor.	116
13.3	Operaciones de mantenimiento habituales.	116
13.4	Preparacion del entorno de desarrollo local.	117
13.5	Arranque de los componentes en modo desarrollo.	117
13.6	Declaracion de una intencion con ejemplos NLU en <code>data/nlu.yml</code>	118
13.7	Estructura minima de una accion personalizada en Rasa.	119
13.8	Fragmento de <code>validar_sql</code> : lista de palabras prohibidas y verificacion de tabla.	119

Parte I

Introducción y Conceptos

Introducción

Este proyecto tiene como finalidad el diseño y desarrollo de un chatbot inteligente orientado a facilitar el acceso a información académica y administrativa en la Universidad de Sevilla, mediante técnicas de procesamiento del lenguaje natural y recuperación aumentada por generación (RAG).

1.1– Contexto

En la actualidad, el alumnado de la Universidad de Sevilla se enfrenta a una notable dispersión informativa. La información académica se encuentra repartida entre múltiples plataformas y páginas web, cada una gestionada de forma independiente por distintas facultades, servicios y departamentos. El portal institucional principal [1], la web de la propia ETSII [2], los portales internos Sevius [3] y Sevius4 [4], la Secretaría Virtual [5] y los procedimientos de matrícula y pago de tasas [6, 7] son solo algunos ejemplos. Esta fragmentación dificulta el acceso eficiente a datos esenciales como horarios, contacto con profesorado o información de las asignaturas.

Además, muchos contenidos relevantes no se presentan de forma unificada ni están estructurados para su consulta directa, sino que requieren navegar por documentos PDF (como proyectos docentes), correos informativos, páginas institucionales específicas u otros portales como Sevius [3] o Sevius4 [4]. Esta situación resulta especialmente compleja para estudiantes de nuevo ingreso o internacionales, que no siempre conocen el funcionamiento interno del entorno universitario y, en muchos casos, deben enfrentarse también a la barrera del idioma, lo que dificulta aún más la comprensión de la información disponible.

La ausencia de un punto de acceso único, interactivo y orientado al usuario final pone de manifiesto la necesidad de una herramienta conversacional que permita acceder a la información de manera contextual, simplificada y guiada. En este sentido, un chatbot capaz de interpretar consultas abiertas en lenguaje natural y conectarse con diferentes fuentes de información representa una solución innovadora, alineada con los principios de accesibilidad, automatización y modernización de los servicios universitarios.

1.2– Motivación

La motivación principal de este proyecto surge de la necesidad de centralizar el acceso a información académica dispersa y mejorar la experiencia del estudiante en su interacción con los sistemas universitarios. Conviene subrayar que esta dificultad no se limita al alumnado de nuevo ingreso o de programas de intercambio: la mayoría de estudiantes, incluso tras varios años en la titulación, no domina todos los procedimientos institucionales ni recuerda en qué portal reside cada dato. Quien necesita consultar un horario, el correo de un profesor o un detalle del plan docente termina con frecuencia recurriendo a buscadores externos o preguntando a compañeros, no porque la información no exista, sino porque está repartida entre fuentes que el usuario no consulta a diario.

Esta problemática no solo genera frustración y pérdida de tiempo, sino que también puede afectar negativamente al rendimiento académico y a la integración del estudiante en la comunidad universitaria. La dispersión de la información en múltiples fuentes web, documentos PDF y portales específicos dificulta la autonomía informativa del alumnado, que debe dedicar un esfuerzo significativo a buscar respuestas que deberían estar disponibles de forma inmediata.

Por tanto, el desarrollo de un asistente conversacional capaz de responder de forma rápida, precisa y en lenguaje natural constituye una mejora sustancial en la calidad del servicio universitario, promoviendo la autonomía del estudiante y reduciendo la carga administrativa de los servicios de atención.

1.3– Objetivos

1.3.1. Objetivo general

El objetivo principal de este proyecto es diseñar y desarrollar un chatbot inteligente que proporcione a los estudiantes de la Universidad de Sevilla una vía centralizada, accesible y eficiente para consultar información académica y administrativa relevante. La solución pretende integrar capacidades de comprensión del lenguaje natural mediante modelos de Inteligencia Artificial Generativa, junto con búsquedas en bases de datos estructuradas y vectorizadas, permitiendo resolver dudas frecuentes relacionadas con horarios, profesorado, documentación, normativas o localización de espacios universitarios. El asistente está dirigido al conjunto del alumnado y resulta de especial utilidad para quienes están menos familiarizados con los procedimientos institucionales, perfil que abarca a los estudiantes de nuevo ingreso o internacionales pero también a una parte significativa del alumnado ya matriculado.

1.3.2. Objetivos técnicos

Los objetivos técnicos específicos del proyecto incluyen:

- Implementar un sistema conversacional basado en Rasa que permita gestionar flujos de diálogo complejos y contextuales.
- Integrar la API de Gemini para la detección de intenciones y generación de respuestas enriquecidas mediante lenguaje natural.
- Diseñar y desarrollar una base de datos híbrida en Supabase que combine información estructurada (asignaturas, horarios y profesorado) con embeddings vectoriales para búsquedas semánticas sobre los planes docentes.
- Desarrollar acciones personalizadas en Python que permitan consultar datos estructurados y realizar búsquedas por similitud en documentación académica.
- Implementar un *widget* web ligero (HTML, CSS y JavaScript) que ofrezca una interfaz intuitiva y accesible para la interacción con el chatbot.
- Desarrollar un panel de administración independiente que permita gestionar los datos del sistema (asignaturas, horarios, profesorado, planes docentes) sin intervención técnica, garantizando la mantenibilidad del proyecto a largo plazo.
- Diseñar el sistema con una arquitectura escalable y extensible, de forma que la incorporación de nuevas titulaciones, centros o dominios funcionales pueda realizarse a través de la carga de datos, sin tocar el núcleo conversacional.
- Garantizar el cumplimiento de la normativa de protección de datos mediante el uso exclusivo de información de carácter público y el anonimato de las sesiones de usuario.

1.3.3. Objetivos académicos y personales

Desde el punto de vista académico y personal, este proyecto permite:

- Aplicar conocimientos adquiridos durante el Grado en Ingeniería del Software en un proyecto real de desarrollo de software.
- Adquirir experiencia práctica en el diseño de sistemas conversacionales y en la integración de modelos de inteligencia artificial generativa.
- Profundizar en el uso de técnicas de Retrieval Augmented Generation (RAG) y bases de datos vectoriales.
- Desarrollar competencias en metodologías ágiles aplicadas a proyectos individuales.
- Contribuir a la mejora de los servicios universitarios mediante una solución tecnológica innovadora y accesible.
- Devolver algo a la comunidad estudiantil de la que se forma parte, dejando una base sobre la que futuras generaciones de estudiantes puedan apoyarse y, si así lo deciden, ampliar el sistema a otros centros y titulaciones de la universidad.
- Demostrar la capacidad de diseñar, implementar y documentar un sistema completo de forma autónoma.

1.4– Soluciones existentes

Antes de justificar la propuesta de Linceus Assistant conviene revisar qué sistemas similares se han desplegado ya en el ámbito universitario, tanto dentro del Sistema Universitario Español como en el contexto internacional. La revisión que sigue se centra en asistentes conversacionales orientados a estudiantes que cubren consultas académicas, administrativas y de orientación, descartando otras categorías colindantes como los *chatbots* de atención a profesorado, los sistemas tutoriales adaptativos o los asistentes de evaluación docente, que persiguen objetivos distintos. La selección no pretende ser exhaustiva pero sí representativa: se han incluido los sistemas con mayor presencia institucional, los que comparten la naturaleza académica con el caso de uso del proyecto, y los que aportan elementos diferenciales técnicos o funcionales relevantes para la comparación.

1.4.1. Asistentes en universidades españolas

El despliegue de asistentes conversacionales en universidades españolas se ha intensificado en los últimos años, en buena parte gracias a la actividad comercial de proveedores especializados en el sector. La empresa alicantina 1MillionBot [8] concentra una parte significativa del mercado nacional, con presencia documentada en más de treinta universidades de España y Latinoamérica.

CATi (Universidad de Sevilla) [9, 10] es el asistente más cercano al ámbito de este proyecto. Está integrado en el Centro de Atención a Estudiantes de la propia Universidad de Sevilla y resuelve consultas sobre acceso, admisión, matrícula, becas, plazos y titulaciones, con una cobertura declarada de aproximadamente 160 preguntas frecuentes y disponibilidad permanente. En su periodo de prueba inicial (enero de 2022) atendió alrededor de siete mil consultas procedentes de cuarenta países distintos, con una tasa de acierto declarada del 98 % sobre lenguaje conversacional abierto. Ahora bien, una interacción real con CATi en abril de 2026 muestra una limitación operativa relevante: cuando el alumno introduce una consulta concreta como “soy de ingeniería del software”, el sistema responde con un **menú cerrado de opciones** (preguntas predefinidas

sobre automatrícula, plazos y procedimientos). Es decir, la primera capa de NLU clasifica el dominio, pero el flujo posterior se resuelve por selección guiada y no por recuperación abierta de información, lo que limita la capacidad de respuesta a preguntas no anticipadas en el guion.

Lola (Universidad de Murcia) [11, 12] fue uno de los primeros chatbots universitarios desplegados en España (julio de 2018). Construido sobre la plataforma DialogFlow de Google e integrado por 1MillionBot, está orientado a estudiantes de nuevo ingreso del Distrito Único de la Región de Murcia y cubre dudas sobre pruebas de acceso, admisión y matrícula. Los datos publicados por la propia universidad reportan 4.609 alumnos atendidos, 13.184 conversaciones y una tasa de respuestas acertadas del 91,67 % en sus primeros meses de funcionamiento.

Alhe (Universidad de Granada) [13, 14] es un proyecto más reciente, enmarcado en el Servicio Automatizado de Atención e Información de la UGR, también construido sobre la plataforma de 1MillionBot. Cubre oferta educativa, acceso, admisión, trámites académicos y servicios al estudiantado (becas, movilidad, comedores, salas de estudio, deportes, actividades culturales). Durante su primer año de operación atendió más de 38.708 consultas con una tasa de acierto superior al 91 %.

LUCA (Universidad de Cádiz) [15] es el asistente conversacional inteligente desplegado por el Área de Sistemas de Información de la UCA. Funciona durante 24 horas, 365 días al año, y se ofrece como punto de entrada inmediato a la información dirigida a futuros estudiantes universitarios.

Isidra y AIex (Universidad de Alcalá) representan dos generaciones distintas. **Isidra** [16] es el asistente institucional de la UAH, basado igualmente en aprendizaje automático, que durante sus cinco primeros días de actividad mantuvo más de 6.341 conversaciones con 4.812 personas distintas y una precisión declarada superior al 90 %. **AIex** [17], presentado en abril de 2026, es un asistente desarrollado por estudiantes de la propia UAH como proyecto académico para atender consultas durante la feria AULA 2026; ilustra que la línea de chatbot universitario también ha penetrado el ámbito formativo, no solo el institucional.

1.4.2. Plataformas internacionales

Más allá del Sistema Universitario Español, el ecosistema internacional ofrece dos referencias relevantes por su distinto enfoque.

UniBuddy [18, 19] opera en más de 600 instituciones a nivel global y combina una plataforma de mensajería persona a persona con un asistente conversacional de IA llamado Unibuddy Assistant. Su orientación principal no es la consulta académica del estudiante matriculado, sino la captación y conversión de candidatos durante el proceso de admisión: el sistema sirve para resolver preguntas factuales sobre la institución, sintetizar conversaciones mediante GPT y emparejar al candidato con embajadores de la universidad. Los análisis de conversaciones se generan con tecnología de OpenAI.

Hubert [20] adopta un enfoque distinto y se especializa en la recogida de *feedback* estudiantil sobre asignaturas y profesorado. En lugar de un formulario clásico, el alumno mantiene una conversación con el bot, que formula entre cuatro y cinco preguntas abiertas y agrupa las respuestas para que el docente las revise. Aunque su dominio queda fuera del caso de uso del prototipo Linceus, ilustra la utilidad de los modelos conversacionales para tareas de evaluación cualitativa que tradicionalmente sufren bajas tasas de respuesta.

1.4.3. Trabajos académicos comparables

La literatura científica sobre chatbots universitarios proporciona también un marco comparativo cuantitativo. Cuatro publicaciones se han tomado como referencia en el Capítulo 12 por reportar métricas comparables con las obtenidas por Linceus. **AdmitBot** [21] es un sistema de orientación para procesos de admisión universitaria que reporta una precisión de *intent* del 82 %. **EDUBOT/LISA** [22], desarrollado en el Politécnico, proporciona soporte de *e-learning* con una

tasa de respuestas correctas en torno al 78 %. El sistema descrito por **Ranoliya et al.** [23] cubre preguntas frecuentes universitarias con una exactitud del 85 % sobre cien consultas. Por último, el trabajo de **Dibya y Sahoo** [24] implementa un chatbot universitario basado en IA y NLP con una precisión declarada del 84 %.

1.4.4. Comparativa y diferenciación

La Tabla 1.1 resume las principales soluciones identificadas frente a los criterios diferenciales del proyecto Linceus Assistant.

Cuadro 1.1: Comparativa de soluciones existentes frente a Linceus Assistant.

Sistema	Dominio	Tecnología	Acceso	Diferenciación
CATi (US) [9]	Acceso, admisión, matrícula, becas	1MillionBot (NLP propietario)	Web institucional	Menús guiados; cobertura de FAQ (~160 preguntas)
Lola (UMU) [11]	Acceso y matrícula nuevo ingreso	DialogFlow + 1MillionBot	Web y app DURM	Pionero (2018); 91,67 % acierto
Alhe (UGR) [13]	Oferta educativa y servicios	1MillionBot	Web y Telegram	91 % acierto en primer año
LUCA (UCA) [15]	Información a futuro estudiante	1MillionBot	Web institucional	Disponibilidad 24/7
Isidra (UAH) [16]	Información institucional	1MillionBot	Web institucional	>90 % precisión declarada
UniBuddy [18]	Captación y admisión	GPT (OpenAI)	SaaS multi-canal	Mensajería P2P + IA
Hubert [20]	<i>Feedback</i> de cursos	NLP propietario	Web y enlaces	Encuesta conversacional
AdmitBot [21]	Admisión universitaria (académico)	NLU clásico	Prototipo	82 % <i>intent accuracy</i>
Linceus	Asignaturas, horarios y profesorado ETSII	Rasa 3.6 + Gemini + RAG (<i>pg-vector</i>)	Web (<i>widget</i>)	RAG sobre planes docentes; código abierto; validación cuantitativa

A la vista de la comparativa pueden destacarse cuatro ejes diferenciales del proyecto Linceus Assistant frente al panorama existente.

El primero, y probablemente el más relevante, es el **tipo de información cubierta**. Los asistentes institucionales españoles (CATi, Lola, Alhe, LUCA, Isidra) se concentran fundamentalmente en el ciclo de captación, acceso y matrícula, es decir, en información de tipo administrativo orientada a estudiantes potenciales o que aún no han comenzado sus estudios. Una vez el alumno se matricula, esos asistentes dejan de cubrir el grueso de las preguntas que pasan a ser pertinentes: qué se imparte en cada asignatura, en qué aula y horario tiene clase un determinado grupo, quién coordina una asignatura concreta o cómo se evalúa.

Linceus Assistant cubre exactamente ese hueco. El sistema atiende al estudiante en su día a día académico mediante tres dominios funcionales (asignaturas con su plan docente completo, horarios por curso y grupo, e información de contacto del profesorado) sin renunciar a ser útil también al estudiante de nuevo ingreso, que en sus primeras semanas necesita justamente este tipo de información para orientarse: localizar el aula del primer día, identificar al coordinador de una asignatura o entender el sistema de evaluación. La herramienta no excluye, por tanto, al perfil que cubren CATi y similares, sino que extiende la cobertura más allá del momento de la matrícula.

Esta diferencia no es solo de alcance, sino de naturaleza del dato. Las consultas administrativas que cubren los asistentes existentes se resuelven con un conjunto cerrado de respuestas tipificadas;

las consultas académicas que cubre Linceus se basan en documentación viva (planes docentes que cambian curso a curso, horarios que se publican cada cuatrimestre, asignaciones de profesorado revisables). Para que la información se mantenga al día sin intervención técnica, el sistema incorpora un **panel de administración conectado directamente a la base de datos**, desde el que el personal autorizado puede actualizar asignaturas, horarios, profesorado y planes docentes a través de la propia interfaz web. Cualquier cambio queda inmediatamente disponible para el chatbot, sin necesidad de regenerar el modelo ni de recurrir al proveedor. Las soluciones comerciales basadas en plataformas SaaS no suelen ofrecer este grado de control directo sobre el corpus.

El segundo es el **modelo conversacional**. La interacción real con CATi expuesta al inicio de esta sección permite documentar de forma directa que el asistente combina una primera capa NLU con un flujo guiado por menús en los turnos posteriores. El resto de asistentes españoles citados (Lola, Alhe, LUCA, Isidra) se construyen sobre tecnologías equivalentes (1MillionBot y DialogFlow), basadas también en clasificación de intención sin componente generativo abierto, y la documentación pública de cada uno de ellos describe su funcionamiento como un emparejamiento entre la pregunta del usuario y un conjunto cerrado de respuestas tipificadas. El patrón dominante en el ecosistema español es, por tanto, NLU clásico con menús cerrados o respuestas predefinidas, sin recuperación abierta sobre documentación.

Linceus Assistant adopta un planteamiento distinto: emplea un módulo de recuperación aumentada por generación (RAG) sobre los documentos oficiales, lo que le permite responder a preguntas no anticipadas en el guion siempre que la información figure en el corpus indexado. La validación recogida en el Capítulo 12 muestra que el sistema responde correctamente al 91,9 % de un conjunto de 74 preguntas reales extraídas de los planes docentes oficiales, lo que excede el rango habitual de *exact-match* reportado para sistemas RAG de dominio cerrado en la literatura.

El tercero es el **modelo de licencia y despliegue**. Las soluciones comerciales identificadas (1MillionBot, UniBuddy, Hubert) son productos cerrados ofrecidos como servicio (SaaS). Linceus Assistant es un sistema desarrollado en el ámbito académico, abierto en su código y desplegable en la infraestructura de la propia universidad mediante Docker Compose. Esta diferencia tiene consecuencias en términos de control sobre los datos, ausencia de coste de licencia y posibilidad de adaptación funcional sin dependencia de proveedor.

El cuarto es la **validación cuantitativa**. Las cifras publicadas por las soluciones comerciales españolas suelen consistir en una tasa de acierto agregada (entre el 91 % y el 98 %), sin desagregar por tipo de consulta ni acompañar de la metodología empleada para el cómputo. Linceus Assistant aporta un esquema de validación en cinco capas independientes (E2E, RAG *baseline*, RAG de profundidad, NLU y *policy*, tráfico real post-despliegue) cuya metodología y resultados se documentan exhaustivamente en el Capítulo 12, con trazabilidad completa frente a umbrales académicos publicados.

En conjunto, el panorama revisado muestra que existe espacio para una propuesta como la presente: orientada al estudiante en su día a día académico (tanto el ya matriculado como el de nuevo ingreso una vez comienza el curso), abierta en su tecnología, basada en RAG sobre documentación oficial real, con un panel de administración que permite mantener el corpus al día sin intervención técnica y validada con un esquema cuantitativo reproducible.

1.5– Estructura de la memoria

La memoria se organiza en cuatro partes que siguen el ciclo de vida del proyecto, desde el planteamiento hasta la validación y el cierre.

Parte I: Introducción y Conceptos. Contextualiza el proyecto, justifica su necesidad frente a las soluciones existentes, fija los objetivos, presenta el estudio comparativo de tecnologías que sustenta las decisiones de diseño y define los conceptos técnicos fundamentales sobre sistemas conversacionales, NLU y arquitectura RAG.

Parte II: Metodología y Planificación. Describe la metodología de desarrollo empleada

(SCRUM adaptado a un proyecto individual), las herramientas de gestión utilizadas y la planificación temporal del trabajo, con la estructura de descomposición en fases, paquetes y sprints.

Parte III: Diseño e Implementación. Recoge la especificación de requisitos, la arquitectura del sistema, el marco tecnológico definitivo, la estructura del proyecto y de la base de datos, y el detalle de implementación de cada épica funcional con sus pipelines de ejecución.

Parte IV: Validación y Cierre. Presenta la estrategia de pruebas en cinco capas y los resultados obtenidos, incluye el manual de usuario y administrador, las conclusiones y líneas de trabajo futuro, y cierra con los anexos sobre uso de IA durante el desarrollo y vídeo de presentación.

Preparación del proyecto

En esta fase previa se llevó a cabo un análisis exhaustivo de las tecnologías disponibles para construir el sistema conversacional, evaluando alternativas en función de criterios como coste, facilidad de integración, capacidad de despliegue local y compatibilidad con técnicas de RAG. Asimismo, se definió la arquitectura técnica del sistema y se identificaron los casos de uso principales que guiarían el desarrollo del proyecto.

2.1– Estudio de las tecnologías y toma de decisiones

2.1.1. Decisión de frameworks conversacionales

Frameworks analizados

Se han considerado distintos frameworks de chatbot **open source** que cumplen con los requisitos de despliegue local, integración en Python y soporte para RAG. Los principales son:

- **Rasa** [25], centrado en Python y con gran control de flujo.
- **Botpress (OSS)** [26], que destaca por su editor visual de flujos.
- **DeepPavlov** [27], más orientado a NLP avanzado con módulos reutilizables.
- **OpenDialog**, centrado en diseño visual y contextos conversacionales.
- **Tock** y **Botonic**, opciones alternativas con características interesantes, aunque con menor adopción o diferente enfoque tecnológico.

Comparación detallada

Flujo conversacional El flujo conversacional se refiere a la capacidad del framework para **definir, controlar y gestionar el diálogo entre el usuario y el sistema**. Incluye cómo se modelan las intenciones, las respuestas, los contextos y la memoria de la conversación, así como el grado de flexibilidad para manejar **diálogos no lineales, interrupciones o escenarios complejos**. Un buen manejo del flujo conversacional permite que el chatbot no solo siga guiones rígidos, sino que pueda adaptarse dinámicamente a las necesidades del usuario, manteniendo coherencia y contexto a lo largo de la interacción.

- **Rasa** ofrece máximo control mediante historias y reglas, permitiendo modelar diálogos complejos y contextuales.
- **Botpress** facilita el diseño con nodos visuales, aunque está más orientado a flujos rígidos y predefinidos.

- **DeepPavlov** permite construir agentes multi-skill, pero requiere definir manualmente los pipelines.
- **OpenDialog** sobresale en el modelado de escenarios y contextos, aunque su dependencia de PHP limita la integración con Python.
- **Tock/Botonic** permiten definir historias o rutas conversacionales, pero con menor madurez que Rasa.

Testing La capacidad de testing en un framework de chatbots hace referencia a las **herramientas disponibles para validar la calidad y el correcto funcionamiento del agente conversacional**. Esto incluye desde pruebas unitarias (validación de intenciones, entidades y respuestas individuales) hasta pruebas end-to-end (que simulan conversaciones completas para comprobar flujos y contextos). Un buen soporte de testing permite **automatizar la detección de errores, reducir el riesgo de alucinaciones o respuestas incoherentes y asegurar la estabilidad** del sistema a medida que evoluciona. Cuanto más integrado esté el testing en el framework, más fácil será mantener y escalar el chatbot con garantías de calidad.

- **Rasa** incorpora herramientas para pruebas unitarias y end-to-end, automatizando la validación de intenciones y flujos.
- **Botpress** ofrece principalmente un simulador manual, sin suite de pruebas automatizadas robusta.
- **DeepPavlov** no incluye framework específico, por lo que las pruebas deben hacerse con scripts en Python.
- **OpenDialog** y **Botonic** dependen casi exclusivamente de pruebas manuales.

Compatibilidad con RAG y embeddings La compatibilidad con **RAG (Retrieval Augmented Generation)** y el uso de **embeddings** mide hasta qué punto un framework permite integrar modelos de lenguaje con bases de datos vectoriales para mejorar las respuestas. Esta característica es clave cuando se trabaja con **información documental extensa o cambiante**. Un buen soporte en este ámbito asegura que el chatbot pueda **ofrecer respuestas actualizadas, precisas y fundamentadas en datos externos**, reduciendo el riesgo de respuestas inventadas (alucinaciones).

- **Rasa** se integra de forma natural con bases vectoriales (pgvector, Qdrant, FAISS), permitiendo retrieval augmented generation.
- **Botpress** dispone de un módulo de *Knowledge Base* que admite cargar documentos, aunque menos flexible en personalización.
- **DeepPavlov** destaca en Q&A y FAQ matching, aunque el RAG debe construirse manualmente.
- **OpenDialog** y **Botonic** no tienen soporte nativo, siendo necesaria integración externa.

Despliegue local y privacidad El despliegue local y la gestión de la privacidad se refieren a la **posibilidad de ejecutar el framework en entornos controlados (on-premise o en servidores propios)** sin depender necesariamente de servicios en la nube. Esta característica es importante cuando se manejan **datos sensibles o confidenciales**, ya que permite aplicar políticas de seguridad personalizadas y cumplir con normativas como el RGPD. Además, la facilidad de despliegue (con Docker, instaladores nativos o stacks ligeros) influye directamente en la **rapidez de la puesta en marcha y en la capacidad de mantener el control total sobre la información del usuario**.

- **Rasa** y **Botpress** son fácilmente desplegables en local mediante Docker o instalación directa.
- **DeepPavlov** funciona como librería Python o servicio REST.
- **OpenDialog** requiere un stack PHP más pesado.
- **Tock** es estable en entornos locales, mientras que Botonic se orienta a proyectos en Node/-React.

Comunidad y documentación La comunidad y la documentación de un framework son indicadores clave de su madurez, accesibilidad y sostenibilidad a largo plazo. Una comunidad activa ofrece foros, repositorios y soporte colaborativo que facilitan la resolución de problemas y el intercambio de buenas prácticas. Al mismo tiempo, una documentación clara, actualizada y completa reduce la curva de aprendizaje y permite implementar soluciones más rápido.

- **Rasa** tiene la comunidad más amplia y documentación extensa.
- **Botpress** dispone de foros y Discord activos, pero con menor alcance.
- **DeepPavlov** cuenta con una comunidad académica sólida, aunque más técnica.
- **OpenDialog** y **Botonic** tienen comunidades pequeñas y más limitadas.

Decisión final

El framework seleccionado es **Rasa** [28], ya que combina:

- Máximo control sobre el flujo conversacional.
- Capacidades de testing integradas y automatizables [29].
- Soporte probado para RAG y bases vectoriales [30].
- Despliegue local sencillo y seguro, compatible con RGPD.
- Amplia comunidad y documentación, lo que garantiza sostenibilidad del proyecto.

Tabla de comparación detallada de frameworks

Característica / Framework	Rasa	Botpress	DeepPavlov	OpenDialog	Tock	Botonic
Flujo conversacional	***	**	**	**	*	*
Testing	***	*	*	*	*	*
Compatibilidad con RAG y embeddings	***	**	**	*	*	*
Despliegue local y privacidad	***	***	**	*	**	**
Comunidad y documentación	***	**	**	*	*	*
Resultado global	* 15	* 10	* 9	* 6	* 6	* 6

Cuadro 2.1: Comparación de frameworks conversacionales

2.1.2. Decisión de base de datos

Bases de Datos analizadas

Se revisaron diferentes opciones open source para almacenar tanto información estructurada como embeddings:

- **Supabase (Postgres + pgvector)** [31, 32].
- **PostgreSQL autogestionado con pgvector** [32].
- **Appwrite, PocketBase, Directus, Hasura.**
- Motores vectoriales dedicados: **Qdrant** [33], **Weaviate** [34], **Milvus, FAISS.**

Comparación detallada

Modelo de datos El modelo de datos define cómo se organiza, almacena y consulta la información dentro de una base de datos o motor de persistencia. Dependiendo de la tecnología, este modelo puede ser relacional (tablas y relaciones SQL), orientado a documentos (estructuras tipo JSON), o especializado en vectores (espacios multidimensionales para embeddings). La elección del modelo es clave para garantizar la eficiencia, flexibilidad y escalabilidad del sistema, ya que influye directamente en cómo se estructuran los datos de los usuarios, los recursos académicos y los embeddings utilizados en búsquedas semánticas o RAG.

Búsqueda vectorial

- **Supabase/Postgres** soportan pgvector, adecuado para corpus medianos.
- **Qdrant, Weaviate y Milvus** son más eficientes en grandes volúmenes de vectores.
- **FAISS** funciona muy bien embebido en Python, pero no es una base de datos como tal.
- El resto (Appwrite, PocketBase, Directus, Hasura) carecen de soporte nativo y requieren integraciones manuales.

Despliegue local y facilidad El despliegue local y la facilidad de instalación miden la **complejidad técnica y los recursos necesarios para poner en marcha una base de datos en un entorno controlado**. Algunas soluciones ofrecen entornos ligeros y rápidos de ejecutar (como binarios únicos o imágenes Docker sencillas), mientras que otras requieren stacks más pesados y múltiples dependencias. Esta característica es importante porque impacta directamente en la **rapidez de la configuración inicial, la curva de aprendizaje y los costes de mantenimiento**.

- **Supabase y Postgres** se despliegan fácilmente con Docker.
- **PocketBase** es el más liviano (un solo binario).
- **Appwrite y Directus** requieren más dependencias.
- **Qdrant/Weaviate/Milvus** añaden complejidad, aunque escalables.

Seguridad y RGPD La seguridad y el cumplimiento normativo son aspectos fundamentales en la gestión de datos, especialmente cuando se trabaja con información sensible de estudiantes o usuarios. Esta característica evalúa los **mecanismos nativos de autenticación, autorización y control de acceso** que ofrece cada base de datos, así como la posibilidad de aplicar políticas de privacidad. Tecnologías que incluyen funciones como **Row-Level Security, roles definidos o autenticación integrada** facilitan el cumplimiento legal y reducen riesgos. En cambio, los motores que carecen de seguridad propia obligan a implementar medidas adicionales en la capa de aplicación, aumentando la complejidad y la responsabilidad del equipo de desarrollo.

- **Supabase** ofrece Row-Level Security (RLS) y autenticación JWT.
- **Postgres** tradicional depende de roles SQL.
- **Appwrite/Directus/Hasura** ofrecen mecanismos de auth integrados.
- **Motores vectoriales** carecen de autenticación propia, requiriendo protección adicional en la aplicación.

Comunidad y documentación La comunidad y la documentación determinan el **nivel de soporte, recursos y acompañamiento disponibles para desarrolladores**. Una base de datos con una comunidad amplia y activa facilita la resolución de problemas, la existencia de librerías y la mejora continua del sistema. Al mismo tiempo, una documentación clara, actualizada y con ejemplos prácticos acelera la adopción de la tecnología y reduce la curva de aprendizaje.

- **Postgres** es el más maduro y con mayor soporte.
- **Supabase** ha crecido mucho, con amplia documentación y ejemplos de IA.
- **Qdrant y Weaviate** cuentan con comunidades activas en el ámbito de vectores.
- **Appwrite, PocketBase, Directus** tienen comunidades menores pero en expansión.

Decisión final

La base de datos seleccionada es **Supabase (Postgres + pgvector)** [31, 32], porque:

- Permite unificar datos estructurados y vectores en una sola plataforma [30].
- Ofrece autenticación, reglas de seguridad a nivel de fila y APIs listas para consumir desde Python y React.
- Facilita el despliegue local sin costes de licencias.
- Su comunidad y documentación reducen la curva de aprendizaje [31].
- Cumple los requisitos de RAG con un volumen de datos manejable para el caso académico.

Tabla de comparación de bases de datos

Característica	Supabase	Postgres	Appwrite	PocketBase	Directus	Hasura	Qdrant	Weaviate	Milvus	FAISS
Modelo de datos	***	***	**	**	**	**	**	**	**	**
Búsqueda vectorial	**	**	*	*	*	*	***	***	***	**
Despliegue local	***	***	**	***	**	**	**	**	**	**
Seguridad y RGPD	***	**	**	*	**	**	*	*	*	*
Comunidad y docs.	***	***	**	**	**	**	**	**	**	**
Total	14	13	9	9	9	9	10	10	10	9

Cuadro 2.2: Comparación de bases de datos

2.1.3. Decisión de API de LLM

APIs analizadas

Se han considerado diferentes opciones de APIs de modelos de lenguaje de gran tamaño (LLMs) open source o con planes gratuitos/educativos:

- **Gemini (Google AI / DeepMind)** [35]
- **OpenAI GPT (GPT-4o, GPT-4o mini)**
- **DeepSeek**
- **Hugging Face Inference API**
- **Cohere y Anthropic (Claude)**

Comparación detallada

Coste y accesibilidad El coste y la accesibilidad hacen referencia a la **disponibilidad económica y facilidad de uso de los modelos de lenguaje (LLMs)**. En un contexto académico, donde no suele existir presupuesto, es esencial contar con opciones que permitan **experimentar gratuitamente o con bajos costes iniciales**. Además del precio, influyen factores como los límites de uso en los planes gratuitos, la velocidad de respuesta, la estabilidad del servicio y la facilidad de registro o integración. Esta característica resulta clave para asegurar que el proyecto pueda desarrollarse de forma **sostenible y realista**, sin que los costes de las llamadas a la API se conviertan en una barrera.

- Gemini ofrece un plan gratuito con un número considerable de llamadas mensuales, lo que permite experimentar y probar sin coste, factor clave en un TFG sin presupuesto.
- OpenAI GPT tiene un coste asociado desde la primera llamada más allá de GPT-3.5, lo que lo hace menos viable económicamente.
- DeepSeek tiene una API gratuita, pero con menos garantías de estabilidad y soporte.
- Hugging Face API ofrece acceso a modelos open source, pero el plan gratuito tiene fuertes limitaciones de velocidad y peticiones.
- Claude (Anthropic) y Cohere requieren suscripción desde el inicio, menos atractivas en un contexto universitario.

Calidad y rendimiento La calidad y el rendimiento de un modelo de lenguaje se refieren a su capacidad para **comprender instrucciones, generar texto coherente y relevante, y mantener el contexto a lo largo de la conversación**. Estos aspectos suelen evaluarse mediante **pruebas comparativas de comprensión, razonamiento y generación en varios idiomas**. Además, influyen factores prácticos como la **velocidad de respuesta, la estabilidad del modelo y su habilidad para manejar entradas extensas sin pérdida de coherencia**. Una mayor calidad en este ámbito se traduce en **respuestas más útiles, precisas y naturales**, lo que resulta fundamental para un chatbot académico que debe manejar información variada con fiabilidad.

Estudios comparativos muestran que Gemini compite con GPT-4 en comprensión y generación de texto, especialmente en multilingüe y razonamiento.

- GPT sigue siendo referencia en benchmarks de calidad, pero con coste.

- DeepSeek ha demostrado buen rendimiento en ciertas tareas de razonamiento matemático, aunque en comprensión semántica y contexto extenso suele quedar por debajo de Gemini y GPT.
- Hugging Face API depende mucho del modelo seleccionado (ej. LLaMA, Mistral, Falcon), pero requieren más trabajo de fine-tuning y optimización para igualar a Gemini/GPT en calidad.

Integración y soporte técnico La integración y el soporte técnico se refieren a la **facilidad con la que un LLM puede incorporarse en el stack tecnológico existente** y al nivel de recursos disponibles para resolver problemas durante el desarrollo. Factores como la existencia de **SDKs oficiales, librerías en distintos lenguajes, ejemplos prácticos y documentación clara** determinan la rapidez de adopción. Además, contar con un ecosistema sólido y acuerdos institucionales (como convenios académicos) puede simplificar la integración y asegurar **mayor estabilidad a largo plazo**. Esta característica es clave, ya que permite centrar el esfuerzo en el desarrollo del chatbot sin tener que invertir demasiado tiempo en resolver barreras técnicas de conexión con el modelo.

- Gemini ofrece SDKs y buena integración en Python/JavaScript, lo que encaja perfectamente con tu stack (Python + React).
- GPT también tiene SDKs muy maduros, aunque con la barrera de costes.
- DeepSeek todavía no cuenta con ecosistema sólido.
- Hugging Face API es flexible pero requiere seleccionar y mantener modelos específicos.
- Gemini además está alineado con acuerdos académicos, como el que mantiene la **Universidad de Sevilla con Google** en diferentes ámbitos, lo que puede facilitar futuras integraciones oficiales del chatbot.

Privacidad y cumplimiento (RGPD) La privacidad y el cumplimiento normativo son esenciales al utilizar modelos de lenguaje que procesan datos de usuarios. Esta característica evalúa el **nivel de control sobre dónde y cómo se almacenan los datos**, si el servicio cumple con estándares legales de protección de información y si existe la opción de **despliegue on-premise**. Los LLMs en la nube suelen garantizar cumplimiento básico de GDPR, pero limitan el control directo. En cambio, los modelos auto-hospedados ofrecen **mayor soberanía sobre los datos**, aunque requieren más infraestructura y experiencia técnica. En un contexto universitario, suele ser más relevante encontrar un equilibrio entre **facilidad de uso, coste cero o reducido y garantías mínimas de privacidad**.

- Gemini y GPT ofrecen despliegues en la nube con cumplimiento GDPR, aunque el control on-premise es limitado.
- Los modelos de Hugging Face pueden auto-hospedarse, lo que da mayor control, pero a costa de infraestructura.
- En un contexto académico, la facilidad de uso y el plan gratuito de Gemini pesan más que el control absoluto.

Decisión final

La API seleccionada es **Gemini** [35], porque:

- Dispone de un plan gratuito generoso en número de llamadas.

- Ofrece calidad comparable a GPT-4 en muchos benchmarks, especialmente en español y razonamiento.
- Tiene SDKs oficiales para Python y JavaScript, encajando con el stack técnico del proyecto.
- Permite asegurar continuidad futura gracias a posibles acuerdos institucionales con la Universidad de Sevilla.
- Representa la opción más equilibrada entre calidad, coste, facilidad de integración y proyección académica.

Tabla de comparación detallada de APIs de LLM

Característica	Gemini	OpenAI GPT	DeepSeek	Hugging Face	Cohere	Claude
Coste y accesibilidad	***	*	**	**	*	*
Calidad y rendimiento	***	***	**	**	**	***
Integración y soporte	***	***	*	**	**	**
Privacidad y RGPD	**	**	*	***	**	**
Total	11	9	6	9	7	8

Cuadro 2.3: Comparación de APIs de LLM

2.2– Infraestructura del sistema

La infraestructura del sistema se apoya en tecnologías de código abierto y servicios con plan gratuito, lo que permite mantener el coste operativo en niveles compatibles con un proyecto académico. El sistema está actualmente en producción, desplegado en una instancia de DigitalOcean mediante Docker Compose, y se ha utilizado tanto para la fase de pruebas como para el piloto con alumnos reales.

El núcleo conversacional se ejecuta sobre el framework Rasa 3.6 y Python como lenguaje base, tanto para el servidor de diálogo como para las acciones personalizadas. La comprensión del lenguaje natural se reparte entre el clasificador local DIET, entrenado con los ejemplos del corpus propio del proyecto, y la API de Gemini (modelo `gemma-3-27b-it`), que asume las tareas en las que el clasificador local no es suficiente: clasificación de fallback, generación de SQL a partir de lenguaje natural, decisión RAG/SQL y redacción final de respuestas. La clave de la API se obtiene a través de Google AI Studio, dentro del plan gratuito.

La capa de datos reside en Supabase, una plataforma gestionada que ofrece PostgreSQL con la extensión `pgvector` activada de serie. Esta combinación permite mantener en la misma base de datos la información estructurada del dominio académico (asignaturas, horarios, profesorado, titulaciones) y los *embeddings* de los planes docentes utilizados por el pipeline RAG, sin necesidad de un motor vectorial independiente.

La interfaz del usuario final se entrega como un *widget* web ligero, implementado directamente en HTML, CSS y JavaScript sin necesidad de *frameworks* de *front-end*, lo que minimiza el peso del recurso y simplifica la integración en cualquier portal institucional. Junto al *widget* se despliega un panel de administración independiente, construido en Flask, que conecta directamente con la base de datos para permitir al personal autorizado mantener actualizado el corpus sin intervención técnica. Todo el conjunto se sirve a través de un proxy inverso `nginx` que actúa como punto de entrada único.

2.2.1. Arquitectura del sistema

La arquitectura sigue un enfoque modular y se descompone en cuatro contenedores Docker orquestados con Docker Compose: `nginx` como proxy inverso y servidor de los activos estáticos, `rasa` con el motor conversacional, `actions` con el servidor de acciones personalizadas, y `admin` con el panel Flask. Esta separación facilita el escalado independiente de cada pieza y la sustitución de componentes sin tocar el resto del sistema.

El flujo de una consulta tipo es el siguiente. El mensaje del usuario llega al servidor Rasa, que invoca el pipeline NLU para clasificar el *intent* y extraer las entidades relevantes. Si la confianza del clasificador local supera el umbral, el flujo continúa por la *custom action* correspondiente; en caso contrario se activa el fallback inteligente, que delega en Gemini la decisión sobre qué acción ejecutar. La acción seleccionada lee y escribe en los *slots* del *tracker* para mantener el contexto, y según la naturaleza de la consulta accede a la base relacional (datos estructurales como créditos, horarios o despachos) o al pipeline RAG sobre `pgvector` (preguntas sobre el contenido de los planes docentes: evaluación, temario, bibliografía). Las respuestas finales se construyen con una llamada a Gemini que recibe los datos recuperados como contexto y los redacta en lenguaje natural respetando un límite de longitud configurable.

2.2.2. Componentes tecnológicos del sistema

La Tabla 2.4 resume el papel de cada tecnología dentro del sistema en producción.

Tecnología	Rol en el sistema
Rasa 3.6	Motor conversacional: pipeline NLU, gestor de diálogo y servidor de acciones.
Python	Lenguaje del servidor de acciones personalizadas y del panel de administración.
Gemini API (<code>gemma-3-27b-it</code>)	Clasificación en fallback, <i>text-to-SQL</i> , decisión RAG/SQL y redacción de respuestas.
Supabase	PostgreSQL gestionado con autenticación y APIs listas para consumir desde Python.
pgvector	Extensión de PostgreSQL para almacenar y consultar <i>embeddings</i> en el pipeline RAG.
HTML, CSS, JavaScript	<i>Widget</i> web del usuario final, sin <i>frameworks</i> de <i>front-end</i> .
Flask	Panel de administración para el mantenimiento del corpus.
nginx	Proxy inverso y servidor estático del <i>widget</i> y del panel admin.
Docker Compose	Orquestación local y en producción de los cuatro contenedores del sistema.
DigitalOcean	Proveedor cloud donde se aloja la instancia de producción.
Google AI Studio	Gestión de la clave de acceso a la API de Gemini.

Cuadro 2.4: Componentes tecnológicos del sistema en producción.

2.3– Casos de uso principales

A partir del análisis del problema y de las decisiones tecnológicas de las secciones anteriores, se identifican los casos de uso que el sistema debe cubrir. Se agrupan en cuatro dominios funcionales: gestión de la titulación activa, información de asignaturas, horarios y profesorado. La gestión de la titulación se presenta en primer lugar porque es la condición previa que habilita el grueso de las consultas: las relativas a asignaturas y horarios solo devuelven resultados cuando la titulación ya se ha fijado, ya que dependen de ella para filtrar los datos. La excepción son las consultas de profesorado, que pueden resolverse sin titulación porque la información de contacto es transversal a las titulaciones de un mismo centro. Cada caso se describe con un escenario positivo (entrada

esperada) y un escenario negativo (entrada que el sistema debe gestionar con elegancia, no solo rechazar).

2.3.1. Gestión del contexto académico

Establecer o cambiar la titulación activa

La elección de titulación es el primer paso de la mayoría de las sesiones. Mientras el usuario no la fija, las consultas de asignaturas y horarios se interrumpen y el asistente solicita la selección mediante botones; las consultas de profesorado, en cambio, se atienden con normalidad. Una vez establecida, la titulación queda fijada como contexto y el usuario puede cambiarla en cualquier momento mediante una nueva orden.

- Escenario positivo: el usuario indica *“Soy de Ingeniería del Software”* o, ya en sesión, *“Cambiar a Tecnologías Informáticas”*; el asistente reconoce la titulación, fija el contexto y confirma el cambio. A partir de ese momento, las consultas que dependen de la titulación se filtran automáticamente por ella.
- Escenario negativo: el usuario propone una titulación ambigua o inexistente (*“ingeniería”* a secas) y el asistente solicita una aclaración antes de cambiar el contexto, sin asumir una titulación por defecto.

Consultar y reiniciar el contexto

- Escenario positivo: el usuario pregunta *“¿Qué titulación tengo activa?”* y el asistente responde con el centro y la titulación seleccionada. Ante *“Reset”* o *“Empezar de cero”*, el asistente borra el contexto y vuelve a mostrar los botones de selección de titulación.
- Escenario negativo: el usuario intenta consultar información sobre asignaturas u horarios sin haber elegido titulación y el asistente le solicita que elija una mediante los botones disponibles.

Consultar las titulaciones disponibles

- Escenario positivo: el usuario pregunta *“¿Qué titulaciones hay disponibles?”* y el asistente devuelve la lista de titulaciones activas en la base de datos, agrupadas por centro y con el número de asignaturas de cada una.
- Escenario negativo: si la base de datos no tiene titulaciones cargadas, el asistente lo indica con claridad en lugar de devolver una lista vacía.

2.3.2. Consulta sobre información de asignaturas

Consultar datos estructurales de una asignatura

El asistente extrae de la pregunta el dato concreto que el usuario está pidiendo y responde solo con ese dato, evitando volcar todo el registro de la asignatura. La respuesta completa se reserva para las preguntas explícitas de tipo general (*“Información de Redes”* o *“Háblame de Bases de Datos”*).

- Escenario positivo: el usuario pregunta *“¿Cuántos créditos tiene Redes?”* y el asistente responde *“Redes tiene 6 créditos”*. Si la pregunta es genérica (*“Información de Redes”*), el asistente devuelve el conjunto de campos: créditos, curso, duración (anual o por cuatrimestre) y tipología (formación básica, obligatoria u optativa).

- Escenario negativo: el usuario consulta “*Información de Programación Interplanetaria*” (asignatura inexistente) y el asistente indica que no encuentra esa asignatura, ofreciendo, si procede, sugerencias por similitud.

Consultar contenido del plan docente

Este caso cubre las preguntas cuyo dato vive dentro del plan docente oficial de la asignatura. El sistema cubre, entre otros, sistema de evaluación y fórmula de calificación, metodología docente, temario y temas concretos, competencias formativas, bibliografía recomendada y, cuando el plan docente lo recoge explícitamente, datos sobre coordinación de la asignatura. La cobertura depende de lo que aparezca en cada plan docente: hay información que solo figura en algunos documentos y otra que no aparece en ninguno; en particular, las tutorías del profesorado no suelen estar recogidas como horario estructurado, por lo que se gestionan aparte (véase Sección 2.3.4).

- Escenario positivo: el usuario pregunta “*¿Qué bibliografía tiene Sistemas Operativos?*” o “*Temario de Bases de Datos*” y el asistente recupera los fragmentos relevantes del plan docente mediante el pipeline RAG y redacta una respuesta en lenguaje natural.
- Escenario negativo: el usuario pide información que el plan docente no contempla (“*¿Cuántos alumnos aprueban Sistemas Operativos cada año?*”) y el asistente responde que ese dato no consta en la documentación oficial, sin inventar la cifra.

Listado y conteo de asignaturas

- Escenario positivo: el usuario pide “*Dame las optativas de cuarto*” y el asistente genera la consulta SQL correspondiente, la ejecuta y devuelve la lista paginada. De forma análoga, ante “*¿Cuántas asignaturas tiene primero?*”, el sistema devuelve el conteo.
- Escenario negativo: el usuario pide un filtro inválido (“*optativas de séptimo curso*”) y el asistente aclara que ese curso no existe en la titulación activa.

2.3.3. Consulta sobre horarios

Consultar horario por asignatura

- Escenario positivo: el usuario pregunta “*¿A qué hora tengo Bases de Datos?*” y el asistente devuelve los horarios disponibles, distinguiendo aulas de teoría y de laboratorio.
- Escenario negativo: el usuario pide “*Horario de Bases de Datos 7*” (asignatura inexistente) y el asistente informa de que no encuentra coincidencias.

Consultar horario por curso y grupo

- Escenario positivo: el usuario solicita “*Horario de 2º grupo 1*” y obtiene la planificación semanal completa de ese curso y grupo. Los grupos se identifican por número (1, 2, 3...), tal como aparecen en los horarios oficiales de la ETSII.
- Escenario negativo: el usuario introduce un grupo inexistente (“*9º grupo 1*” o “*2º grupo 99*”) y el asistente aclara que esa combinación no figura en la base de datos.

Consultar horario por día

- Escenario positivo: el usuario pregunta “*¿Qué clases tengo el lunes en primero grupo 2?*” y el asistente devuelve solo las sesiones del día indicado.
- Escenario negativo: el usuario pregunta por un día sin clases (sábado o domingo) y el asistente lo indica explícitamente en lugar de devolver una lista vacía.

2.3.4. Consulta sobre datos de contacto del profesorado

Consultar despacho, correo y categoría académica

- Escenario positivo: el usuario pregunta “¿Cuál es el despacho del profesor Parejo?” o “Correo de Ana Bernárdez” y el asistente devuelve la información correspondiente: despacho, correo institucional, departamento y categoría académica.
- Escenario negativo: el usuario introduce un nombre inexistente y el asistente indica que no lo encuentra; si la similitud con algún profesor real es alta, ofrece la sugerencia para que el usuario la confirme.

Consultar tutorías de un profesor

- Escenario positivo: el usuario pregunta “Tutorías de la profesora Bernárdez”. Los horarios estructurados de tutoría, en general, no se publican de forma estable en ningún portal oficial ni aparecen recogidos en los planes docentes, por lo que el asistente redirige al correo institucional del profesor para que el alumno pueda solicitar cita directamente.
- Escenario negativo: el usuario pide tutorías de un profesor inexistente y el asistente indica que no lo encuentra.

Búsqueda de profesores por asignatura o departamento

- Escenario positivo: el usuario pregunta “¿Quién da Fundamentos de Programación?” o “Profesores del Departamento de Lenguajes y Sistemas Informáticos” y el asistente devuelve la lista correspondiente con sus datos de contacto.
- Escenario negativo: el usuario pide profesorado de una asignatura o departamento que no figura en la base de datos y el asistente informa del error.

Conceptos iniciales

Este capítulo presenta la terminología fundamental sobre la que se construye el sistema. El objetivo no es reproducir la documentación oficial de cada tecnología, sino establecer un marco conceptual propio del proyecto, coherente con las decisiones de diseño adoptadas y con la forma en que cada componente aparece en el resto de la memoria.

3.1– Sistema conversacional

Un sistema conversacional es una aplicación software diseñada para interactuar con usuarios mediante lenguaje natural. El sistema interpreta cada entrada textual, mantiene el estado del diálogo a lo largo del tiempo y genera respuestas coherentes en función del contexto y de los objetivos definidos.

En el caso de Linceus Assistant, el sistema se especializa en consultas académicas del ámbito de la ETSII: horarios, información de asignaturas, datos de contacto del profesorado y normativa universitaria. Esta especialización condiciona tanto el diseño del modelo de lenguaje como la estructura de la base de datos.

3.2– Componentes del sistema conversacional con Rasa

El framework Rasa organiza el sistema en dos capas bien diferenciadas: la comprensión del lenguaje natural (NLU) y la gestión del diálogo. Cada capa opera con sus propios mecanismos y se configura en archivos separados.

3.2.1. Comprensión del lenguaje natural (NLU)

El módulo NLU procesa cada mensaje del usuario para extraer dos tipos de información: la intención que expresa y las entidades que contiene.

Intent Un intent representa el propósito o el objetivo del mensaje del usuario. Cada entrada se clasifica en una única intención que determina el tipo de acción que debe ejecutar el asistente. En el proyecto se definen intents específicos para cada dominio funcional: consulta de horarios, búsqueda de profesores, información sobre asignaturas o preguntas sobre normativa.

Entity Una entidad es un fragmento de información concreta extraído del mensaje que completa la intención detectada. Por ejemplo, en la frase “¿Cuándo es la clase de Bases de Datos del grupo 2?”, el intent es `consultar_horario`, la entidad `nombre_asignatura` toma el valor *Bases de Datos* y la entidad `grupo` toma el valor 2. Las entidades se definen en el archivo `domain.yml` y se anotan en los ejemplos de entrenamiento dentro de `data/nlu.yml`.

3.2.2. Gestión del diálogo

Una vez que el NLU ha identificado el intent y las entidades, el gestor de diálogo decide qué acción ejecutar a continuación. Este componente trabaja con tres elementos principales: slots, el tracker y las políticas de diálogo.

Slot Un slot es una variable de memoria que persiste durante toda la conversación. Mientras que una entidad se extrae del mensaje actual y se descarta al siguiente turno, un slot conserva el valor extraído para que esté disponible en acciones posteriores. En Rasa, los slots se declaran en `domain.yml` y se tipan según el dato que almacenan: texto, número, valor booleano o valor categórico.

Tracker El tracker es la estructura interna que almacena el estado completo de la conversación en cada instante. Contiene el historial de mensajes del usuario, los intents detectados, las entidades extraídas, los valores actuales de los slots y las acciones ya ejecutadas. Las custom actions acceden al tracker para leer el contexto de la conversación antes de generar una respuesta.

3.2.3. Mecanismos de respuesta: actions

Las acciones son las operaciones que ejecuta el asistente como respuesta a cada turno de conversación. Rasa distingue dos tipos:

- **Respuestas predefinidas (`utter_*`):** respuestas de texto estático declaradas en la sección `responses` del `domain.yml`. Se usan para saludos, despedidas y mensajes fijos.
- **Custom actions (`action_*`):** clases Python definidas en `actions/actions.py` que implementan lógica personalizada. En este proyecto se emplean para consultar la base de datos, invocar la API de Gemini y construir respuestas dinámicas.

3.2.4. Formularios

Un formulario es un mecanismo que permite recopilar varios datos del usuario de forma secuencial antes de ejecutar una acción. Si el usuario no proporciona todos los campos necesarios en un solo mensaje, el formulario los solicita uno a uno hasta completarlos. En el proyecto se utilizan formularios en aquellas consultas que requieren asignatura y grupo simultáneamente para devolver un resultado concreto.

3.2.5. Fallback

El fallback es el comportamiento de recuperación que se activa cuando el sistema no es capaz de clasificar el mensaje con confianza suficiente, o cuando no existe ninguna acción definida para la situación actual. En lugar de devolver un error o un silencio, el mecanismo de fallback responde al usuario con un mensaje de disculpa y, opcionalmente, le redirige a un recurso externo relacionado con el tema detectado.

3.3– Archivos de configuración de Rasa

La configuración del sistema se distribuye en varios archivos, cada uno con una responsabilidad bien definida. La tabla 3.1 resume su función dentro del proyecto.

La distinción entre `stories.yml` y `rules.yml` es relevante para el diseño: las stories son ejemplos de los que el modelo aprende a generalizar, mientras que las rules son restricciones absolutas. Para el proyecto, los saludos, las despedidas y los fallbacks se gestionan mediante rules; los flujos que dependen del contexto acumulado en la conversación se modelan con stories.

Archivo	Función
domain.yml	Definición central del asistente: intents, entities, slots, respuestas y lista de acciones.
data/nlu.yml	Ejemplos de frases anotadas para entrenar el modelo NLU.
data/stories.yml	Flujos de conversación de varios turnos usados como ejemplos de entrenamiento.
data/rules.yml	Comportamientos deterministas que se aplican siempre, independientemente del contexto.
config.yml	Pipeline de componentes NLU y políticas de diálogo.
endpoints.yml	URL del servidor de acciones y configuración del tracker store.
actions/actions.py	Implementación Python de las custom actions.

Cuadro 3.1: Archivos de configuración principales del sistema Rasa

3.4– Flujo de procesamiento de un mensaje

Cuando el usuario envía un mensaje al asistente, el sistema lo procesa en tres etapas consecutivas:

1. **NLU:** el texto se tokeniza, se vectoriza y se clasifica para obtener el intent y las entidades. En el proyecto, esta etapa combina el clasificador DIET de Rasa con una llamada opcional a la API de Gemini para los casos en que la confianza del clasificador local no supera el umbral definido.
2. **Gestión del diálogo:** el gestor consulta las rules y las stories para decidir la siguiente acción. El tracker se actualiza con el nuevo mensaje y los valores de los slots modificados.
3. **Ejecución de la acción:** si la acción es una respuesta predefinida, se envía directamente. Si es una custom action, se ejecuta el código Python correspondiente, que puede consultar la base de datos, llamar a la API de Gemini o construir una respuesta RAG.

3.5– Generación aumentada por recuperación (RAG)

La generación aumentada por recuperación (RAG, del inglés *Retrieval Augmented Generation*) es una técnica que combina la búsqueda de información en una base de conocimiento con la generación de texto mediante un modelo de lenguaje [30]. El objetivo es reducir las alucinaciones propias de los modelos generativos proporcionándoles contexto factual relevante antes de generar la respuesta.

En Linceus Assistant, el pipeline RAG opera en tres pasos. Primero, la consulta del usuario se transforma en un vector de alta dimensión mediante un modelo de embeddings. Segundo, ese vector se compara con los fragmentos almacenados en la base de datos vectorial para recuperar los más similares semánticamente. Tercero, los fragmentos recuperados se incorporan al prompt que se envía a Gemini, que genera la respuesta final fundamentada en esos contenidos.

3.6– Backend y persistencia de datos

3.6.1. Supabase y PostgreSQL

Supabase es una plataforma de backend basada en PostgreSQL que ofrece base de datos relacional, autenticación, API REST y soporte para la extensión pgvector [31]. En el proyecto, Supabase centraliza dos tipos de información: los datos estructurados del dominio académico (asignaturas, profesores, horarios, normativa) y los vectores semánticos del sistema RAG.

3.6.2. pgvector

pgvector es una extensión de PostgreSQL que permite almacenar vectores numéricos como columnas de una tabla relacional y realizar búsquedas de similitud sobre ellos de forma eficiente [32]. Su ventaja en este proyecto es que evita la necesidad de mantener un motor vectorial independiente: los embeddings y los datos estructurados conviven en la misma base de datos, simplificando las consultas y el despliegue.

3.6.3. Embeddings

Un embedding es una representación numérica de un texto en un espacio vectorial de alta dimensión. Textos semánticamente similares producen vectores próximos según alguna métrica de distancia, como el coseno o la distancia euclídea. En el pipeline RAG del proyecto, los fragmentos de documentación académica se convierten en embeddings al ingestar los datos, y las consultas del usuario se transforman en embeddings en tiempo de ejecución para recuperar los fragmentos más relevantes.

Parte II

Metodología y Planificación

Metodología

Este capítulo describe cómo se ha organizado y ejecutado el desarrollo de Linceus Assistant. Se cubren cinco aspectos: la metodología de desarrollo ágil empleada y su adaptación al contexto del proyecto, la gestión del código fuente, los estándares de codificación aplicados, la gestión del trabajo y las tareas, y el esquema de versionado del sistema.

4.1– Metodología de desarrollo: Scrum adaptado

4.1.1. Scrum: fundamentos

Scrum es un marco de trabajo ágil para el desarrollo iterativo e incremental de productos complejos [36]. Su unidad de trabajo es el **sprint**, una iteración de duración fija (típicamente de una a cuatro semanas) al final de la cual se obtiene un incremento del producto potencialmente entregable. Los roles clásicos son el *Product Owner*, responsable de priorizar el *product backlog*; el *Scrum Master*, que facilita el proceso; y el *Development Team*, que ejecuta el trabajo. Las ceremonias principales son la *Sprint Planning* (planificación del sprint), la *Daily Scrum* (sincronización diaria), la *Sprint Review* (revisión del incremento) y la *Sprint Retrospective* (mejora del proceso).

Una de las características definitorias de Scrum es su flexibilidad: la guía oficial no prescribe prácticas de ingeniería específicas, sino un conjunto mínimo de reglas que el equipo adapta a su contexto. Esto lo hace especialmente adecuado para proyectos de alcance variable como un TFG, donde los requisitos evolucionan con el conocimiento adquirido durante el desarrollo.

4.1.2. Adaptación al proyecto individual

Linceus Assistant es un proyecto desarrollado por una sola persona, lo que hace innecesarias varias ceremonias pensadas para coordinar equipos. A continuación se describen las adaptaciones realizadas respecto al Scrum estándar:

- **Sin Daily Scrum.** La sincronización diaria carece de sentido en un equipo unipersonal. El seguimiento del progreso se realiza de forma continua a través del tablero de tareas y el registro de decisiones técnicas en `docs/sprints/`.
- **Retrospectivas bajo demanda.** No se celebra una retrospectiva formal al cierre de cada sprint. En su lugar, se realiza una revisión del proceso únicamente cuando se detecta un problema recurrente o una desviación significativa respecto a la planificación. Este enfoque evita la sobrecarga documental sin perder la capacidad de mejora continua.
- **Sprint Planning al final de la Review.** La planificación del sprint siguiente se integra en la sesión de revisión del sprint anterior. Una vez evaluado el incremento entregado y ajustado el backlog, se definen directamente los objetivos del próximo sprint. El flujo real es,

por tanto: *Review del sprint* → *Actualización del backlog* → *Definición de objetivos* → *Descomposición en tareas*.

- **Rol único con supervisión académica.** Al no haber equipo, los tres roles de Scrum (Product Owner, Scrum Master, Developer) los asume la misma persona. Las funciones propias del Product Owner —definir prioridades, validar el incremento y orientar el alcance del producto— se ejercen en colaboración con el tutor académico, cuyas aportaciones quedan recogidas en la Bitácora de Reuniones descrita en la Sección 4.4.

4.1.3. Sprints y duración

Los sprints tienen una duración de **3 a 4 semanas**, ajustada según la carga lectiva del cuatrimestre y la complejidad de la épica en curso. Cada sprint cierra con un incremento funcional verificable y un breve registro de decisiones técnicas (archivo `docs/sprints/sprint-N.md`) que documenta los cambios de diseño no triviales realizados durante la iteración.

La correspondencia entre sprints y épicas del backlog se detalla en el Capítulo 5.

4.2– Gestión del código fuente

El control de versiones del proyecto se gestiona con **Git**, y el repositorio remoto se aloja en **GitHub**. Esta combinación proporciona trazabilidad completa de los cambios, facilita la revisión del historial de decisiones técnicas y permite integrar el pipeline de despliegue continuo descrito en la Sección 5.2.3.

4.2.1. Política de ramas

El repositorio sigue una política de dos ramas principales:

- **main:** rama de producción. Solo recibe merges procedentes de `testing` una vez que el incremento ha superado las pruebas de aceptación y el sistema está listo para desplegarse. Todo merge a `main` dispara automáticamente el pipeline de GitHub Actions que despliega la nueva versión en el servidor de DigitalOcean.
- **testing:** rama de integración y validación. Recibe el trabajo de desarrollo, se usa para ejecutar las suites de pruebas y se mantiene como línea estable entre sprints.

El desarrollo de cada épica o tarea de cierta envergadura se realiza en una **rama de feature** con nombre descriptivo (por ejemplo, `feature/rag-pipeline` o `fix/horarios-slot`), que se integra en `testing` mediante un merge directo una vez finalizada y probada localmente.

4.2.2. Convenciones de commits

Los mensajes de commit siguen el estándar **Conventional Commits** [37], con el formato `<tipo>(<ámbito>) : <descripción>`. Los tipos empleados con mayor frecuencia son `feat` (nueva funcionalidad), `fix` (corrección de errores), `refactor` (reestructuración sin cambio de comportamiento), `test` (adición o modificación de pruebas) y `docs` (documentación). Los commits son frecuentes y de alcance reducido: cada commit representa un cambio coherente y compilable, lo que facilita la bisección del historial cuando aparecen regresiones.

4.3– Estándares de codificación

El código Python del proyecto se ajusta a la guía de estilo **PEP 8** [38], verificada automáticamente mediante dos herramientas complementarias: **pylint** [39] y **flake8** [40]. Pylint cubre el

análisis semántico —detección de errores reales, variables no definidas, llamadas incorrectas— mientras que `flake8` se centra en la conformidad de estilo: espacios sobrantes, imports no utilizados, longitud de línea e indentación. Ambas herramientas se configuran con un límite de 120 caracteres por línea, valor que equilibra legibilidad y practicidad para un proyecto con prompts al LLM y consultas SQL que no admiten partición arbitraria.

En los casos en que una desviación de estilo es intencionada —un import que actúa como re-exportación pública, una línea larga que contiene una cadena de texto no partible, o un par de instrucciones que forman una unidad semántica indivisible— se documenta explícitamente con una directiva de supresión en la propia línea. Esto hace que las excepciones sean visibles y razonadas, en lugar de silenciosas.

Para el código JavaScript del widget de chat (`frontend/`), se aplica **ESLint 8** [41] con la configuración `eslint:recommended`, extendida con reglas de estilo básicas (comillas simples, punto y coma obligatorio). La configuración se almacena en `.eslintrc.json` en la raíz del repositorio y se ejecuta con `npm run lint`, que lanza `eslint frontend/` sobre los tres ficheros del directorio (`chatbot-widget.js`, `admin.js`, `login.js`).

4.4– Gestión del trabajo

El seguimiento del proyecto se articula en torno a la **Bitácora de Reuniones**, un documento estructurado que recoge de forma acumulativa el desarrollo de cada reunión con el tutor y los objetivos acordados para el sprint siguiente. Tras cada reunión, la bitácora se actualiza a partir de la transcripción de la sesión, extrayendo los puntos más relevantes (decisiones tomadas, problemas detectados, cambios de prioridad) y registrando las tareas pendientes para el siguiente ciclo. Este documento actúa de facto como el *product backlog* del proyecto: la lista de tareas pendientes refleja en todo momento el estado acordado con el tutor, sin necesidad de herramientas adicionales de gestión.

Complementariamente, las decisiones técnicas no triviales tomadas durante el desarrollo se documentan en los registros de decisiones de sprint (`docs/sprints/SN/Registro_decisiones.md`), que mantienen trazabilidad entre el código y el razonamiento que lo originó.

El tiempo dedicado al proyecto se registra con **Clockify**, donde cada entrada recibe el nombre `SX -- descripción de la tarea` (siendo X el número de sprint), lo que permite agrupar el esfuerzo por sprint y correlacionarlo con el backlog sin necesidad de herramientas adicionales.

4.5– Versionado del sistema

El sistema sigue un esquema de versionado semántico de tres niveles: **vMAJOR.MINOR.PATCH**, adaptado al ciclo de vida de un chatbot con modelo entrenado.

- **MAJOR**: se incrementa en dos situaciones: al incorporar una **nueva épica funcional** (un nuevo dominio de capacidades), o al introducir un **cambio arquitectónico de envergadura dentro de una épica existente** que afecta a su lógica de forma global (por ejemplo, la migración completa a Text-to-SQL o la introducción del pipeline RAG). Ejemplos reales del proyecto: `v1.x.x` cubre asignaturas con NLU puro, `v2.x.x` introduce Text-to-SQL y RAG, `v3.x.x` añade horarios, `v4.x.x` añade profesorado, `v5.x.x` incorpora cambios transversales de infraestructura.
- **MINOR**: se incrementa al añadir **nueva funcionalidad dentro de una épica existente**, como nuevos intents, nuevos filtros de búsqueda o mejoras significativas en la lógica de recuperación, sin alterar la arquitectura general.
- **PATCH**: se incrementa en correcciones de errores, ajustes de prompts, mejoras de ejemplos NLU o pequeñas optimizaciones que no alteran el comportamiento funcional esperado.

El versionado está ligado al modelo Rasa: cada reentrenamiento que modifica los ficheros NLU, stories o reglas genera un nuevo artefacto de modelo (`models/linceus.vX.Y.Z.tar.gz`) identificado con la versión del sistema. Esto garantiza que cada versión desplegada en producción es reproducible: el par {tag de Git, artefacto de modelo} define unívocamente el estado del sistema en ese punto. El historial completo de versiones se mantiene en `docs/tabla_versiones.md`.

Planificación

Este capítulo presenta la planificación del proyecto Linceus Assistant. La organización del trabajo sigue el ciclo de vida clásico de un proyecto de ingeniería del software, dividido en **tres fases**: *Inicio*, *Desarrollo* y *Cierre*. Esta estructura es coherente con la guía PMBOK y con la práctica habitual en proyectos académicos, y permite separar el trabajo de planificación previa, el desarrollo iterativo del producto y la validación con usuarios y entrega final.

Dentro de cada fase se identifican **paquetes de trabajo** que agrupan tareas con un objetivo común y un entregable concreto. Las épicas de desarrollo en sentido SCRUM (asignaturas, profesores, horarios, etc.) se ubican en la fase de Desarrollo y se ejecutan de forma incremental sobre sprints de 3–4 semanas, según se detalla en el Capítulo 4.

5.1– Estructura de Descomposición del Trabajo (EDT)

La Estructura de Descomposición del Trabajo (EDT), o *Work Breakdown Structure* (WBS), organiza jerárquicamente todo el trabajo necesario para completar el proyecto. El nivel 0 representa el proyecto completo, el nivel 1 las tres fases del ciclo de vida y el nivel 2 los paquetes de trabajo dentro de cada fase. Las tareas individuales (nivel 3) se detallan en el diccionario EDT de la Sección 5.2.

La Figura 5.1 muestra la EDT a nivel de fases y paquetes de trabajo.

5.2– Diccionario EDT

El diccionario EDT define cada paquete de trabajo y sus tareas constituyentes, especificando descripción, entregables y estimación de esfuerzo. Las estimaciones siguen la escala: **S** (Small, ≤ 1 semana), **M** (Medium, 1–2 semanas) y **L** (Large, 2–3 semanas).

5.2.1. Fase 1 — Inicio

La fase de inicio agrupa todo el trabajo previo a la implementación: planificación, estudio del estado del arte, análisis de tecnologías y diseño arquitectónico. Es la fase más corta en duración pero condiciona todas las decisiones posteriores.

1.1 — Gestión del proyecto

Paquete transversal: aunque su esfuerzo principal se concentra en el inicio (planificación) y en el cierre (memoria, defensa), la gestión del proyecto está activa durante todo el ciclo de vida.

1.2 — Investigación tecnológica

Análisis del estado del arte y comparativa de tecnologías, base de las decisiones de diseño documentadas en el Capítulo 7.



Figura 5.1: Estructura de Descomposición del Trabajo (EDT) — fases y paquetes de trabajo.

ID	Tarea	Descripción y entregables	Est.
1.1.1	Anteproyecto y planificación	Definición del alcance, objetivos, EDT inicial y cronograma orientativo. <i>Entregable: anteproyecto aprobado.</i>	M
1.1.2	Seguimiento iterativo	Revisión del progreso al cierre de cada sprint, ajuste del backlog y gestión de riesgos. <i>Entregable: registro de decisiones (docs/sprints/).</i>	L
1.1.3	Comunicación con tutor	Reuniones periódicas de seguimiento, actas y validación de hitos. <i>Entregable: actas de seguimiento.</i>	M

Cuadro 5.1: Diccionario EDT — 1.1: Gestión del proyecto.

ID	Tarea	Descripción y entregables	Est.
1.2.1	Estado del arte de chatbots	Revisión de chatbots universitarios publicados (AdmitBot, EDUBOT/LiSA, Ranoliya FAQ, Dibya & Sahoo) y umbrales académicos de calidad. <i>Entregable: marco bibliográfico.</i>	M
1.2.2	Comparativa de frameworks	Análisis de Rasa, Botpress y DeepPavlov según coste, integración Python, RAG y despliegue local. <i>Entregable: tabla comparativa, decisión documentada.</i>	L
1.2.3	Comparativa de LLM/embeddings	Análisis de Gemini, OpenAI, DeepSeek y Ollama según coste, calidad y privacidad. <i>Entregable: tabla comparativa.</i>	M
1.2.4	Setup del entorno	Instalación de Python, Rasa, Supabase, Gemini API, Git/GitHub. Estructura inicial del repositorio. <i>Entregable: entorno funcional reproducible.</i>	S

Cuadro 5.2: Diccionario EDT — 1.2: Investigación tecnológica.

1.3 — Diseño de la arquitectura

Definición de la arquitectura por capas, modelo de datos y contratos entre componentes antes de empezar el desarrollo.

ID	Tarea	Descripción y entregables	Est.
1.3.1	Arquitectura por capas	Definición de las capas (presentación, NLU, lógica de negocio, IA, datos, admin) y de las interfaces entre ellas. <i>Entregable: diagrama de componentes.</i>	M
1.3.2	Diagrama de despliegue	Modelo de despliegue sobre DigitalOcean con contenedores Docker, Supabase como BD gestionada y Gemini como servicio externo. <i>Entregable: diagrama de despliegue.</i>	S
1.3.3	Modelo de datos	Diseño del esquema relacional (asignaturas, profesores, horarios, planes docentes) y vectorial (chunks + embeddings con pgvector). <i>Entregable: diagrama ER.</i>	M

Cuadro 5.3: Diccionario EDT — 1.3: Diseño de la arquitectura.

5.2.2. Fase 2 — Desarrollo

Fase central del proyecto. Se ejecuta de forma iterativa siguiendo el marco SCRUM adaptado descrito en el Capítulo 4, con sprints de 3–4 semanas. Cada paquete corresponde a una épica funcional del producto y se cierra con un incremento entregable.

2.1 — Épica Asignaturas

Funcionalidad nuclear: consultas sobre asignaturas (ficha, listado, conteo) mediante *Text-to-SQL* sobre la base de datos relacional y RAG sobre los planes docentes.

ID	Tarea	Descripción y entregables	Est.
2.1.1	Esquema y carga de datos	Tablas asignaturas, titulaciones, centros y carga inicial de las tres titulaciones GII de la ETSII. <i>Entregable: scripts SQL + datos.</i>	M
2.1.2	NLU de asignaturas	Intents (consulta_asignatura_especifica, consulta_asignaturas_listado, consulta_asignaturas_conteo) y entidades. <i>Entregable: ficheros NLU.</i>	M
2.1.3	Acción Text-to-SQL	Generación dinámica de SQL con Gemini, validación, ejecución y formateo de la respuesta natural. <i>Entregable: módulo actions/asignaturas/.</i>	L
2.1.4	Resolución de seguimiento	Herencia de slot ultimo_nombre_asignatura para preguntas elípticas (“y cuántos créditos?”). <i>Entregable: resolver_asignatura.</i>	M
2.1.5	Testing de la épica	Suite de aceptación con ≥ 30 casos golden + casos negativos y con typos. <i>Entregable: informe de pruebas.</i>	M

Cuadro 5.4: Diccionario EDT — 2.1: Épica Asignaturas.

2.2 — Épica Profesores

Búsqueda de profesorado por nombre o por asignatura con datos de contacto, despacho, departamento y atajo de coordinador/suplente vía RAG sobre el plan docente.

ID	Tarea	Descripción y entregables	Est.
2.2.1	Esquema profesores	Tablas profesores, departamentos, profesor_asignatura con FK hacia asignaturas. <i>Entregable: script SQL.</i>	S
2.2.2	Importación desde us.es PDI	Scraper del directorio oficial de la US para importar profesorado de la ETSII; deduplicación por email. <i>Entregable: scraper integrado en panel admin.</i>	L
2.2.3	NLU y acción	Intent consulta_profesor con pipeline Text-to-SQL + fallback RAG sobre plan docente. <i>Entregable: actions/profesores/.</i>	L
2.2.4	Testing de la épica	Suite con 43 casos cubriendo nombre, asignatura, tutorías y coordinador. <i>Entregable: informe de pruebas.</i>	M

Cuadro 5.5: Diccionario EDT — 2.2: Épica Profesores.

2.3 — Épica Horarios

Consultas de horario por curso/grupo o por asignatura, con extracción automática del PDF oficial de horarios de la ETSII.

ID	Tarea	Descripción y entregables	Est.
2.3.1	Esquema horarios	Tablas grupos_clase, horarios, aulas con cuatrimestre como dimensión. <i>Entregable: script SQL.</i>	M
2.3.2	Extractor PDF ETSII	Parser con pdfplumber de la cuadrícula oficial; reconoce códigos xGy-Cz y separa teoría de laboratorio. <i>Entregable: admin/horarios_extractores/.</i>	L
2.3.3	NLU y acciones	Intents consulta_horario (curso+grupo) y consulta_horario_asignatura; herencia de slot para seguimiento. <i>Entregable: actions/horarios/.</i>	L
2.3.4	Testing de la épica	Suite con 45 casos (horario semanal, por día, con/sin cuatrimestre, con typos). <i>Entregable: informe de pruebas.</i>	M

Cuadro 5.6: Diccionario EDT — 2.3: Épica Horarios.

2.4 — Pipeline RAG

Infraestructura transversal de *Retrieval Augmented Generation* sobre los planes docentes de las asignaturas, utilizada por las épicas de asignaturas y profesores.

2.5 — Panel admin y frontend

Interfaces visibles del producto: el widget de chat para el alumno y el panel de administración para mantener los datos curados.

5.2.3. Fase 3 — Cierre

Fase final del proyecto: validación funcional exhaustiva, despliegue en producción, ejecución del piloto con usuarios reales y entrega de la documentación académica.

ID	Tarea	Descripción y entregables	Est.
2.4.1	Extracción y <i>chunking</i>	Parseo de planes docentes (HTML/PDF), <i>chunking</i> semántico con metadatos (asignatura, sección, año). <i>Entregable: pipeline.</i>	M
2.4.2	Embeddings y pgvector	Generación con <code>text-embedding-004</code> y almacenamiento en pgvector con índices HNSW; función SQL <code>buscar_plan_docente()</code> . <i>Entregable: pipeline de vectorización.</i>	M
2.4.3	Action RAG	Acción que combina retrieval semántico con re-ranking por sección y umbral de similitud; respuesta con citas. <i>Entregable: módulo RAG en actions/.</i>	L
2.4.4	Anti-alucinación	Prompts reforzados que prohíben inventar nombres, emails o aulas no presentes en los chunks. <i>Entregable: prompts revisados.</i>	M

Cuadro 5.7: Diccionario EDT — 2.4: Pipeline RAG.

ID	Tarea	Descripción y entregables	Est.
2.5.1	Widget de chat	SPA en JavaScript vanilla, embebible mediante <code><script></code> , con renderizado Markdown y botones rápidos. <i>Entregable: frontend/.</i>	M
2.5.2	API admin (Flask)	Endpoints REST para CRUD de centros, titulaciones, asignaturas, profesores y planes docentes. <i>Entregable: admin/.</i>	L
2.5.3	SPA admin	Interfaz web con navegación jerárquica Centro → Titulación → Asignatura. <i>Entregable: frontend/admin.*.</i>	M
2.5.4	Integración de scrapers	Scrapers de Sevius y us.es expuestos en el panel con flujo <i>preview</i> → <i>insert</i> . <i>Entregable: rutas /sync.</i>	M

Cuadro 5.8: Diccionario EDT — 2.5: Panel admin y frontend.

3.1 — Validación y testing

Suite de pruebas multinivel siguiendo el modelo de calidad ISO/IEC 25010 y la metodología *Behavior-Driven Development*.

ID	Tarea	Descripción y entregables	Est.
3.1.1	Plan de aceptación	Diseño del plan de pruebas según ISO 25010 + BDD: positivos, negativos, edge, robustez. <i>Entregable: plans/aceptacion-prototipo.md</i> .	M
3.1.2	Suite E2E	159 casos sobre el stack completo (NLU + actions + SQL + RAG + LLM). <i>Entregable: 94,3 % PASS</i> .	L
3.1.3	RAG retrieval + profundidad	Capa baseline (50 preguntas) y profundidad (74 preguntas reales sobre 19 asignaturas). <i>Entregables: 94,0 % y 91,9 %</i> .	M
3.1.4	NLU + Policy	Validación con <code>rasa test core</code> sobre 44 stories. <i>Entregable: 100 % accuracy de acciones</i> .	S
3.1.5	Métricas no funcionales	Latencia p50/p95 y coste por turno con extrapolación a 700 alumnos. <i>Entregable: coste_latencia.md</i> .	M

Cuadro 5.9: Diccionario EDT — 3.1: Validación y testing.

3.2 — Piloto con usuarios

Despliegue del prototipo y validación con conversaciones reales de alumnos de la ETSII.

ID	Tarea	Descripción y entregables	Est.
3.2.1	Captura de tráfico real	Registro de las conversaciones piloto en <code>conversation.log</code> . <i>Entregable: 59 sesiones / 203 turnos</i> .	S
3.2.2	Auditoría manual	Replay turno a turno desde el panel admin; marcado de sesiones validadas. <i>Entregable: 58/59 sesiones validadas (98,3 %)</i> .	M
3.2.3	Iteración correctiva	Fixes detectados durante la auditoría (alias compactado, anti-alucinación, fecha actual en prompt, seguimiento cross-intent). <i>Entregable: registro de decisiones D-064..D-069</i> .	M

Cuadro 5.10: Diccionario EDT — 3.2: Piloto con usuarios.

3.3 — Despliegue en producción

Puesta en funcionamiento del sistema en una infraestructura accesible públicamente y configuración del pipeline de despliegue continuo.

ID	Tarea	Descripción y entregables	Est.
3.3.1	Provisión del droplet	Aprovisionamiento del servidor en DigitalOcean (Ubuntu 24.04 LTS), <i>firewall</i> y dominio. <i>Entregable: servidor accesible</i> .	S
3.3.2	Containerización	<code>docker-compose</code> con cuatro servicios (nginx, rasa, actions, admin) e imagen Rasa custom con spaCy. <i>Entregable: docker/</i> .	M
3.3.3	Pipeline CI/CD	<i>Workflow</i> de GitHub Actions que despliega vía SSH al droplet en cada <i>merge</i> a main. <i>Entregable: deploy.yml</i> .	S

Cuadro 5.11: Diccionario EDT — 3.3: Despliegue en producción.

3.4 — Memoria y defensa

Documentación académica final del proyecto y preparación del acto de defensa.

ID	Tarea	Descripción y entregables	Est.
3.4.1	Redacción de la memoria	Capítulos de introducción, marco, metodología, arquitectura, planificación, pruebas y conclusiones. <i>Entregable: memoria.</i>	L
3.4.2	Diagramas técnicos	Diagrama de componentes y de despliegue en notación UML2 (PlantUML). <i>Entregable: <code>img/diagrama_*.png</code>.</i>	S
3.4.3	Defensa	Diapositivas, ensayo y demo en vivo del prototipo desplegado. <i>Entregable: presentación.</i>	M

Cuadro 5.12: Diccionario EDT — 3.4: Memoria y defensa.

5.3– Resumen cuantitativo de la EDT

La Tabla 5.13 resume los paquetes de trabajo por fase y la distribución del esfuerzo estimado.

Fase	Paquete de trabajo	Tareas	S	M	L
1. Inicio	1.1 Gestión del proyecto	3	0	2	1
	1.2 Investigación tecnológica	4	1	2	1
	1.3 Diseño de la arquitectura	3	1	2	0
2. Desarrollo	2.1 Épica Asignaturas	5	0	4	1
	2.2 Épica Profesores	4	1	1	2
	2.3 Épica Horarios	4	0	2	2
	2.4 Pipeline RAG	4	0	3	1
	2.5 Panel admin y frontend	4	0	3	1
3. Cierre	3.1 Validación y testing	5	1	3	1
	3.2 Piloto con usuarios	3	1	2	0
	3.3 Despliegue en producción	3	2	1	0
	3.4 Memoria y defensa	3	1	1	1
Total		45	8	26	11

Cuadro 5.13: Resumen cuantitativo de la EDT por fase y paquete de trabajo.

5.4– Backlog del producto

El backlog del producto se deriva de los paquetes de la fase de Desarrollo y los organiza según su prioridad de implementación. Cada épica se desarrolla de forma incremental siguiendo el marco SCRUM adaptado descrito en el Capítulo 4, de modo que las funcionalidades se entregan en *releases* sucesivas validables por separado.

La priorización responde a dos criterios:

1. **Valor para el usuario:** Las épicas que cubren las consultas más frecuentes de los estudiantes (asignaturas, profesorado, horarios) se implementan primero.
2. **Dependencia técnica:** La infraestructura transversal (pipeline RAG, panel admin) se desarrolla en paralelo a las épicas que la consumen, pero su *release* depende de tener al menos una épica funcional sobre la que probarla.

La Tabla 5.14 muestra el orden de prioridad de las épicas dentro de la fase de Desarrollo.

Prioridad	Épica / paquete	Dependencia
1	2.1 Asignaturas (BD relacional + Text-to-SQL)	—
2	2.2 Profesores (BD + RAG plan docente)	2.1 (esquema BD compartido)
3	2.4 Pipeline RAG (transversal)	2.1 (inventario de fuentes)
4	2.3 Horarios (BD + extractor PDF)	2.1 (esquema BD)
5	2.5 Panel admin y frontend	Todas las anteriores

Cuadro 5.14: Priorización del backlog dentro de la fase de Desarrollo.

5.5– Planificación de sprints

El proyecto se organiza en sprints de **3–4 semanas de duración**, siguiendo una adaptación de SCRUM para un proyecto individual (véase Capítulo 4). Cada sprint cierra con un incremento funcional del sistema y un breve registro de decisiones técnicas.

5.6– Cronograma

Parte III

Diseño e Implementación

Análisis de requisitos

Este capítulo recoge el análisis de requisitos del sistema Linceus Assistant en su versión correspondiente al cierre del prototipo (fase 1). El análisis ha sido un proceso iterativo: la propuesta inicial planteaba un alcance amplio que incluía notificaciones a estudiantes, soporte multilingüe e integraciones en tiempo real con plataformas institucionales, pero la limitación de tiempo propia de un trabajo individual ha obligado a priorizar el núcleo conversacional y el dominio académico, y a aplazar el resto a iteraciones posteriores. La sección 6.6 cierra el capítulo precisamente con un repaso de cómo ha evolucionado el alcance respecto del documento original, dado que la guía de referencia [42] recomienda dejar trazabilidad de las desviaciones de planificación.

Los requisitos se organizan en cuatro tipos siguiendo la práctica habitual en ingeniería del software: requisitos funcionales (qué hace el sistema), requisitos de información (qué datos maneja), requisitos no funcionales (con qué calidad lo hace) y prototipos de interfaz (cómo se muestra al usuario). Cada requisito funcional se acompaña, cuando procede, de los identificadores de los casos de prueba que lo verifican, lo que permite cruzarlo con el Capítulo 12.

6.1– Identificación de actores

El sistema atiende a tres tipos de usuarios bien diferenciados, que justifican la estructura de los manuales recogida en el Capítulo 13.

El **estudiante** es el usuario principal del prototipo. Comprende tanto al alumnado matriculado en la ETSII como al alumnado de nuevo ingreso o internacional que utiliza el sistema antes de iniciar el curso. Interactúa con el bot a través del *widget* integrado en la página principal, sin necesidad de autenticación, y formula consultas en español natural sobre asignaturas, horarios y profesorado.

El **administrador del sistema** es el perfil técnico encargado de mantener actualizada la base de conocimiento. Su interacción se realiza exclusivamente a través del panel de administración (puerto 5050), e incluye la sincronización de centros, titulaciones y asignaturas desde Sevius, la vectorización de planes docentes en el módulo RAG y la auditoría de las conversaciones registradas durante el piloto. La existencia del panel obedece principalmente a un requisito de **mantenibilidad y escalabilidad**: el sistema debe poder ser asumido por otra persona tras el cierre del TFG sin necesidad de manipular directamente la base de datos.

El **desarrollador** es el perfil destinado a continuar el proyecto en futuras iteraciones. Consume el repositorio, las convenciones documentadas y los registros de decisiones técnicas para incorporar nuevas intenciones, ampliar el dominio o modificar las acciones existentes. Sus requisitos se materializan en buenas prácticas y trazabilidad, no en funcionalidades específicas del producto.

6.2– Requisitos funcionales

Los requisitos funcionales se agrupan por dominio. La columna *Pruebas* de cada tabla indica los identificadores de los casos de aceptación que verifican el requisito en el Capítulo 12.

6.2.1. Dominio de asignaturas

Cuadro 6.1: Requisitos funcionales del dominio de asignaturas.

ID	Descripción	Prio.	Pruebas
RF-A1	El sistema debe responder con la ficha completa de una asignatura cuando el usuario la nombra de forma explícita o mediante alias o acrónimo (ADDA, FP, IISSI2).	Alta	E-P01..E-P12
RF-A2	El sistema debe devolver listados de asignaturas filtrados por curso, cuatrimestre, tipo (formación básica, obligatoria, optativa) o créditos.	Alta	L-P01..L-P10
RF-A3	El sistema debe responder al conteo de asignaturas que cumplan un criterio dado.	Media	C-P01..C-T02
RF-A4	El sistema debe recuperar información del plan docente (criterios de evaluación, profesorado, bloques, idioma, tribunales) mediante el módulo RAG.	Alta	§12.3.2
RF-A5	El sistema debe heredar el contexto cuando una consulta elíptica (“y cuántos créditos tiene?”) se refiere a la última asignatura mencionada.	Media	E-S01..E-S03
RF-A6	El sistema debe resolver consultas multi-asignatura en un único turno (“cómo se evalúan DP1 y PSG2”).	Media	E-M01
RF-A7	El sistema debe distinguir entre la pregunta “qué es X”(ficha) y “cuándo es X”(horario), aun cuando ambas comparten léxico.	Alta	E-P09, HA-P01

6.2.2. Dominio de horarios

Cuadro 6.2: Requisitos funcionales del dominio de horarios.

ID	Descripción	Prio.	Pruebas
RF-H1	El sistema debe devolver el horario semanal de un curso y grupo dados.	Alta	H-P01..H-P11
RF-H2	El sistema debe devolver el horario filtrado por día de la semana.	Alta	H-P01, H-P10
RF-H3	El sistema debe filtrar el horario por cuatrimestre cuando el usuario lo especifica.	Media	H-PC01..H-PC03
RF-H4	El sistema debe responder al horario de una asignatura concreta sin requerir curso ni grupo.	Alta	HA-P01..HA-P12
RF-H5	El sistema debe distinguir aulas de teoría de aulas de laboratorio cuando el usuario lo solicita expresamente.	Media	HA-P11
RF-H6	El sistema debe solicitar curso y grupo cuando la consulta los requiere y faltan en el mensaje.	Alta	H-P10, H-P12
RF-H7	El sistema debe rechazar de manera explícita consultas con curso o grupo inexistentes (curso 8, grupo 15).	Media	H-N01, H-N02

Cuadro 6.3: Requisitos funcionales del dominio de profesorado.

ID	Descripción	Prio.	Pruebas
RF-P1	El sistema debe devolver los datos de contacto de un profesor (email, despacho, teléfono, web personal) por nombre o apellido.	Alta	P-P01..P-P10
RF-P2	El sistema debe listar el profesorado que imparte una asignatura.	Alta	P-PA01..P-PA08
RF-P3	El sistema debe identificar al coordinador y a los suplentes de una asignatura mediante recuperación semántica sobre el plan docente.	Alta	P-PA05..P-PA08
RF-P4	El sistema debe responder a consultas sobre tutorías redirigiendo al correo del profesor (decisión de diseño documentada en D-061).	Media	P-T01..P-T05
RF-P5	El sistema debe permitir filtrar el profesorado de un departamento concreto.	Media	P-P10
RF-P6	El sistema debe tolerar errores tipográficos moderados en nombres de profesores (“parejjo”, “galinndo”).	Media	P-W01..P-W07
RF-P7	El sistema debe declarar explícitamente la ausencia de información cuando un profesor no figura en la base de datos, sin inventar contenido.	Alta	P-N01..P-N04

Cuadro 6.4: Requisitos funcionales transversales.

ID	Descripción	Prio.	Pruebas
RF-T1	El sistema debe permitir cambiar dinámicamente la titulación activa durante una conversación, ya sea desde el selector o mediante lenguaje natural.	Alta	CC-P01..CC-P11
RF-T2	El sistema debe rechazar titulaciones fuera del alcance ETSII con un mensaje explícito (<i>cambia a Biología</i>).	Alta	CC-N01
RF-T3	El sistema debe identificar consultas fuera de ámbito (clima, peticiones generales) y responder de forma cortés sin entrar al pipeline de consulta académica.	Alta	F-01..F-04
RF-T4	El sistema debe bloquear intentos de <i>prompt injection</i> y <i>jailbreak</i> (“ignora las instrucciones anteriores”).	Alta	R-01..R-04
RF-T5	El sistema debe ofrecer un mensaje de bienvenida y soportar la pregunta meta-conversacional “¿qué eres?” redirigiendo a la naturaleza del bot.	Media	R-04
RF-T6	El sistema debe permitir al usuario reportar respuestas incorrectas mediante un mecanismo de <i>feedback</i> integrado en el <i>widget</i> .	Media	—

6.2.3. Dominio de profesorado

6.2.4. Comportamientos transversales

6.2.5. Funcionalidades del panel de administración

El panel de administración no se ofrece al estudiante final, sino al perfil de administrador descrito en la Sección 6.1. Su existencia responde a un requisito de mantenibilidad: garantizar que la persona que asuma la continuidad del sistema pueda gestionar los datos sin necesidad de manipular directamente la base de datos. Los requisitos se recogen en la Tabla 6.5.

Cuadro 6.5: Requisitos funcionales del panel de administración.

ID	Descripción	Prio.
RF-AD1	El panel debe permitir la navegación jerárquica Centro → Titulación → Asignatura sobre los datos cargados (D-037).	Alta
RF-AD2	El panel debe ofrecer la sincronización de centros, titulaciones y asignaturas desde Sevius como fuente de verdad institucional, mediante un flujo <i>preview-first</i> (D-038, D-039).	Alta
RF-AD3	El panel debe permitir vectorizar el plan docente de una asignatura, integrándolo en el corpus de <i>embeddings</i> consumido por el módulo RAG (D-040, D-041).	Alta
RF-AD4	El panel debe importar y enriquecer profesorado a partir del directorio PDI de <code>us.es</code> usando el filtro de centro (D-043, D-044).	Media
RF-AD5	El panel debe ofrecer una vista jerárquica Centro → Departamento → Profesor, con un <i>bucket</i> para profesores sin departamento asignado (D-051).	Media
RF-AD6	El panel debe exponer las conversaciones registradas en <code>conversation_log</code> para revisión manual del evaluador, con la posibilidad de marcarlas como revisadas.	Alta
RF-AD7	El panel debe agregar el <i>feedback</i> explícito por turno aportado por los usuarios y permitir su consulta agregada.	Media

6.3– Requisitos de información

El modelo de información del sistema se estructura en torno a tres bloques. El primero recoge los datos de **dominio académico** consumidos por el bot, que son la fuente de verdad para responder a las consultas del usuario. El segundo recoge los datos del **módulo RAG**, esto es, los fragmentos vectorizados de los planes docentes con sus respectivos *embeddings*. El tercero recoge los datos de **operación**, es decir, las conversaciones registradas durante el piloto y el *feedback* explícito asociado.

6.3.1. Entidades de dominio

Las entidades del dominio académico son las recogidas en la Tabla 6.6. La descripción técnica completa de cada tabla, con sus tipos, claves y restricciones, figura en el Capítulo 10.

6.3.2. Entidades del módulo RAG

El módulo RAG opera sobre fragmentos textuales (*chunks*) extraídos de los planes docentes. Cada *chunk* se almacena junto con su *embedding* vectorial calculado con el modelo de *embeddings* de Gemini, y con un conjunto de metadatos que permiten filtrar y reordenar resultados durante la recuperación: titulación, asignatura, sección del plan docente (profesorado, evaluación, bloques,

Cuadro 6.6: Entidades de información del dominio académico.

Entidad	Descripción
Centro	Unidad organizativa que agrupa titulaciones (ETSII).
Titulación	Plan de estudios (GII-IS, GII-IC, GII-TI).
Asignatura	Materia con su código, créditos, curso, cuatrimestre y tipo.
Departamento	Unidad académica responsable del profesorado.
Profesor	Docente con datos de contacto, despacho y departamento.
Profesor-Asignatura	Relación que vincula un profesor con la docencia de una asignatura, con el grupo y el rol (titular, coordinador, suplente).
Grupo de clase	Asignatura impartida en un grupo concreto a un curso.
Aula	Espacio físico identificado por código (A0.10, B1.31, F1.30) y tipo (teoría/laboratorio).
Horario	Sesión docente que vincula grupo, aula, día, franja horaria y cuatrimestre.
Plan docente	Documento PDF oficial asociado a una asignatura, fuente del corpus RAG.

etc.), grupo y curso. La sección como metadato fue una decisión de diseño documentada (D-031, D-058.b): se usa como señal para reordenar resultados pero no como filtro estricto, dado que la sección de origen no siempre es informativa para responder a la pregunta.

6.3.3. Entidades de operación

La operación del sistema en producción registra dos entidades. La tabla `conversation_log` almacena turno a turno cada interacción con el bot: identificador anónimo de sesión, mensaje del usuario, *intent* clasificado, *slots* activos, respuesta del bot, indicador de revisión manual y marca temporal. La tabla `feedback` recoge la valoración explícita del usuario sobre un turno concreto cuando este utiliza los botones de aprobación o rechazo del *widget*. Ninguna de las dos almacena datos personales identificables, dado que la sesión es anónima por diseño.

6.4– Requisitos no funcionales

Los requisitos no funcionales se enuncian en términos medibles y se cruzan con la evidencia recogida en el Capítulo 12. Los umbrales son los que el equipo del TFG considera aceptables para un prototipo de fase 1 desplegable a la población de la ETSII; se han ajustado durante el desarrollo a la luz de las cifras observadas durante el piloto.

Conviene comentar dos requisitos en particular. El cumplimiento de la **normativa de protección de datos** (RNF-10) se materializa por construcción más que por restricciones de despliegue: el sistema no requiere autenticación, no almacena datos personales identificables, y los datos académicos consultados (planes docentes, horarios, directorio de profesorado) son de carácter público y proceden de fuentes oficiales (Sevius, `us.es`). La propuesta inicial contemplaba un despliegue *on-premise* como mecanismo de garantía adicional; en la implementación final, dado que la información tratada no incluye datos sensibles, se ha optado por un despliegue en infraestructura externa (DigitalOcean y Supabase) que simplifica la operación sin comprometer la privacidad del usuario.

El requisito de **soporte multilingüe**, que figuraba como funcional en la propuesta original, se ha reformulado y aplazado: el corpus académico está mayoritariamente redactado en español, los modelos de *embeddings* y de generación se han ajustado al castellano y la dotación de tiempo del prototipo no permitía abrir un segundo flujo de pruebas para el inglés. La sección 6.6 lo recoge entre las funcionalidades aplazadas a fase 2.

Cuadro 6.7: Requisitos no funcionales y evidencia.

ID	Descripción	Umbral	Evidencia
RNF-1	Latencia mediana de respuesta extremo a extremo.	$p50 \leq 4 \text{ s}$	§12.6.1 (3,4 s)
RNF-2	Latencia en cola larga.	$p95 \leq 12 \text{ s}$	§12.6.1 (10,5 s)
RNF-3	Coste medio por turno (servicio Gemini).	$\leq 0,05 \text{ cts}$	§12.6.2 (0,012 cts)
RNF-4	Tasa de éxito en suite de aceptación E2E.	$\geq 90 \%$	§12.2 (94,3 %)
RNF-5	Tasa de éxito sobre tráfico real post-despliegue.	$\geq 85 \%$	§12.5 (98,3 %)
RNF-6	Disponibilidad del servicio.	$\geq 99 \%$ mensual	Pendiente fase 2
RNF-7	Idioma soportado.	Español peninsular	Validado en piloto
RNF-8	Compatibilidad de navegadores.	Chrome, Firefox, Edge y Safari modernos.	Verificado manualmente
RNF-9	Diseño <i>responsive</i> en dispositivos móviles.	Funcional en pantallas $\geq 320 \text{ px}$.	Verificado manualmente
RNF-10	Anonimato del usuario y ausencia de PII.	Sesiones identificadas mediante UUID anónimo.	Por diseño
RNF-11	Reproducibilidad del despliegue.	Stack levantara mediante docker compose up.	§13.2
RNF-12	Trazabilidad de decisiones técnicas.	Registro en docs/sprints/.	Sprints S2–S7

6.5– Prototipos de interfaz

El prototipo no se ha apoyado en herramientas de *wireframing* (Figma o similares) durante la fase de diseño: la interfaz se ha construido directamente sobre código HTML/CSS/JS *vanilla*, tomando como referencia la identidad visual de `www.us.es` (decisión D-027) para garantizar coherencia con el ecosistema institucional. La validación visual y de usabilidad se ha realizado de forma iterativa contra los *stakeholders* del piloto, incorporando los cambios identificados durante el feedback (D-029, D-030).

A continuación se recogen las capturas de las dos interfaces principales. Su inserción definitiva queda como tarea de redacción final de la memoria.



Figura 6.1: Widget conversacional accesible para el estudiante.

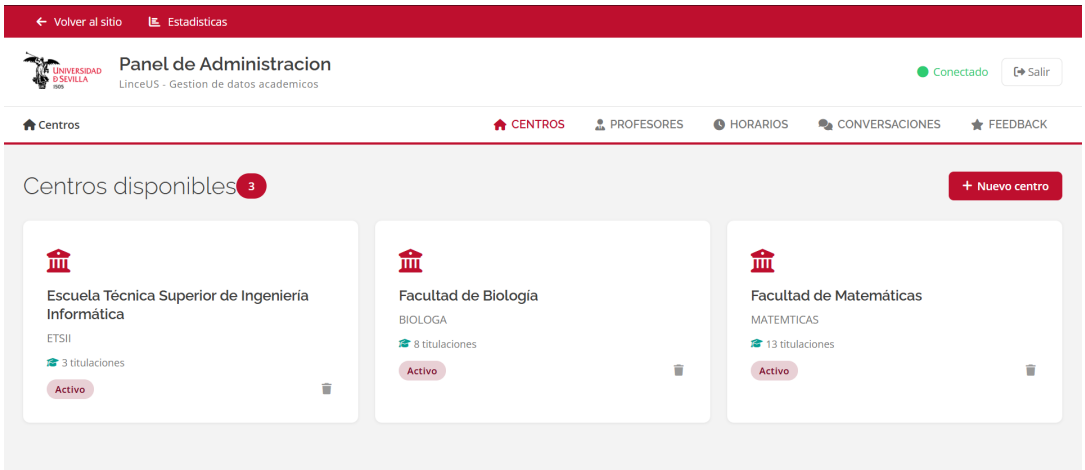


Figura 6.2: Panel de administración: gestión de asignaturas.

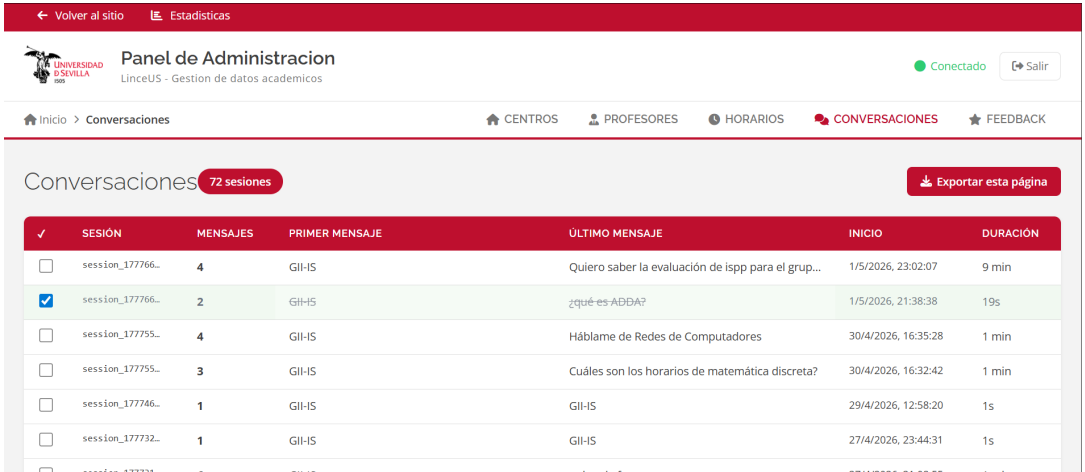


Figura 6.3: Panel de administración: auditoría de conversaciones.

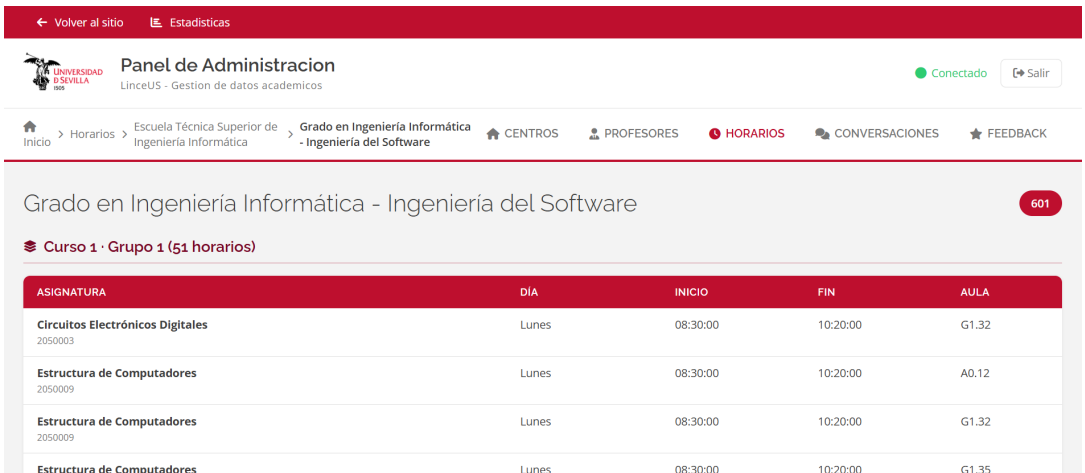


Figura 6.4: Panel de administración: gestión de horarios.

Panel de Administración
LinceUS - Gestion de datos académicos

Conectado Salir

Inicio > Profesores > Escuela Técnica Superior de Ingeniería Informática > Departamento de Lenguajes y Sistemas Informáticos

CENTROS PROFESORES HORARIOS CONVERSACIONES FEEDBACK

Departamento de Lenguajes y Sistemas Informáticos 92 Enriquecer desde us.es

NOMBRE	DEPARTAMENTO	CATEGORIA	EMAIL	DESPACHO	PERFIL
Álvarez García, Juan Antonio	Departamento de Lenguajes y Sistemas Informáticos	Catedrático de Universidad	jaalvarez@us.es	F1.52	✉
Arévalo Maldonado, Carlos	Departamento de Lenguajes y Sistemas Informáticos	Profesor Titular de Universidad	carlosarevalo@us.es	F1.41	✉
Ayala Hernández, Daniel	Departamento de Lenguajes y Sistemas Informáticos	Profesor Permanente Laboral	dayala1@us.es	F1.81	✉
Barba Rodríguez, Irene	Departamento de Lenguajes y Sistemas Informáticos	Profesora Titular de Universidad	irenebr@us.es	F0.46	✉

Figura 6.5: Panel de administración: gestión de profesorado.

6.6– Evolución del alcance respecto a la propuesta inicial

La propuesta original del proyecto, recogida en el anteproyecto [42], contemplaba un alcance superior al efectivamente cubierto por el prototipo. La Tabla 6.8 resume las desviaciones principales y su justificación. En todos los casos las decisiones se han tomado con el objetivo de **garantizar la calidad de las funcionalidades implementadas**, antes que de cubrir un alcance amplio con calidad insuficiente, en línea con la recomendación de la guía de referencia.

Cuadro 6.8: Evolución del alcance respecto a la propuesta inicial.

Funcionalidad	Estado	Justificación
Frontend en React	Reformulado	Implementado en HTML/CSS/JavaScript <i>vanilla</i> sobre nginx. La inversión de tiempo se ha priorizado en el desarrollo del chatbot, donde el valor añadido del proyecto es mayor.
Notificaciones <i>push</i> a estudiantes	Aplazado	Sustituido por el panel de administración , que ofrece mayor valor a la persona que continúe el proyecto y materializa los requisitos de mantenibilidad y escalabilidad.
Soporte multilingüe (inglés)	Aplazado	El corpus académico es mayoritariamente español. Se aplaza a fase 2 cuando el sistema esté estabilizado y exista presupuesto para una segunda suite de pruebas.
Despliegue <i>on-premise</i>	Reformulado	La información tratada es de carácter público y la sesión es anónima, por lo que no requiere despliegue en infraestructura propia. Se ha optado por DigitalOcean + Supabase.
Cambio dinámico de titulación	Añadido	Funcionalidad incorporada tras el feedback del piloto (D-029, D-030, D-053). No figuraba en la propuesta inicial.
Integración con <i>us.es</i> para profesorado	Añadido	Sustituye la propuesta inicial de <i>scrapers</i> por departamento (D-043, D-044): fuente única, mantenible y completa.
Auditoría manual de conversaciones	Añadido	Surge como necesidad durante la fase de validación; permite la métrica de tráfico real post-despliegue (§12.5).

El balance final es satisfactorio: el sistema cubre con calidad los tres dominios académicos centrales de la propuesta y aporta dos funcionalidades no previstas (panel de administración y cambio dinámico de titulación) que refuerzan los requisitos de mantenibilidad y experiencia de

usuario. Las funcionalidades aplazadas son recuperables en una segunda iteración sin necesidad de modificar el núcleo del sistema, lo que se discute con detalle en el Capítulo 14.

Arquitectura del sistema y decisiones de diseño

Este capítulo describe la arquitectura del sistema LinceUS Assistant, las decisiones de diseño adoptadas durante su desarrollo y los diagramas técnicos que representan la estructura de componentes y el modelo de despliegue. El objetivo es ofrecer una visión completa de cómo se organiza internamente el sistema, cómo se comunican sus partes y qué criterios han guiado las principales elecciones técnicas.

7.1– Arquitectura general

LinceUS Assistant sigue una **arquitectura por capas** (*layered architecture*) con tres niveles claramente diferenciados: capa de presentación, capa de procesamiento conversacional y capa de datos. Esta separación permite que cada capa evolucione de forma independiente y que los cambios en una no afecten directamente a las demás, siempre que se respeten las interfaces definidas entre ellas.

7.1.1. Capa de presentación

La capa de presentación es la interfaz visible para el usuario final. Consiste en un **widget de chat web** desarrollado en JavaScript puro (vanilla JS), HTML y CSS, diseñado como un componente autocontenido y embebible en cualquier página institucional de la Universidad de Sevilla.

Las principales características de esta capa son:

- **Independencia tecnológica:** El widget no depende de frameworks como React o Angular; se implementa con JavaScript estándar (ES6+), lo que minimiza las dependencias y facilita su integración en páginas web existentes mediante una única etiqueta `<script>`.
- **Comunicación REST:** El widget se comunica con el servidor Rasa mediante peticiones HTTP POST al endpoint `/webhooks/rest/webhook`, enviando mensajes del usuario y recibiendo las respuestas del asistente en formato JSON.
- **Renderizado enriquecido:** Soporta formato Markdown en las respuestas del bot, incluyendo tablas, enlaces y listas, para presentar la información académica de forma clara y estructurada.
- **Diseño institucional:** Emplea la identidad visual de la Universidad de Sevilla (colores corporativos, logotipo) para integrarse visualmente en el entorno universitario.

7.1.2. Capa de procesamiento conversacional

Esta capa constituye el núcleo del sistema y está compuesta por varios servicios que colaboran entre sí:

- **Rasa Server:** Motor principal de comprensión del lenguaje natural (NLU) y gestión de diálogo (Core). Recibe los mensajes del usuario, los procesa a través del pipeline NLU para extraer intenciones y entidades, y decide la siguiente acción a ejecutar mediante las políticas de diálogo configuradas.
- **Action Server (Rasa SDK):** Servidor independiente que ejecuta las acciones personalizadas escritas en Python. Cuando Rasa Core determina que debe ejecutarse una acción personalizada (por ejemplo, consultar la base de datos), delega la ejecución en este servidor a través de una petición HTTP al endpoint `/webhook`.
- **Google Gemini API:** Servicio externo utilizado para dos propósitos: (1) generación de consultas SQL a partir de lenguaje natural (*Text-to-SQL*) mediante el modelo `gemma-3-27b-it`, y (2) generación de embeddings vectoriales mediante el modelo `gemini-embedding-001` para la búsqueda semántica en el pipeline RAG.
- **Ollama (opcional):** Servicio local de inferencia de modelos LLM que actúa como alternativa a Gemini cuando la API externa no está disponible o se agotan las cuotas de uso. Permite ejecutar modelos como `llama3.2:3b` en CPU sin depender de servicios en la nube.

7.1.3. Capa de datos

La capa de datos se implementa sobre **Supabase**, una plataforma que proporciona una instancia gestionada de PostgreSQL con extensiones adicionales:

- **PostgreSQL:** Almacena toda la información académica estructurada: universidades, centros, titulaciones, asignaturas, planes docentes y sus metadatos asociados.
- **pgvector:** Extensión que añade el tipo de dato `vector` y operadores de similitud (distancia coseno, euclidiana y producto punto). Almacena los embeddings de los fragmentos de documentos vectorizados y permite realizar búsquedas semánticas eficientes mediante índices HNSW.
- **Funciones SQL personalizadas:** Funciones almacenadas en la base de datos que encapsulan la lógica de búsqueda vectorial (`buscar_plan_docente()`, `buscar_plan_docente_global()`), permitiendo filtrar por asignatura, grupo y curso académico con un umbral de similitud configurable.

7.2– Diagrama de componentes

El diagrama de componentes muestra los módulos principales del sistema, sus responsabilidades y las interfaces a través de las cuales se comunican. La Figura 7.1 representa esta vista.

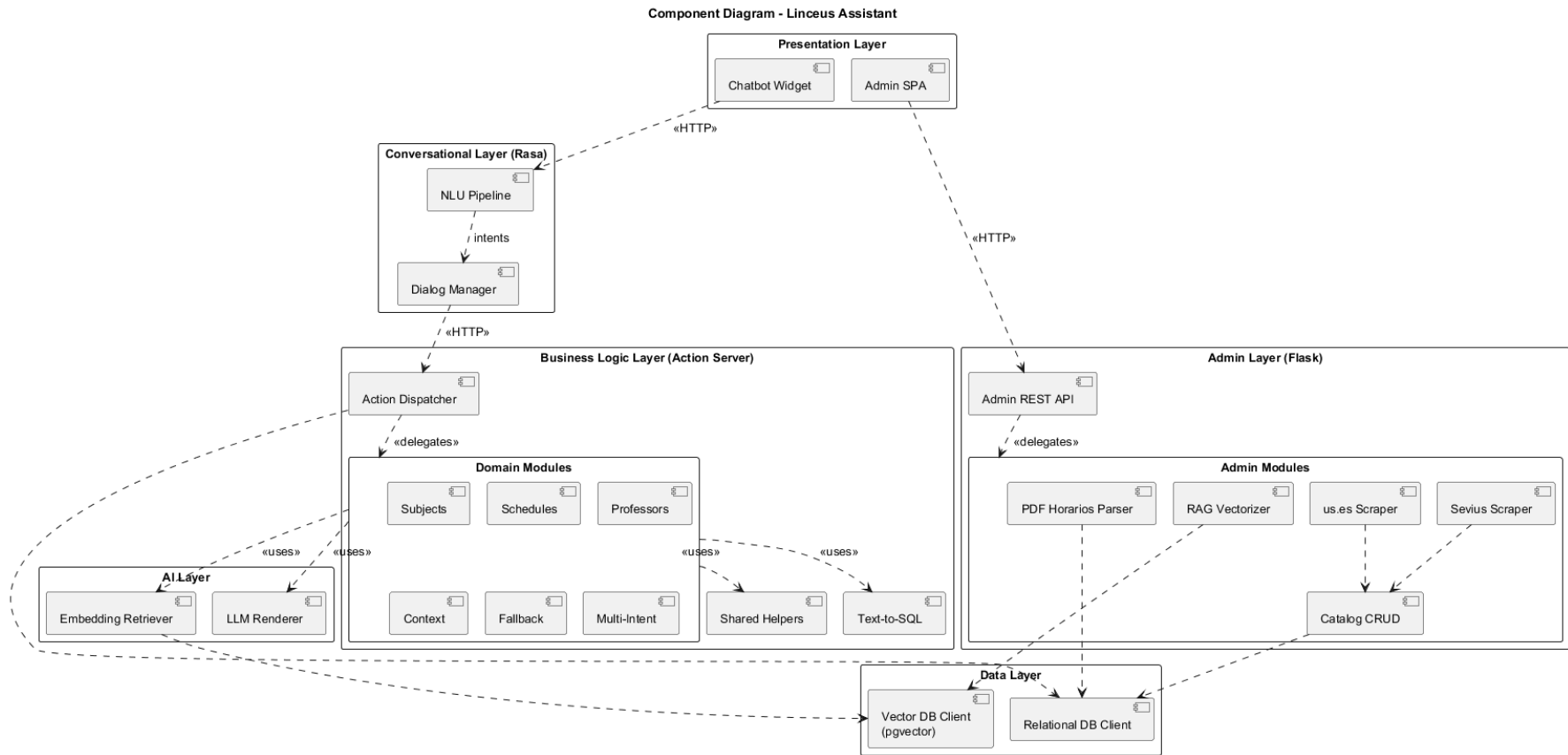


Figura 7.1: Diagrama de componentes del sistema Linceus.

A continuación se describe la responsabilidad de cada componente y las interfaces entre ellos:

- **Chat Widget** → **Rasa NLU**: El widget envía el texto del usuario mediante una petición HTTP POST al endpoint REST de Rasa. La comunicación es síncrona: el widget espera la respuesta para mostrarla en la interfaz.
- **Rasa NLU** → **Rasa Core**: Tras procesar el mensaje a través del pipeline NLU (tokenización, extracción de características, clasificación de intención y extracción de entidades), el resultado se pasa al módulo Core, que decide la siguiente acción.
- **Rasa Core** → **Action Server**: Cuando la acción seleccionada es personalizada (prefijo `action_`), Core envía una petición HTTP al Action Server con el tracker completo de la conversación (historial, slots, última intención y entidades).
- **Action Server** → **Text-to-SQL**: Para consultas sobre asignaturas, el Action Server invoca el motor Text-to-SQL, que construye un prompt con el esquema de la base de datos y la pregunta del usuario, lo envía a Gemini y recibe una consulta SQL validada.
- **Action Server** → **Gemini / Ollama**: El Action Server puede invocar Gemini para generación de SQL, detección de titulación o generación de embeddings. Si Gemini no está disponible, se recurre a Ollama como alternativa local.
- **RAG Pipeline** → **pgvector**: La búsqueda semántica genera un embedding de la pregunta del usuario y lo compara contra los embeddings almacenados en pgvector usando similitud coseno, con un umbral mínimo de 0.5.
- **Text-to-SQL** → **PostgreSQL**: Las consultas SQL generadas se ejecutan directamente contra la base de datos PostgreSQL de Supabase, con filtros de seguridad inyectados automáticamente (restricción por titulación).

7.3– Diagrama de despliegue

El diagrama de despliegue describe la distribución física de los componentes del sistema en nodos de ejecución y las conexiones de red entre ellos. La Figura 7.2 muestra esta configuración.

7.3.1. Nodos de ejecución

El sistema se despliega sobre los siguientes nodos:

1. **Navegador web del usuario**: Ejecuta el widget de chat (JavaScript) y se comunica con el servidor Rasa mediante HTTP. No requiere instalación alguna por parte del usuario.
2. **Máquina local de desarrollo**: Alberga los tres servicios principales del backend:
 - *Rasa Server* (`localhost:5005`): Sirve la API REST del chatbot y carga el modelo entrenado (`.tar.gz`).
 - *Action Server* (`localhost:5055`): Ejecuta las acciones personalizadas con un timeout de 300 segundos para acomodar consultas LLM en CPU.
 - *Ollama* (`localhost:11434`): Servicio opcional de inferencia local de modelos LLM.
 - *RAG Pipeline*: Proceso batch que se ejecuta bajo demanda para vectorizar nuevos documentos PDF.

3. **Supabase Cloud (AWS eu-west-3):** Instancia gestionada de PostgreSQL con pgvector, accesible mediante connection pooling en el puerto 6543. Aloja tanto los datos estructurados como los embeddings vectoriales.
4. **Google Cloud:** Proporciona acceso a la API de Gemini para generación de texto (Text-to-SQL) y generación de embeddings. La comunicación se realiza mediante HTTPS con autenticación por API key.

7.3.2. Protocolos de comunicación

Origen	Destino	Protocolo	Puerto
Navegador	Rasa Server	HTTP POST (REST)	5005
Rasa Server	Action Server	HTTP POST (webhook)	5055
Action Server	Supabase	TCP (psycopg2)	6543
Action Server	Gemini API	HTTPS (REST)	443
Action Server	Ollama	HTTP (REST)	11434
RAG Pipeline	Supabase	TCP (psycopg2)	6543
RAG Pipeline	Gemini Embeddings	HTTPS (REST)	443

Cuadro 7.1: Protocolos y puertos de comunicación entre nodos del sistema.

7.4– Decisiones de diseño

A lo largo del desarrollo del sistema se han tomado decisiones de diseño que han condicionado la arquitectura, el rendimiento y la mantenibilidad del proyecto. En esta sección se documentan las más relevantes, indicando el contexto, las alternativas consideradas y la justificación de cada elección.

7.4.1. Text-to-SQL con LLM en lugar de reglas estáticas

Contexto: El sistema debe responder consultas sobre asignaturas que pueden formularse de formas muy diversas (por nombre, curso, tipología, créditos, duración o combinaciones de estos filtros). Implementar todas las variantes mediante reglas *if-else* o expresiones regulares resultaría en un código excesivamente largo, frágil y difícil de mantener.

Decisión: Se optó por un enfoque **Text-to-SQL basado en LLM**, donde el modelo Gemini (gemma-3-27b-it) recibe un prompt que incluye el esquema de la tabla `asignaturas`, las columnas permitidas y la pregunta del usuario, y genera la consulta SQL correspondiente.

Justificación:

- **Flexibilidad:** El modelo interpreta preguntas en lenguaje natural sin necesidad de anticipar todas las formulaciones posibles.
- **Mantenibilidad:** Añadir nuevos atributos o filtros solo requiere actualizar el prompt, no reescribir lógica de parsing.
- **Seguridad:** Se aplican mecanismos de protección como una *whitelist* de columnas permitidas, prohibición de subconsultas e inyección automática del filtro de titulación.

Temperatura 0.0: Se configura el modelo con temperatura cero para obtener respuestas deterministas, crítico para la generación de SQL donde la variabilidad es indeseable.

7.4.2. Gemini como servicio principal con Ollama como alternativa local

Contexto: El sistema requiere un modelo de lenguaje grande para Text-to-SQL y generación de embeddings. Depender exclusivamente de una API externa implica riesgos de disponibilidad y costes, mientras que depender solo de ejecución local exige hardware potente.

Decisión: Se adoptó una **estrategia dual**: Gemini como servicio principal (plan gratuito con 15.000 peticiones/día) y Ollama como alternativa local para situaciones donde la API externa no esté disponible.

Justificación:

- El plan gratuito de Gemini es suficiente para el volumen de uso académico del proyecto.
- Ollama con `llama3.2:3b` permite seguir operando sin conexión a internet, aunque con menor calidad en las respuestas.
- Esta dualidad alinea el proyecto con los requisitos de privacidad (RGPD), ya que el flujo local no envía datos a terceros.

7.4.3. Embeddings de 2000 dimensiones con HNSW

Contexto: El modelo `gemini-embedding-001` genera embeddings de 3072 dimensiones de forma nativa. Sin embargo, el índice HNSW de `pgvector` presenta limitaciones de rendimiento y almacenamiento con dimensionalidades muy altas.

Decisión: Se redujo la dimensionalidad de salida a **2000 dimensiones** mediante el parámetro `output_dimensionality` de la API de Gemini.

Justificación:

- 2000 dimensiones ofrecen un equilibrio entre la calidad de la representación semántica y la eficiencia del índice HNSW (parámetros $m = 16$, $ef_construction = 64$).
- Reduce el tamaño de almacenamiento por vector en aproximadamente un 35 %.
- Las pruebas empíricas muestran que la pérdida de precisión en la búsqueda semántica es negligible para el corpus de documentos del proyecto (planes docentes de la ETSII).

7.4.4. Chunking con solapamiento para preservar contexto

Contexto: Los planes docentes en PDF contienen información densa que debe fragmentarse para su vectorización. Un chunking demasiado grande diluye la relevancia; uno demasiado pequeño pierde contexto.

Decisión: Se implementó un chunking de **800 caracteres con 100 caracteres de solapamiento** y un mínimo de 50 caracteres por fragmento.

Justificación:

- 800 caracteres (≈ 150 – 200 tokens) es suficiente para capturar un párrafo o sección temática completa.
- El solapamiento de 100 caracteres garantiza que la información en los límites de fragmentos no se pierda, preservando la coherencia semántica.
- Se aplica detección de secciones por expresiones regulares (“Datos básicos”, “Objetivos”, “Contenidos”, “Evaluación”, “Bibliografía”) para etiquetar cada fragmento con su sección de origen.

7.4.5. Búsqueda vectorial con fallback a búsqueda por palabras clave

Contexto: La generación de embeddings para las consultas de búsqueda depende de la API de Gemini, que puede agotar su cuota diaria (100 peticiones por minuto, 1000 por día).

Decisión: Se implementó un sistema de **dobles búsquedas**: búsqueda vectorial como método principal y búsqueda por palabras clave (ILIKE) como fallback automático.

Justificación:

- La búsqueda vectorial ofrece mejor precisión semántica (encuentra “sistema de evaluación” cuando el usuario pregunta “¿cómo se califica?”).
- El fallback por palabras clave garantiza que el sistema siga operativo aunque la API de embeddings no esté disponible, con resultados razonables para consultas que contienen términos exactos.
- El umbral de similitud coseno se fija en 0.5, bajo el cual los resultados se descartan por considerarse irrelevantes.

7.4.6. Fuzzy matching para resolución de nombres

Contexto: Los usuarios introducen nombres de asignaturas con errores ortográficos, sin tildes, con abreviaturas o con variaciones coloquiales (“FP” por “Fundamentos de Programación”, “ADDA” por “Análisis y Diseño de Datos y Algoritmos”).

Decisión: Se integró la librería **RapidFuzz** con un umbral de similitud del 80 % combinado con una tabla de alias y acrónimos mantenida en el código.

Justificación:

- El algoritmo de distancia de Levenshtein de RapidFuzz tolera errores tipográficos habituales sin requerir entrenamiento adicional.
- La tabla de alias cubre las abreviaturas más comunes utilizadas por los estudiantes de la ETSII.
- Si la coincidencia difusa falla, se recurre al LLM (Gemini) como último recurso para intentar identificar la asignatura.

7.4.7. Slots conversacionales para memoria de contexto

Contexto: Los usuarios formulan preguntas de seguimiento que hacen referencia implícita a asignaturas o titulaciones mencionadas previamente (“¿Y cuántos créditos tiene?”, “¿Es obligatoria?”).

Decisión: Se utilizan **slots de Rasa** para mantener el estado de la conversación:

- `contexto_titulacion`: Titulación activa seleccionada por el usuario.
- `ultimo_codigo_consultado`: Código de la última asignatura consultada.
- `ultimo_nombre_asignatura`: Nombre de la última asignatura mencionada.
- `ultimos_resultados_asignaturas`: Lista de resultados de la última consulta (para paginación).

Justificación:

- Los slots permiten que las acciones personalizadas recuperen el contexto sin que el usuario deba repetir información.

- La separación entre código y nombre de asignatura facilita tanto las consultas SQL (por código) como las respuestas al usuario (por nombre legible).
- La titulación por defecto es GII-IS (Ingeniería del Software), configurable por el usuario mediante la intención `cambiar_contexto_academico`.

7.4.8. Pipeline NLU de 9 etapas con doble featurización

Contexto: El pipeline NLU de Rasa debe ser capaz de clasificar correctamente intenciones en español, tolerando errores ortográficos, abreviaturas y variaciones léxicas propias del lenguaje coloquial estudiantil.

Decisión: Se configuró un pipeline de 9 componentes que combina **featurización a nivel de palabra** (n-gramas 1–2) y **a nivel de carácter** (n-gramas 1–4):

1. `WhitespaceTokenizer`: Tokenización por espacios en blanco.
2. `RegexFeaturizer`: Patrones regex para códigos de asignatura y formatos especiales.
3. `LexicalSyntacticFeaturizer`: Características gramaticales del contexto local.
4. `CountVectorsFeaturizer` (palabra): N-gramas de palabras (1–2).
5. `CountVectorsFeaturizer` (carácter): N-gramas de caracteres (1–4).
6. `DIETClassifier`: Clasificador dual de intenciones y entidades (150 épocas, dropout 0.2).
7. `EntitySynonymMapper`: Normalización de sinónimos de entidades.
8. `ResponseSelector`: Selección de respuestas tipo FAQ.
9. `FallbackClassifier`: Detección de baja confianza (umbral 0.7).

Justificación:

- La featurización a nivel de carácter es especialmente eficaz para manejar errores ortográficos (“Fundamnetos” se reconoce por similitud de caracteres con “Fundamentos”).
- El umbral de fallback de 0.7 equilibra la precisión (evitar respuestas incorrectas) con la cobertura (no rechazar demasiadas consultas legítimas).
- El `DIETClassifier` con 150 épocas y dropout de 0.2 proporciona un modelo robusto sin sobreajuste al conjunto de entrenamiento.

7.4.9. Hash SHA-256 para evitar reprocesamiento de documentos

Contexto: El pipeline de vectorización RAG procesa documentos PDF que pueden ser costosos de re-vectorizar (extracción de texto, chunking, generación de embeddings con API de pago por uso).

Decisión: Se almacena un **hash SHA-256** de cada PDF procesado en la tabla `planes_docentes`. Antes de procesar un documento, se compara el hash actual con el almacenado.

Justificación:

- Si el hash coincide, el documento no ha cambiado y se omite el reprocesamiento, ahorrando llamadas a la API de embeddings.
- Si el hash difiere, se eliminan los fragmentos anteriores y se reprocesa el documento completo, garantizando la consistencia de los datos.
- Este mecanismo es especialmente relevante dado el límite de 1000 embeddings por día del plan gratuito de Gemini.

7.5– Patrones arquitectónicos aplicados

El diseño del sistema incorpora varios patrones arquitectónicos reconocidos en la ingeniería del software:

7.5.1. Arquitectura por capas (*Layered Architecture*)

Como se ha descrito en la Sección 7.1, el sistema se organiza en tres capas con responsabilidades bien definidas. Cada capa solo se comunica con la inmediatamente inferior, lo que reduce el acoplamiento y facilita el reemplazo de componentes (por ejemplo, cambiar el frontend sin afectar la lógica de negocio).

7.5.2. Patrón *Action Server* (microservicio de lógica)

Rasa delega la ejecución de lógica de negocio en un servidor de acciones independiente. Este patrón es similar al patrón *Backend for Frontend* (BFF), donde un servicio intermedio adapta las respuestas de los servicios de datos al formato esperado por el cliente conversacional. La separación permite escalar el Action Server independientemente del motor Rasa.

7.5.3. Patrón *Strategy* para clientes LLM

Los clientes de Gemini (`gemini_client.py`) y Ollama (`ollama_client.py`) implementan una interfaz similar para la generación de texto. El Action Server puede alternar entre ambos proveedores en función de la disponibilidad, aplicando el patrón *Strategy* para desacoplar la lógica de negocio del proveedor concreto de LLM.

7.5.4. Patrón *Pipeline* para procesamiento RAG

El pipeline de vectorización sigue el patrón *Pipeline* (o *Pipes and Filters*), donde cada etapa transforma los datos y los pasa a la siguiente: extracción de PDF → chunking → generación de embeddings → inserción en base de datos. Cada etapa es independiente y puede modificarse sin afectar a las demás.

7.5.5. Patrón *Repository* para acceso a datos

El módulo `db.py` centraliza todas las operaciones contra la base de datos, proporcionando una abstracción que oculta los detalles de conexión (pool de conexiones `psycopg2`) y ejecución de consultas al resto del sistema. Esto facilita el testing unitario mediante mocks y permite cambiar la implementación de persistencia sin modificar la lógica de negocio.

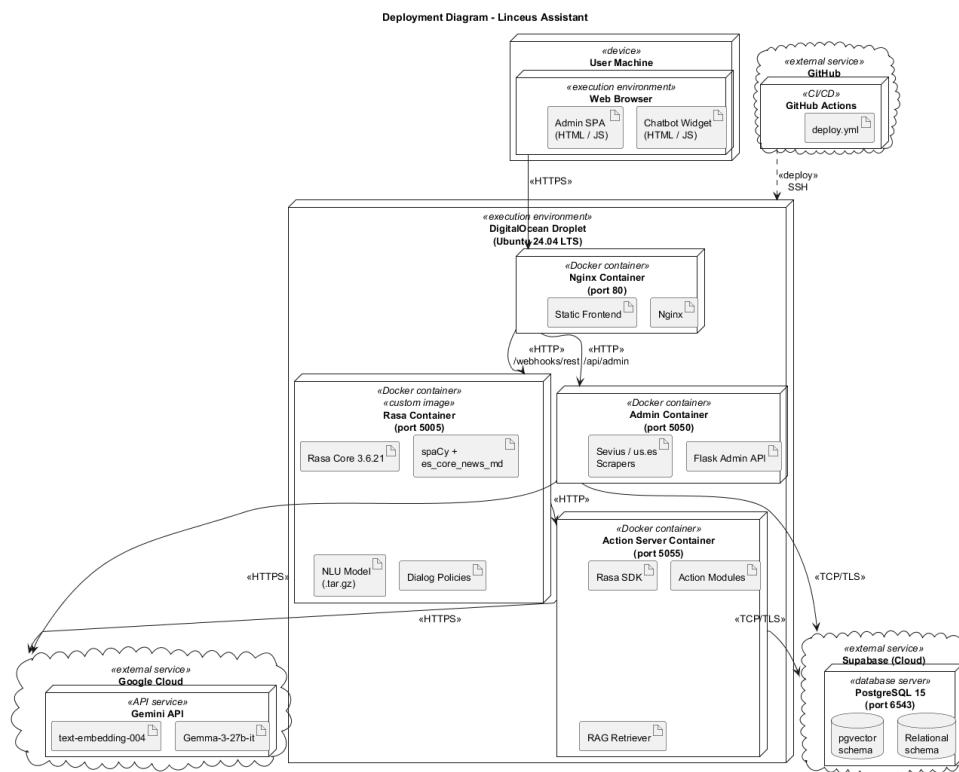


Figura 7.2: Diagrama de despliegue del sistema Linceus en DigitalOcean (Ubuntu 24.04 LTS) con el pipeline de despliegue automatizado vía GitHub Actions.

Marco tecnológico

Este capítulo describe el conjunto de tecnologías, frameworks y paradigmas que conforman la base técnica del proyecto LinceUS Assistant. Se complementa con el Capítulo 7, donde se detallan la arquitectura del sistema, las decisiones de diseño y los diagramas de componentes y despliegue. Aquí se justifica la elección de cada componente del stack tecnológico.

8.1– Tecnologías de procesamiento conversacional

8.1.1. Rasa Framework

Rasa es un framework de código abierto para la construcción de asistentes conversacionales basados en IA. A diferencia de soluciones comerciales como Dialogflow o Lex de AWS, Rasa ofrece:

- **Control total sobre el modelo:** Los datos de entrenamiento y los modelos permanecen bajo control del desarrollador
- **Personalización completa:** Políticas de diálogo y pipelines de NLU totalmente configurables
- **Ejecución local:** No depende de servicios externos para funcionar
- **Multilinguaje:** Soporte nativo para español mediante modelos de spaCy

Componentes de Rasa utilizados

Rasa NLU (Natural Language Understanding)

Rasa NLU se encarga de interpretar la entrada del usuario y extraer:

- **Intenciones (intents):** ¿Qué quiere hacer el usuario? (e.g., `consultar_horario`, `buscar_asignatura`)
- **Entidades (entities):** Información específica mencionada por el usuario (e.g., nombre de asignatura, curso, centro)

Rasa Core

Rasa Core gestiona el flujo de conversación mediante:

- **Stories:** Conversaciones de ejemplo que entrenan el modelo de diálogo
- **Rules:** Reglas determinísticas para situaciones específicas
- **Políticas:** Algoritmos de aprendizaje que deciden la siguiente acción del bot

Rasa Actions

Las acciones personalizadas permiten al bot ejecutar código Python para:

- Consultar la base de datos
- Generar respuestas dinámicas
- Llamar a APIs externas
- Procesar lógica de negocio compleja

8.1.2. Procesamiento de Lenguaje Natural

spaCy

spaCy proporciona capacidades avanzadas de NLP en español:

- **Tokenización:** Separación del texto en palabras y signos de puntuación
- **Lematización:** Reducción de palabras a su forma base
- **Análisis morfológico:** Identificación de categorías gramaticales
- **Vectores de palabras:** Representaciones numéricas del significado de las palabras

El modelo `es_core_news_md` incluye vocabulario específico del español y ha sido entrenado en corpus de textos en español.

Coincidencia difusa (Fuzzy Matching)

La librería `RapidFuzz` implementa algoritmos de distancia de cadenas que permiten:

- Tolerar errores ortográficos (e.g., "Fundamnetos de Programacion")
- Reconocer abreviaturas (e.g., "FP" por "Fundamentos de Programación")
- Manejar variaciones en la escritura (e.g., mayúsculas, tildes)

Se utiliza el algoritmo de *Levenshtein distance* con un umbral de similitud del 80 % para considerar coincidencias válidas.

8.2– Tecnologías de Inteligencia Artificial

8.2.1. Ollama y modelos de lenguaje grandes (LLMs)

Ollama es una plataforma que permite ejecutar modelos de lenguaje de gran tamaño localmente, sin necesidad de conexión a servicios cloud. En LinceUS Assistant se utiliza para dos propósitos principales:

Text-to-SQL

El sistema implementa un pipeline de generación de consultas SQL a partir de lenguaje natural:

1. El usuario formula una pregunta en lenguaje natural
2. Ollama (usando un modelo como Llama 3 o Mistral) genera la consulta SQL correspondiente
3. La consulta se valida y ejecuta contra la base de datos
4. Los resultados se procesan y formatean para presentarlos al usuario

Este enfoque permite consultas complejas sin necesidad de definir explícitamente todas las posibles preguntas.

Generación de respuestas contextualizadas

Para consultas que no se ajustan a patrones predefinidos, el modelo LLM genera respuestas naturales basándose en:

- El contexto de la conversación
- La información recuperada de la base de datos
- El conocimiento previo del modelo

8.2.2. Embeddings y búsqueda semántica

Para la recuperación de información a partir de los planes docentes oficiales, se utiliza:

- **Modelos de embeddings:** Transforman texto en vectores numéricos que capturan el significado semántico
- **pgvector:** Extensión de PostgreSQL que permite almacenar y buscar vectores eficientemente
- **Búsqueda de similitud:** Encuentra documentos relevantes usando distancia coseno entre vectores

Este enfoque implementa un sistema RAG (*Retrieval-Augmented Generation*) que mejora la precisión de las respuestas al combinar búsqueda semántica con generación de lenguaje.

8.3– Tecnologías de base de datos

8.3.1. PostgreSQL

PostgreSQL es el sistema de gestión de bases de datos relacional utilizado. Se ha elegido por:

- **Robustez:** Base de datos ACID con alta integridad de datos
- **Extensibilidad:** Soporte para tipos de datos personalizados y extensiones
- **Rendimiento:** Optimización de consultas complejas y soporte para índices avanzados
- **Código abierto:** Sin costes de licencia y comunidad activa

8.3.2. Supabase

Supabase proporciona una capa de servicios sobre PostgreSQL:

- **API REST automática:** Generación automática de endpoints para operaciones CRUD
- **Real-time:** Suscripciones a cambios en la base de datos
- **Autenticación:** Sistema de usuarios y permisos integrado
- **Almacenamiento:** Para archivos y documentos
- **Funciones Edge:** Ejecución de código serverless

La API REST de Supabase simplifica la integración con el backend de Python, eliminando la necesidad de escribir queries SQL manualmente para operaciones básicas.

8.3.3. pgvector

La extensión pgvector añade capacidades de búsqueda vectorial a PostgreSQL:

- Almacenamiento eficiente de vectores de alta dimensionalidad (hasta 16,000 dimensiones)
- Índices IVFFlat y HNSW para búsquedas rápidas de vecinos más cercanos
- Operadores de similitud (distancia euclidiana, producto punto, similitud coseno)

Esto permite implementar búsqueda semántica directamente en la base de datos, sin necesidad de sistemas externos como Pinecone o Weaviate.

8.4– Tecnologías de frontend

8.4.1. Node.js

El frontend del asistente está desarrollado usando Node.js, que proporciona:

- **JavaScript del lado del servidor:** Mismo lenguaje en frontend y backend de la interfaz
- **NPM:** Gestor de paquetes con millones de librerías disponibles
- **Rendimiento:** Motor V8 de alta velocidad

8.4.2. Framework web

Se utiliza un framework minimalista que permite:

- Servir la interfaz de chat web
- Gestionar la conexión WebSocket con el backend de Rasa
- Renderizar mensajes con formato enriquecido (tablas, enlaces, imágenes)

8.5– Herramientas de desarrollo

8.5.1. Control de versiones

Git es el sistema de control de versiones utilizado, con **GitHub** como plataforma de hosting. La estructura del repositorio incluye:

- `/actions`: Acciones personalizadas de Rasa
- `/data`: Datos de entrenamiento (intents, stories, rules)
- `/models`: Modelos entrenados de Rasa
- `/frontend`: Código de la interfaz web
- `/docs`: Documentación del proyecto

8.5.2. Gestión de dependencias

Las dependencias de Python se gestionan mediante:

- **requirements.txt**: Lista de todas las librerías necesarias con versiones específicas
- **Entorno virtual (venv)**: Aislamiento de dependencias del sistema

8.5.3. Variables de entorno

La librería `python-dotenv` gestiona la configuración mediante archivos `.env`:

- Credenciales de Supabase (URL, API key)
- Configuración de Ollama (URL del servidor, modelo a utilizar)
- Parámetros de configuración del sistema

Esto permite cambiar la configuración sin modificar el código y evita exponer credenciales en el repositorio público.

8.6– Consideraciones de seguridad

8.6.1. Protección de datos

- **Credenciales:** Nunca se almacenan en el código, siempre en variables de entorno
- **API keys:** Se utilizan claves específicas con permisos mínimos necesarios
- **Conexiones:** HTTPS para todas las comunicaciones con servicios externos

8.6.2. Validación de entrada

- Sanitización de consultas SQL para prevenir inyección SQL
- Validación de tipos de datos en entradas de usuario
- Limitación de longitud de mensajes

8.6.3. Ejecución local de modelos

El uso de Ollama para ejecutar modelos localmente aporta beneficios de privacidad:

- Las consultas de los usuarios no se envían a servicios externos
- No hay dependencia de APIs comerciales que puedan cambiar términos de uso
- Control total sobre los datos procesados

8.7– Stack tecnológico completo

La siguiente tabla resume todas las tecnologías empleadas en el proyecto:

Capa	Tecnología	Versión
Frontend	Node.js	18.x
Frontend	HTML/CSS/JavaScript	ES6+
Backend conversacional	Python	3.10
Backend conversacional	Rasa	3.6.21
Backend conversacional	Rasa SDK	3.6.2
Backend conversacional	spaCy	3.8.0
IA y LLMs	Ollama	latest
IA y LLMs	Modelos LLM	Llama 3 / Mistral
Base de datos	PostgreSQL	15.x
Base de datos	pgvector	0.5.0
Plataforma BD	Supabase	cloud
Librerías NLP	RapidFuzz	3.0.0
Librerías NLP	python-dotenv	1.0.0
Control versiones	Git & GitHub	2.x

Cuadro 8.1: Stack tecnológico completo del proyecto LinceUS Assistant.

Estructura del proyecto

Este capítulo describe la organización del repositorio LinceUS Assistant, detallando el propósito de cada directorio y fichero relevante. La estructura ha sido diseñada para separar claramente las responsabilidades entre los distintos módulos del sistema y facilitar el mantenimiento y la escalabilidad del proyecto.

9.1– Visión general del repositorio

El repositorio sigue una organización modular en la que cada directorio agrupa elementos con una responsabilidad concreta dentro del sistema. La separación entre la lógica conversacional, los datos de entrenamiento, la capa de acceso a datos, la interfaz de usuario y la documentación responde al principio de separación de responsabilidades (*Separation of Concerns*).

El árbol de directorios principal es el siguiente:

```
linceus-assistant/  
+-- actions/  
|   +-- __init__.py  
|   +-- actions.py  
|   +-- asignaturas.py  
|   +-- config.py  
|   +-- contexto.py  
|   +-- db.py  
|   +-- gemini_client.py  
|   +-- ollama_client.py  
|   \-- text_to_sql.py  
+-- data/  
|   +-- nlu/  
|   |   +-- asignaturas.yml  
|   |   +-- contexto.yml  
|   |   \-- general.yml  
|   +-- rules.yml  
|   \-- stories.yml  
+-- docs/  
|   +-- sprints/  
|   \-- other/  
+-- frontend/  
|   +-- chatbot-widget.js  
|   +-- chatbot-icon.html  
|   +-- pagina-principal.html  
|   \-- pagina-principal.css
```



```
+-- memoria_tfg/  
+-- tests/  
|   |-- test_stories.yml  
+-- config.yml  
+-- domain.yml  
+-- credentials.yml  
+-- endpoints.yml  
|-- requirements.txt
```

9.2– Directorio raíz

Los ficheros situados en la raíz del repositorio configuran el comportamiento global del sistema Rasa y la gestión de dependencias del proyecto.

- `config.yml`: Define el pipeline de procesamiento de lenguaje natural (NLU) y las políticas de diálogo de Rasa. Especifica los componentes utilizados para tokenización, extracción de características e identificación de intenciones y entidades.
- `domain.yml`: Declara el universo del asistente: intenciones, entidades, slots, acciones y respuestas predefinidas. Es el fichero central que Rasa utiliza para construir el modelo de diálogo.
- `credentials.yml`: Contiene las credenciales de conexión con los canales de comunicación externos (REST, Slack, etc.). Las claves sensibles se gestionan mediante variables de entorno para no exponerlas en el repositorio.
- `endpoints.yml`: Configura los endpoints de los servicios externos con los que se comunica Rasa, como el servidor de acciones personalizadas (*action server*).
- `requirements.txt`: Lista todas las dependencias de Python necesarias para ejecutar el proyecto, con versiones fijas para garantizar la reproducibilidad del entorno.

9.3– Directorio `actions/`

El directorio `actions/` contiene la lógica de negocio del asistente, implementada como acciones personalizadas de Rasa. Cuando el motor conversacional determina que debe ejecutar una acción, invoca el código Python correspondiente de este directorio a través del *action server*.

9.3.1. Organización de los módulos

Cada fichero encapsula una responsabilidad específica:

- `actions.py`: Punto de entrada principal. Define las acciones de Rasa (`ActionQueryAsignaturas`, `ActionFallback`, etc.) y orquesta las llamadas al resto de módulos según la intención detectada.
- `asignaturas.py`: Módulo especializado en la consulta de información sobre asignaturas. Implementa la lógica de búsqueda por nombre, curso, tipología y otras características, integrando coincidencia difusa (*fuzzy matching*) para tolerancia a errores ortográficos.
- `db.py`: Capa de acceso a datos. Centraliza todas las operaciones contra la base de datos Supabase (PostgreSQL), abstrayendo las consultas SQL del resto de la lógica de negocio.

- `config.py`: Carga y expone la configuración del sistema a partir de variables de entorno. Gestiona credenciales de Supabase, parámetros de Ollama y otros ajustes globales.
- `contexto.py`: Gestiona la memoria de contexto de la conversación. Mantiene información sobre la sesión del usuario (asignatura mencionada previamente, curso, etc.) para responder de forma coherente a preguntas de seguimiento.
- `text_to_sql.py`: Implementa el pipeline de generación de consultas SQL a partir de lenguaje natural mediante un modelo de lenguaje grande (LLM). Transforma preguntas del usuario en consultas SQL válidas para su ejecución contra la base de datos.
- `gemini_client.py`: Cliente de integración con la API de Gemini (Google). Gestiona las llamadas al modelo para la detección dinámica de intenciones y la generación de respuestas enriquecidas.
- `ollama_client.py`: Cliente de integración con Ollama para la ejecución local de modelos de lenguaje grande. Permite realizar inferencias sin depender de servicios externos en la nube.

9.3.2. Flujo de ejecución de una acción

Cuando el asistente recibe una consulta, el flujo típico dentro del directorio `actions/` es el siguiente:

1. `actions.py` recibe la petición del *action server* de Rasa.
2. Se extrae la intención y las entidades identificadas del tracker de Rasa.
3. Se consulta `contexto.py` para recuperar información de la sesión activa.
4. Se delega en el módulo específico (`asignaturas.py`, etc.) que ejecuta la consulta a través de `db.py`.
5. Si la consulta requiere generación de lenguaje natural, se invoca `gemini_client.py` u `ollama_client.py`.
6. La respuesta se devuelve a Rasa para ser enviada al usuario.

9.4– Directorio `data/`

El directorio `data/` contiene todos los datos de entrenamiento del modelo Rasa. Estos ficheros en formato YAML definen el comportamiento conversacional del asistente.

9.4.1. Subdirectorio `nlu/`

El subdirectorio `nlu/` agrupa los ficheros de entrenamiento del componente de comprensión del lenguaje natural (NLU), organizados temáticamente:

- `asignaturas.yml`: Ejemplos de entrenamiento para intenciones relacionadas con consultas sobre asignaturas: créditos, tipología, curso, duración, etc.
- `contexto.yml`: Ejemplos para intenciones de seguimiento contextual, como referencias anafóricas («¿y esa asignatura?», «¿cuántos créditos tiene?»).
- `general.yml`: Intenciones generales del asistente: saludo, despedida, afirmación, negación, solicitud de ayuda y gestión del fallback.

9.4.2. Ficheros de flujo conversacional

- `stories.yml`: Historias de entrenamiento que definen conversaciones de ejemplo. Rasa Core aprende de estas secuencias para predecir la siguiente acción del bot ante nuevas interacciones.
- `rules.yml`: Reglas determinísticas de alto nivel. A diferencia de las historias, las reglas se aplican siempre que se cumpla la condición, sin aprendizaje estadístico. Se utilizan para comportamientos críticos como el manejo del fallback o la finalización de formularios.

9.5– Directorio `frontend/`

El directorio `frontend/` contiene la interfaz web del asistente. Está diseñado como un widget embebible que puede integrarse en cualquier página web institucional.

- `chatbot-widget.js`: Componente principal del chat. Implementa la lógica de comunicación con el servidor Rasa mediante la API REST, gestiona el estado de la conversación en el cliente y renderiza los mensajes del asistente con soporte para texto enriquecido.
- `chatbot-icon.html`: Página de demostración que muestra el widget integrado con el icono flotante característico del asistente.
- `pagina-principal.html` y `pagina-principal.css`: Prototipo de la página principal de LinceUS, que sirve como entorno de prueba de la integración del chatbot en un contexto universitario real.

9.6– Directorio `data/` de Rasa vs. `docs/`

El directorio `docs/` almacena la documentación técnica del proyecto, organizada en sprints y documentos de referencia:

- `sprints/`: Carpetas por sprint con los documentos de planificación, decisiones técnicas y resultados de cada iteración del desarrollo.
- `other/`: Documentos de referencia transversales, como el esquema de la base de datos, investigaciones sobre tecnologías alternativas y planes de implementación.

9.7– Directorio `tests/`

El directorio `tests/` contiene los casos de prueba del sistema conversacional.

- `test_stories.yml`: Historias de prueba end-to-end siguiendo el formato nativo de Rasa para testing conversacional. Cada historia simula una conversación completa y verifica que el asistente responde con las acciones correctas en cada turno.

La estrategia de pruebas se complementa con scripts externos ubicados en `docs/other/`, que cubren pruebas de integración del pipeline Text-to-SQL y del módulo de contexto conversacional.

9.8– Separación de responsabilidades

La estructura del repositorio refleja una separación clara entre los distintos módulos del sistema, lo que aporta las siguientes ventajas:

- **Mantenibilidad:** Cada módulo puede modificarse de forma independiente sin afectar al resto del sistema, siempre que se respete la interfaz entre capas.
- **Testabilidad:** Los módulos de acceso a datos y de clientes de IA pueden probarse de forma aislada mediante mocks.
- **Escalabilidad:** Añadir soporte para nuevas épicas (por ejemplo, ampliar el dominio a otros centros de la Universidad de Sevilla) implica crear nuevos ficheros en `actions/` y ampliar los datos en `data/nlu/`, sin modificar la arquitectura existente.
- **Colaboración:** La organización modular facilita el trabajo paralelo en distintas partes del sistema.

Estructura de la base de datos

Este capítulo describe el diseño y la estructura de la base de datos de LinceUS Assistant. La base de datos constituye el núcleo del sistema, almacenando toda la información académica necesaria para responder a las consultas de los estudiantes.

10.1– Diseño conceptual

El diseño de la base de datos se ha estructurado siguiendo el modelo relacional, organizando la información en entidades que representan los elementos principales del dominio universitario: universidades, centros, titulaciones, asignaturas, profesores y horarios.

10.1.1. Principios de diseño

El esquema de base de datos se ha diseñado siguiendo los siguientes principios:

1. **Normalización:** Las tablas están normalizadas hasta la tercera forma normal (3NF) para evitar redundancia y mantener la integridad referencial
2. **Escalabilidad:** El diseño permite añadir nuevas universidades, centros y titulaciones sin modificar la estructura
3. **Flexibilidad:** Campos de metadatos en formato JSONB permiten almacenar información variable sin alterar el esquema
4. **Búsqueda semántica:** Tablas de chunks con vectores de embeddings para búsqueda basada en similitud
5. **Trazabilidad:** Campos de timestamp (created_at, updated_at) en todas las tablas principales

10.2– Modelo entidad-relación

A continuación se detallan las principales entidades y sus relaciones del sistema.

10.3– Entidades principales

10.3.1. Universidades

La tabla `UNIVERSIDADES` almacena la información básica de cada universidad. Aunque el sistema está diseñado inicialmente para la Universidad de Sevilla, la estructura permite incorporar múltiples instituciones.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
codigo	varchar	Código de la universidad (e.g., "US")
nombre	varchar	Nombre completo de la universidad
nombre_corto	varchar	Nombre abreviado
dominio_web	varchar	Dominio web oficial (e.g., "us.es")
ciudad	varchar	Ciudad donde se ubica
activa	boolean	Indica si está operativa
created_at	timestampz	Fecha de creación del registro
updated_at	timestampz	Fecha de última actualización

Cuadro 10.1: Estructura de la tabla UNIVERSIDADES

10.3.2. Centros

Los centros representan las escuelas y facultades de cada universidad (e.g., ETSII, Facultad de Química).

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
universidad_id	uuid	Referencia a UNIVERSIDADES (FK)
codigo	varchar	Código del centro (e.g., "ETSII")
nombre	varchar	Nombre completo del centro
nombre_corto	varchar	Nombre abreviado
direccion	varchar	Dirección física del centro
telefono	varchar	Teléfono de contacto
email	varchar	Email de contacto
web	varchar	URL del sitio web del centro
activo	boolean	Indica si está operativo

Cuadro 10.2: Estructura de la tabla CENTROS

10.3.3. Departamentos

Los departamentos son unidades organizativas que agrupan profesores e imparten asignaturas.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
universidad_id	uuid	Referencia a UNIVERSIDADES (FK)
codigo	varchar	Código del departamento
nombre	varchar	Nombre completo
siglas	varchar	Siglas del departamento (e.g., "LSI")
email	varchar	Email de contacto
web	varchar	URL del sitio web
activo	boolean	Indica si está operativo

Cuadro 10.3: Estructura de la tabla DEPARTAMENTOS

10.3.4. Titulaciones

Las titulaciones representan los grados, másteres y doctorados ofrecidos por los centros.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
centro_id	uuid	Referencia a CENTROS (FK)
codigo	varchar	Código interno (e.g., "GII-IS")
codigo_oficial	varchar	Código RUCT oficial
nombre	varchar	Nombre completo de la titulación
nombre_corto	varchar	Nombre abreviado
tipo	varchar	GRADO, MASTER o DOCTORADO
plan_estudios_anio	int	Año del plan de estudios vigente
creditos_totales	int	Créditos ECTS totales
duracion_anios	int	Duración en años
requisito_idioma	varchar	Nivel de idioma requerido (e.g., "B1")
activa	boolean	Indica si está vigente

Cuadro 10.4: Estructura de la tabla TITULACIONES

10.3.5. Asignaturas

Las asignaturas son las materias que componen cada titulación.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
titulacion_id	uuid	Referencia a TITULACIONES (FK)
departamento_id	uuid	Referencia a DEPARTAMENTOS (FK)
codigo	varchar	Código de la asignatura
nombre	varchar	Nombre oficial de la asignatura
curso	int	Curso en el que se imparte (1-4)
creditos	decimal	Créditos ECTS
duracion	varchar	A (anual), C1 (1er cuatr.), C2 (2º cuatr.)
tipologia	varchar	TRONCAL, OBLIGATORIA, OP-TATIVA
es_formacion_basica	boolean	Si es de formación básica
es_optativa	boolean	Si es optativa
nombre_normalizado	varchar	Nombre sin tildes ni mayúsculas
palabras_clave	text[]	Array de palabras clave
activa	boolean	Indica si está vigente

Cuadro 10.5: Estructura de la tabla ASIGNATURAS

El campo `nombre_normalizado` facilita la búsqueda tolerante a errores, mientras que `palabras_clave` permite búsquedas por términos relacionados.

10.3.6. Profesores

La tabla de profesores almacena la información del personal docente.

10.3.7. Horarios

Los horarios relacionan grupos de clase con aulas, profesores y franjas horarias.

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
departamento_id	uuid	Referencia a DEPARTAMENTOS (FK)
nombre	varchar	Nombre del profesor
apellidos	varchar	Apellidos del profesor
nombre_completo	varchar	Nombre completo (generado)
nombre_normalizado	varchar	Versión normalizada para búsqueda
email	varchar	Email de contacto
telefono	varchar	Teléfono de contacto
despacho	varchar	Número de despacho (e.g., "F1.45")
edificio	varchar	Edificio donde se ubica
planta	varchar	Planta del despacho
web_personal	varchar	URL de página personal
orcid	varchar	Identificador ORCID
activo	boolean	Indica si está en activo

Cuadro 10.6: Estructura de la tabla PROFESORES

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
grupo_id	uuid	Referencia a GRUPOS_CLASE (FK)
aula_id	uuid	Referencia a AULAS (FK)
profesor_id	uuid	Referencia a PROFESORES (FK)
dia_semana	int	Día de la semana (1-5)
hora_inicio	time	Hora de inicio de la clase
hora_fin	time	Hora de fin de la clase
fecha_inicio	date	Fecha de inicio del periodo
fecha_fin	date	Fecha de fin del periodo
notas	text	Observaciones
activo	boolean	Indica si está vigente

Cuadro 10.7: Estructura de la tabla HORARIOS

10.4– Tablas de vectorización (RAG)

Para implementar el sistema RAG (*Retrieval-Augmented Generation*), se han diseñado tablas especializadas que almacenan fragmentos de documentos junto con sus representaciones vectoriales.

10.4.1. Planes docentes

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
asignatura_id	uuid	Referencia a ASIGNATURAS (FK)
curso_academico	varchar	Curso académico (e.g., "2025-26")
grupo	varchar	Grupo al que corresponde
url_documento	varchar	URL del PDF original
hash_documento	varchar	Hash SHA256 del documento
fecha_documento	date	Fecha del documento
coordinador_nombre	varchar	Nombre del coordinador
estado_rag	varchar	pendiente, procesando, completado, error
fecha_procesamiento	timestampz	Fecha de procesamiento
error_procesamiento	text	Mensaje de error si lo hay
updated_at	timestampz	Última actualización

Cuadro 10.8: Estructura de la tabla PLANES_DOCENTES

10.4.2. Chunks de planes docentes

Campo	Tipo	Descripción
id	uuid	Identificador único (PK)
plan_docente_id	uuid	Referencia a PLANES_DOCENTES (FK)
contenido	text	Texto del fragmento
embedding	vector	Vector de 768 dimensiones
seccion	varchar	Sección del documento (e.g., "Evaluación")
subseccion	varchar	Subsección si aplica
numero_pagina	int	Número de página del PDF
orden_chunk	int	Orden del chunk en el documento
metadata	jsonb	Metadatos adicionales en formato JSON
updated_at	timestampz	Última actualización

Cuadro 10.9: Estructura de la tabla PLANES_DOCENTES_CHUNKS

El campo `embedding` utiliza el tipo `vector` proporcionado por la extensión `pgvector`. Los vectores se generan usando modelos de embeddings especializados y permiten búsquedas de similitud semántica mediante consultas como:

```
SELECT contenido, embedding <=> query_vector AS distance
FROM planes_docentes_chunks
ORDER BY distance
```

```
LIMIT 5;
```

10.5– Relaciones entre entidades

Las principales relaciones del modelo son:

- Una **universidad** tiene múltiples **centros** y **departamentos**
- Un **centro** ofrece múltiples **titulaciones** y tiene múltiples **aulas**
- Una **titulación** contiene múltiples **asignaturas**
- Una **asignatura** es impartida por un **departamento** y tiene múltiples **planes docentes** y **grupos de clase**
- Un **departamento** agrupa a múltiples **profesores**
- Un **profesor** imparte múltiples **asignaturas** y está asignado a **horarios**
- Un **grupo de clase** tiene un conjunto de **horarios** asociados
- Un **horario** relaciona un **grupo**, un **aula** y un **profesor**

10.6– Índices y optimización

Para garantizar un rendimiento óptimo en las consultas, se han creado los siguientes índices:

- **Índices B-tree**: En todas las claves primarias y foráneas
- **Índices de texto**: En campos `nombre_normalizado` para búsquedas rápidas
- **Índices GIN**: En campos de array (`palabras_clave`) y JSONB (`metadata`)
- **Índices IVFFlat**: En campos `embedding` para búsquedas de similitud vectorial

Ejemplo de creación de índice vectorial:

```
CREATE INDEX idx_planes_chunks_embedding
ON planes_docentes_chunks
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

10.7– Integridad referencial

Todas las relaciones entre tablas están protegidas por restricciones de clave foránea con políticas de borrado apropiadas:

- **ON DELETE CASCADE**: En relaciones donde el hijo no tiene sentido sin el padre (e.g., `chunks` sin `plan docente`)
- **ON DELETE RESTRICT**: En relaciones donde se debe prevenir el borrado accidental (e.g., no borrar una universidad con centros asociados)
- **ON DELETE SET NULL**: En relaciones opcionales donde el padre puede desaparecer sin eliminar al hijo

10.8– Estrategia de datos de prueba

Para el desarrollo y pruebas del sistema se han creado datos de prueba que incluyen:

- Universidad de Sevilla con varios centros (ETSII, Facultad de Informática)
- Titulaciones de Ingeniería Informática (IS, IC, TI)
- Conjunto representativo de asignaturas de cada titulación
- Profesores del Departamento de Lenguajes y Sistemas Informáticos
- Horarios del curso 2025-2026
- Planes docentes oficiales de las asignaturas vectorizados en el corpus RAG

Los datos de prueba permiten validar el funcionamiento del sistema sin necesidad de tener acceso a datos reales completos.

Implementación

Este capítulo describe cómo se han traducido las decisiones de diseño en código operativo. La descripción no se centra en el detalle de cada función, sino en el *pipeline de ejecución* que sigue cada épica desde que el usuario envía un mensaje hasta que el asistente devuelve una respuesta. Cada épica se presenta con un diagrama del flujo y una explicación de los puntos clave: cómo se extraen los datos del mensaje, qué papel juegan la base de datos y el modelo de lenguaje, y qué información se persiste para mantener el contexto en turnos posteriores.

El código se organiza en una carpeta `actions/` con un módulo por épica (`asignaturas`, `profesores`, `horarios`, `contexto`, `fallback`, `multi_intent`) y una carpeta `shared/` con utilidades transversales: conexión a Supabase (`db.py`), cliente de Gemini (`gemini_client.py`), *fuzzy matching* de nombres (`matching.py`) y configuración de alias (`config.py`). Los pipelines descritos a continuación se apoyan en estos módulos compartidos.

11.1– Anatomía común de una *custom action*

Antes de entrar en cada épica, conviene fijar el patrón al que responden todas las acciones del proyecto. Una *custom action* de Rasa es una clase Python que hereda de `Action` y expone un método `run(dispatcher, tracker, domain)`. Dentro de ese método, todas las épicas siguen una misma secuencia de cinco pasos.

1. **Validación de contexto:** si la consulta requiere titulación y el slot `contexto_titulacion` está vacío, la acción interrumpe el flujo y muestra los botones de selección de titulación. Esta comprobación está centralizada en `shared/follow_up.py` para evitar duplicarla en cada épica.
2. **Extracción y normalización:** se leen las entidades NLU del último mensaje y los slots persistentes. Los nombres se normalizan con NFKD (eliminación de tildes), paso a minúsculas y limpieza de partículas (*de*, *del*, *la*, *info*). Los acrónimos (*FP*, *ADDA*, *DPI*) se expanden contra el diccionario `ALIAS_ASIGNATURAS` de `shared/config.py`.
3. **Resolución:** con el dato extraído se busca en la base de datos. Cuando hay varias coincidencias se aplica *fuzzy matching* (`rapidfuzz`) para descartar falsos positivos. En consultas que no se pueden expresar como filtro estructurado (por ejemplo, contenido del plan docente), se delega en el pipeline RAG.
4. **Generación de respuesta:** los datos recuperados se entregan al modelo Gemini junto con un prompt específico de la épica. Gemini se encarga de redactar la respuesta en lenguaje natural respetando un límite de 1500 caracteres. Si la llamada falla, cada épica dispone de una plantilla *hardcoded* de respaldo.
5. **Persistencia de contexto:** la acción devuelve una lista de eventos `SlotSet` para guardar lo que pueda ser relevante en turnos siguientes (último código consultado, último profesor,

curso o grupo) y la marca `ultima_action_ejecutada`, que el sistema de seguimiento usa para discriminar consultas de tipo “hay más”.

Los diagramas que siguen utilizan tres colores para diferenciar el tipo de operación: **teal** para pasos de procesamiento sobre datos del usuario, **rojo** para llamadas al modelo de lenguaje y **naranja** para accesos a base de datos.

11.2– Épica de asignaturas

La épica de asignaturas concentra cuatro acciones: la consulta específica sobre una asignatura concreta (`ActionConsultaEspecificica`), el listado filtrado (`ActionConsultaListado`), el conteo (`ActionConsultaConteo`) y la consulta de horario por nombre de asignatura (`ActionConsultaHorario`). La consulta específica es la más compleja y articula la lógica de resolución de nombre que reutilizan el resto de épicas.

11.2.1. Consulta específica de asignatura

La acción `ActionConsultaEspecificica` responde a preguntas del tipo “¿cuántos créditos tiene Redes?” o “¿cómo se evalúa Ingeniería del Software II?”. La Figura 11.1 muestra su pipeline.

El pipeline tiene dos ramas según la naturaleza de la pregunta. Una pregunta sobre datos estructurales (créditos, curso, tipología, duración) se resuelve con un `SELECT` sobre la tabla `asignaturas`. Una pregunta sobre contenido del plan docente (evaluación, temario, metodología, bibliografía) requiere recuperación vectorial: la consulta se transforma en *embedding*, se buscan los *chunks* más similares en `plan_docente` mediante `pgvector` y esos *chunks* se entregan a Gemini como contexto para que redacte la respuesta.

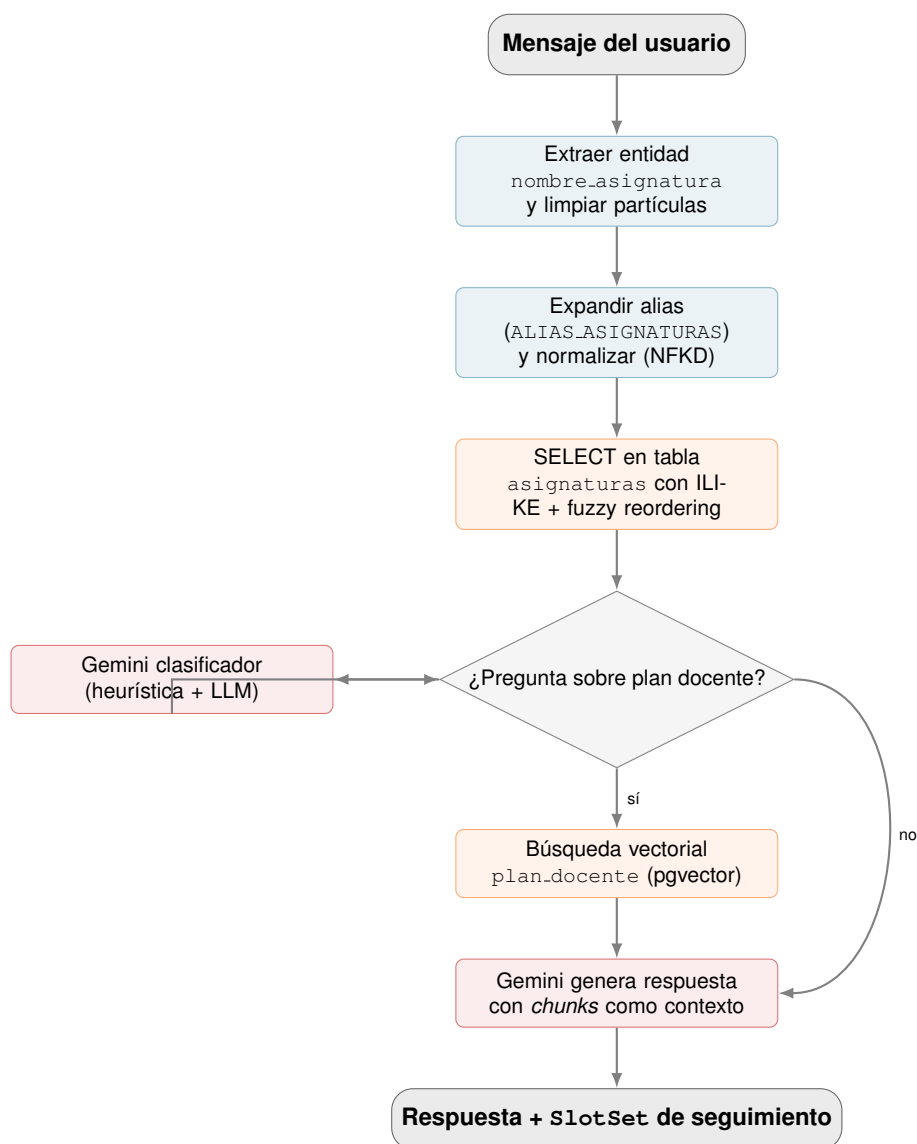
La decisión entre las dos ramas se toma con un clasificador en dos pasos. Primero se aplica una heurística rápida que busca palabras clave (*evalúa*, *temario*, *bibliografía*, *competencias*, *metodología*). Si la heurística no es concluyente, se invoca a Gemini con un prompt específico de clasificación binaria a temperatura cero. Esta arquitectura evita llamadas innecesarias al LLM en los casos triviales y mantiene la precisión cuando la frase es ambigua.

La resolución del nombre de asignatura merece atención por sí misma. La función `resolver_asignatura()` aplica una cascada de cinco estrategias: entidad NLU directa, expansión de acrónimo, búsqueda por regex sobre alias conocidos, herencia desde el slot `ultimo_nombre_asignatura` si la consulta es elíptica (“¿y de Matemáticas?”) y, como último recurso, *fuzzy matching* con `rapidfuzz` sobre los nombres normalizados de la base de datos con un umbral de `partial_ratio` ≥ 85 . Cuando el `SELECT` devuelve más de un candidato, se aplica un reordenamiento adicional con `token_set_ratio` contra la pregunta original para evitar confundir *Administración* con *Ampliación de Administración*.

11.2.2. Listado y conteo de asignaturas

Las acciones `ActionConsultaListado` y `ActionConsultaConteo` responden a consultas agregadas: “dame las optativas de cuarto” o “¿cuántas asignaturas tiene primero?”. Su pipeline (Figura 11.2) se basa en una técnica *text-to-SQL*: se delega en Gemini la generación de la consulta SQL a partir del enunciado del usuario.

El SQL generado por el modelo no se ejecuta directamente. Pasa por un validador propio (`validar_sql`) que rechaza cualquier sentencia que no empiece por `SELECT`, que acceda a tablas no permitidas o que contenga patrones peligrosos (`UNION`, `OR 1=1`, `SLEEP`, modificación de datos). Tras la validación, un paso adicional inyecta automáticamente el filtro `titulacion_id` a partir del slot `contexto_titulacion` para que un usuario que ha elegido GII-IS no reciba

Figura 11.1: Pipeline de `ActionConsultaEspecific`.

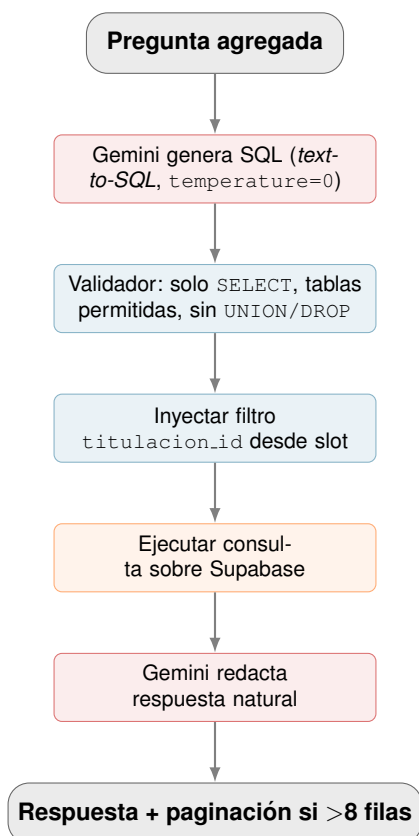


Figura 11.2: Pipeline de listado y conteo (*text-to-SQL*).

resultados de GII-TI. Solo entonces se ejecuta la consulta con `psycopg2` y se entrega el resultado a Gemini para la redacción final, que avisa explícitamente cuando la respuesta es una muestra paginada.

La acción de conteo es estructuralmente idéntica pero el prompt instruye al modelo para devolver `SELECT COUNT (*)`. Si la pregunta hace referencia a una asignatura específica con palabras propias del plan docente (“¿cuántos temas tiene FP?”), la heurística delega en `ActionConsultaEspecifica` en vez de generar un `COUNT`, porque la respuesta correcta proviene del documento, no de la base relacional.

11.2.3. Horario por asignatura

`ActionConsultaHorarioAsignatura` responde a “¿cuándo tengo Redes?” o “aula de ADDA grupo I”. Reutiliza la resolución de nombre de la consulta específica y añade detección de grupo y cuatrimestre. El cuatrimestre se infiere de la fecha actual (septiembre–enero implica primer cuatrimestre, febrero–julio el segundo) salvo que el usuario lo indique explícitamente. La consulta SQL hace `JOIN` entre `horarios`, `grupos_clase`, `asignaturas` y `aulas`; tras decisión D-068, cada aula es una fila distinta, lo que permite separar limpiamente sesiones de teoría y de laboratorio en el formateo final.

11.3– Épica de profesores

`ActionConsultaProfesor` responde a un abanico amplio de consultas sobre el profesorado: correo, despacho, departamento, profesores que imparten una asignatura concreta o, indirectamente, tutorías. La Figura 11.3 resume su flujo.

El pipeline también se apoya en *text-to-SQL*, pero con dos peculiaridades. La primera es la

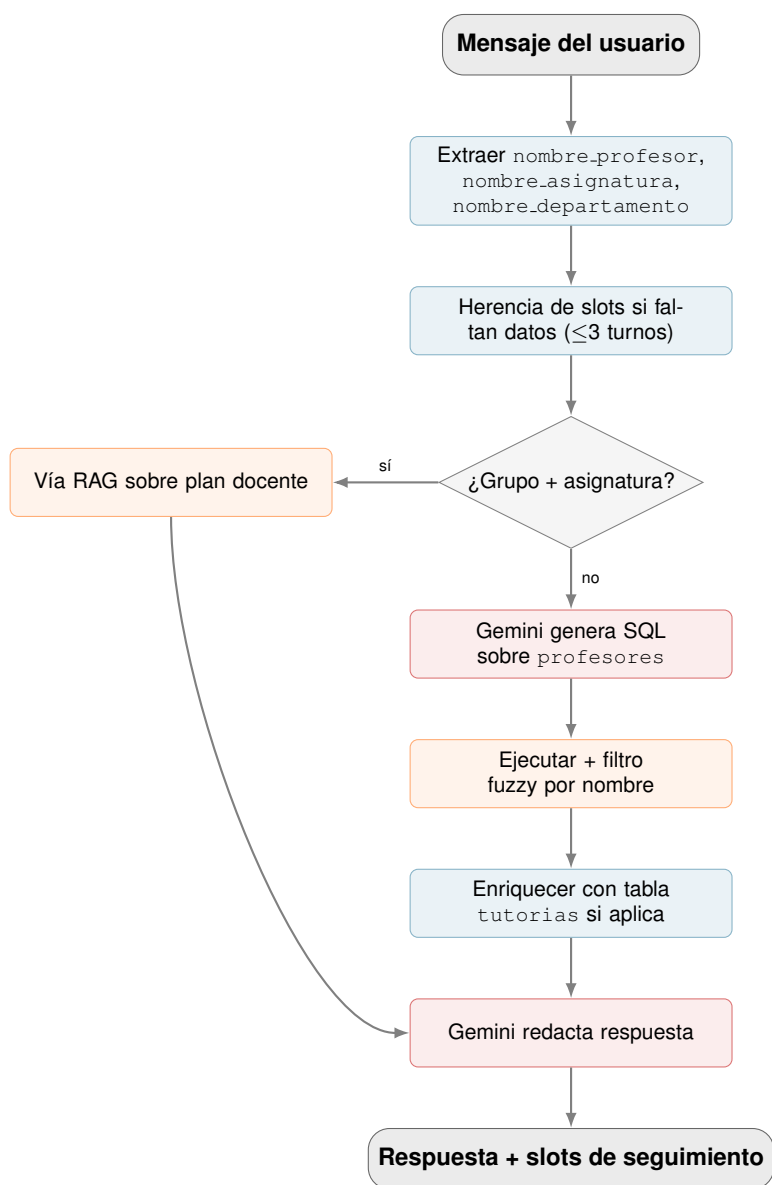


Figura 11.3: Pipeline de ActionConsultaProfesor.

herencia agresiva de contexto: si el usuario dice “¿y su correo?” sin nombrar a nadie, la acción lee `ultimo_profesor_consultado` y `ultimo_nombre_asignatura` de los últimos tres turnos para reconstruir la consulta. La segunda es el desvío hacia RAG cuando la pregunta combina asignatura y grupo, porque la tabla `profesor_asignatura` no tiene poblada la columna `grupo` (decisión Cat-2): la única fuente fiable de quién coordina o suplente en un grupo concreto es el plan docente, así que en ese caso se omite el SQL y se va directo al pipeline RAG.

Cuando el usuario pregunta por nombre, los resultados del `SELECT` se filtran con `shared/matching.py`: se calcula la proporción de tokens de la pregunta que aparecen en el `nombre_normalizado` de cada profesor y se separan en dos listas, firmes ($\text{umbral} \geq 0,60$) y sugerencias (entre 0,30 y 0,60). Si no hay coincidencias firmes pero sí sugerencias, la acción guarda la mejor en el slot `ultima_sugerencia` para que el usuario pueda confirmarla con un “sí” en el siguiente turno; este flujo de afirmación está implementado en `ActionResolverAfirmacion`.

Las tutorías merecen una nota: la tabla `tutorias` permanece deliberadamente vacía (decisión D-061) porque ningún portal oficial publica horarios estables. Cuando el usuario pregunta por tutorías, la respuesta redirige al correo institucional del profesor, que sí está en la base de datos.

11.4– Épica de horarios

`ActionConsultaHorario` responde a consultas por curso y grupo: “¿qué horario tiene 2º grupo 1?” o “¿qué clases hay los lunes en primero grupo 2?”. La Figura 11.4 muestra su pipeline.

El pipeline parsea con expresiones regulares cuatro elementos: curso (2º, *segundo*, *curso 2*), grupo (*grupo 1*, *g1*), día (lunes a viernes) y cuatrimestre (explícito o inferido por fecha). Si faltan curso o grupo, intenta heredarlos de los slots `ultimo_curso_consultado` y `ultimo_grupo_consultado` si fueron fijados en los últimos tres turnos. Existe además un seguimiento entre épicas: si la consulta no aporta ninguna pista de horario pero hay una asignatura reciente en el slot, la acción delega en `ActionConsultaHorarioAsignatura` en vez de pedir más datos.

La consulta SQL hace `JOIN` entre `horarios`, `grupos_clase`, `asignaturas`, `titulaciones` y `aulas`, filtrando por curso, grupo y, opcionalmente, día y cuatrimestre. El resultado se reagrupa por (*día*, *hora_inicio*, *hora_fin*, *asignatura*) para juntar las distintas aulas de una misma sesión, y se categoriza cada aula como teoría (prefijo *A*) o laboratorio. Solo entonces se entrega a Gemini con instrucciones para que respete la fecha actual (“*hoy*”, “*mañana*”) y advierta cuando la consulta cae en sábado o domingo.

11.5– Épica de contexto académico

La épica de contexto agrupa cuatro acciones cortas: `ActionCambiarContexto` fija la titulación del usuario, `ActionConsultarContexto` la lee, `ActionResetContexto` la borra y `ActionConsultaTitulaciones` muestra las disponibles. La pieza interesante es la primera, porque transforma una frase libre del usuario en uno de los códigos canónicos GII-IS, GII-TI o GII-IC.

La normalización se apoya en una combinación de coincidencia exacta sobre el diccionario `TITULACION_MAP`, un guard que exige al menos un token discriminante (*software*, *computadores*, *tecnologías*, *is*, *ti*, *ic*) y, si nada de lo anterior funciona, un *fuzzy matching* con `fuzz.WRatio` a un umbral conservador de 85. El umbral se elevó tras la revisión R3 para evitar que frases ambiguas como “*ingeniería*” produjeran cambios de contexto silenciosos. El resto de épicas dependen del valor de `contexto_titulacion` para inyectar el filtro `titulacion_id` en sus consultas, por lo que un fallo en la normalización contamina toda la sesión: de ahí el cuidado con los umbrales.

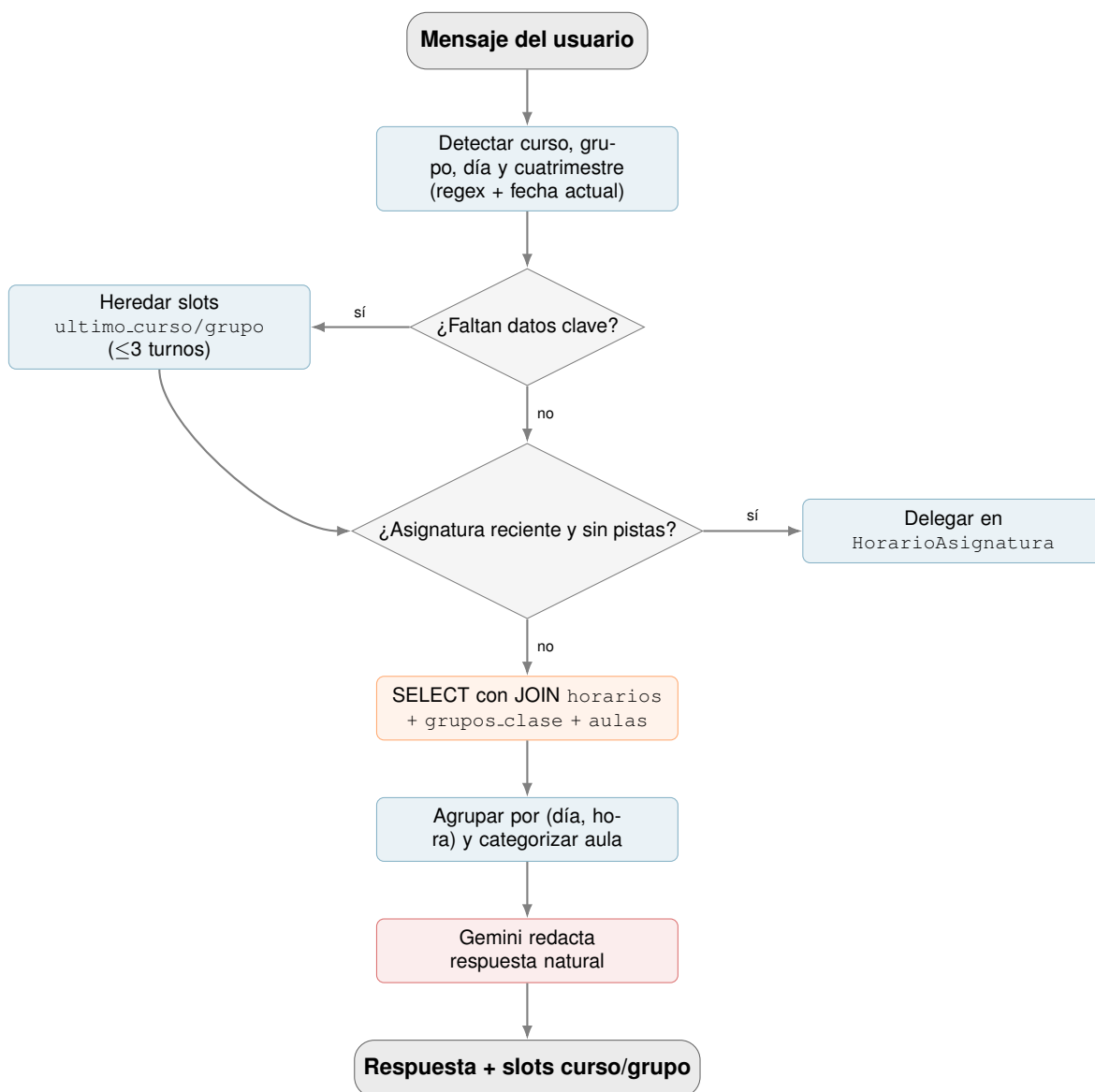


Figura 11.4: Pipeline de ActionConsultaHorario.

11.6– Épica de fallback inteligente

El fallback de Rasa se activa cuando la confianza del clasificador NLU está por debajo del umbral configurado en `config.yml`. En lugar de devolver un mensaje genérico de disculpa, `ActionSmartFallback` delega en Gemini la decisión de qué acción ejecutar a partir de la pregunta y del historial reciente. La Figura 11.5 muestra el flujo.

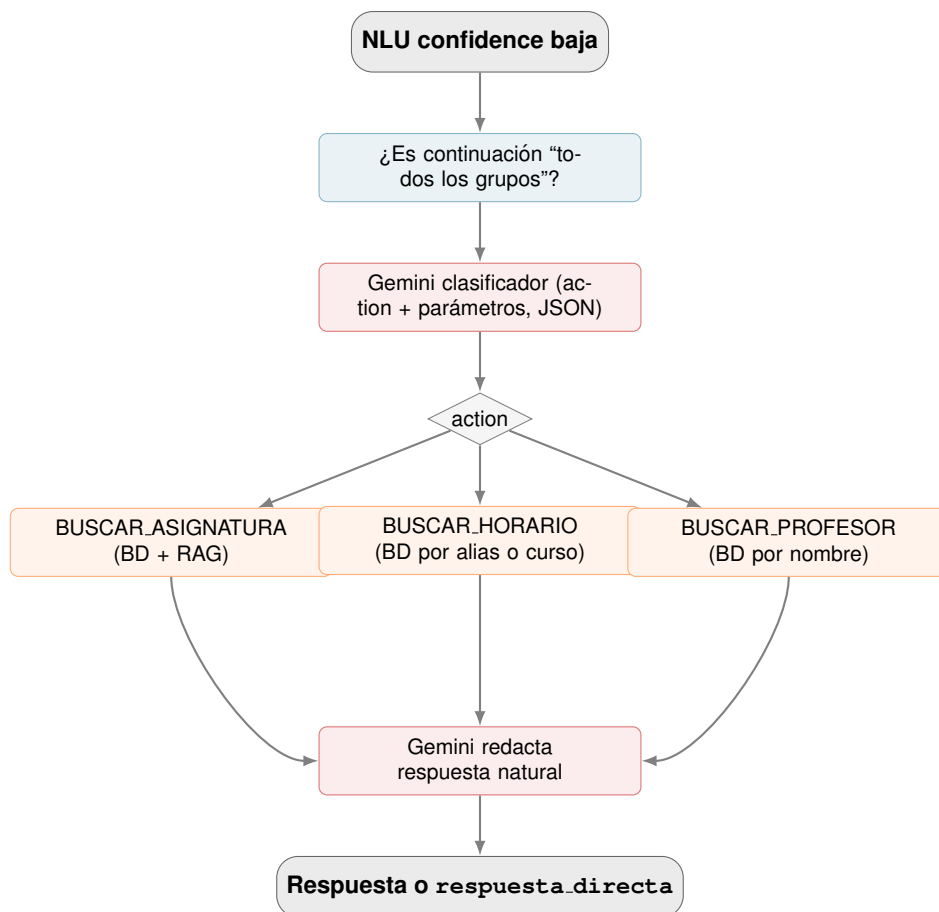


Figura 11.5: Pipeline de `ActionSmartFallback`.

El *prompt* de clasificación recibe la pregunta, el contexto de titulación, el último nombre de asignatura del tracker, los tres últimos pares de mensajes y la lista de acciones disponibles (`BUSCAR_ASIGNATURA`, `BUSCAR_HORARIO`, `BUSCAR_PROFESOR`, `NINGUNO`). Gemini devuelve un JSON con la acción elegida, sus parámetros y, si elige `NINGUNO`, una respuesta directa breve. La temperatura se fija a cero para que la clasificación sea determinista.

Antes de invocar al modelo, una regex detecta el patrón “*todos los grupos*” sobre el último horario consultado, porque es una continuación frecuente que no merece coste de LLM. Si la clasificación elige una acción de búsqueda, la implementación correspondiente vive como función dentro del propio módulo `fallback`: no se reinvoa la *custom action* de la épica, sino una versión simplificada que devuelve datos crudos para que un segundo *prompt* los redacte. Esta separación evita ciclos de Rasa y reduce la latencia.

11.7– Épica de *multi-intent*

`ActionMultiIntent` cubre los mensajes que combinan varias intenciones en una sola frase: “*soy de Ingeniería del Software, ¿qué horario tengo en segundo grupo 2?*” contiene a la vez un cambio de contexto y una consulta de horario. Rasa identifica este caso con un intent

compuesto del tipo `cambiar_contexto_academico+consulta_horario`, que se separa en sub-intents y se ejecutan en orden.

La ejecución es secuencial por una razón funcional: el cambio de contexto debe completarse antes de la consulta de horario, porque esta última lee `contexto_titulacion` para inyectar el filtro de titulación en su SQL. Cada sub-intent invoca un ejecutor que devuelve un diccionario con el tipo de resultado, los datos crudos y un texto preliminar; los textos se acumulan y se entregan a un último prompt de Gemini que pide integrarlos en un único mensaje natural. Si Gemini falla, la respuesta de respaldo es la concatenación directa de los textos preliminares.

11.8– Módulos compartidos

Tres módulos de `shared/` concentran la complejidad transversal del sistema y merecen mención explícita.

11.8.1. Cliente de Gemini

`shared/gemini_client.py` expone una función `llamar_gemini(prompt, modelo, timeout, options, context)` que envuelve el SDK oficial. El modelo por defecto es `gemma-3-27b-it`. La función centraliza los *timeouts*, los reintentos ante errores transitorios y, si la variable de entorno `LINCEUS_BENCH_METRICS` está activa, registra la latencia y el número de tokens de cada llamada en un fichero JSONL etiquetado con el campo `context` (“*asignaturas.text_to_sql*”, “*profesores.respuesta*”, etc.). Este registro fue clave durante la fase de optimización para identificar los prompts más caros y aplanar la curva de latencia.

11.8.2. Conexión a base de datos

`shared/db.py` define una clase `DatabaseConnection` que lee las credenciales de Supabase desde el `.env` y expone una instancia global `db_client`. Cada *custom action* obtiene un cursor a través de esta instancia, ejecuta su consulta parametrizada con `psycopg2` y la cierra. La parametrización (`cursor.execute(sql, parametros)`) es la barrera principal contra la inyección SQL en las épicas que usan *text-to-SQL*; el validador de `validar_sql` es la segunda.

11.8.3. Matching difuso de nombres

`shared/matching.py` implementa la función `score_por_normalizado(consulta, normalizado)` que devuelve un valor entre 0 y 1 según la proporción de tokens de la consulta presentes en el campo `normalizado` de la base de datos. Sobre esta función se construye `clasificar_por_normalizado()`, que separa una lista de resultados en firmes ($\text{umbral} \geq 0,60$) y sugerencias (entre 0,30 y 0,60). Los umbrales fueron calibrados durante las pruebas piloto: por debajo de 0,30 los falsos positivos contaminaban las respuestas y por encima de 0,60 se perdían matches válidos por errores ortográficos del usuario.

Parte IV

Validación y Cierre

Pruebas y Validación

Este capítulo recoge la estrategia de pruebas seguida durante el desarrollo de Linceus Assistant y los resultados obtenidos en la fase de validación previa al cierre del prototipo. La evaluación de un sistema conversacional basado en modelos de lenguaje plantea retos que no se resuelven con una única batería de pruebas: el comportamiento del bot depende del clasificador NLU, de las acciones que consultan la base de datos, del módulo de recuperación semántica y del modelo generativo que produce la respuesta final, de modo que un fallo puede originarse en cualquiera de esas capas. Por ese motivo, la validación se ha organizado en cinco capas independientes que aíslan riesgos distintos y, en conjunto, ofrecen una imagen razonablemente completa del estado del sistema.

El alcance de las pruebas cubre las tres épicas funcionales que conforman el prototipo (asignaturas, horarios y profesorado), el flujo transversal de cambio de titulación, la gestión de seguimiento conversacional y la respuesta del bot ante consultas fuera de ámbito o intentos de manipulación del prompt. Quedan fuera de este capítulo las pruebas de las importaciones del panel de administración (sincronización con Sevius, vectorización de documentos y carga de horarios), que se enmarcan como trabajo de la siguiente fase, y la validación subjetiva mediante encuesta SUS, que requiere una segunda iteración con usuarios reales.

12.1– Estrategia general

12.1.1. Marco metodológico

El plan de pruebas toma como referencia el modelo de calidad de producto software definido en la norma **ISO/IEC 25010:2023** [43], que recoge entre las actividades del ciclo de vida del software la identificación de criterios de aceptación. De las ocho características recogidas en el modelo se cubren cuatro de manera explícita: *adecuación funcional* (corrección y completitud), *capacidad de interacción* (protección frente a errores del usuario), *fiabilidad* (tolerancia a fallos) y *seguridad* (en lo relativo a la integridad de las importaciones, descritas en su propio plan). Las cuatro características restantes (rendimiento, compatibilidad, mantenibilidad y flexibilidad) se discuten cualitativamente en otros capítulos de esta memoria y no forman parte del cómputo de la suite.

La redacción de cada caso sigue la plantilla *Given–When–Then* propia de **Behavior-Driven Development** [44]. Esta plantilla traslada cada escenario al lenguaje natural del dominio y permite traducirlo de forma mecánica a un caso ejecutable por el *runner*: las precondiciones se materializan en *slots* de Rasa, la acción del usuario se corresponde con la consulta enviada al endpoint y el resultado esperado se descompone en el *intent* clasificado y en condiciones sobre el texto de la respuesta. Esta correspondencia, además de facilitar la revisión por parte del tribunal, agiliza la incorporación de nuevos casos a medida que aparecen errores en producción.

Sobre esta base, los escenarios se agrupan en tres categorías ampliamente aceptadas en la literatura sobre evaluación de chatbots [45, 46]: *positivos*, en los que el bot debe responder co-

rectamente al camino feliz; *negativos*, en los que la información solicitada no existe en la base de datos y el bot debe declararlo sin alucinar; y *escenarios de robustez*, que comprenden errores tipográficos, intentos de *jailbreak*, consultas fuera de dominio, reutilización de *slots* entre turnos y cambios de titulación durante la conversación. Dentro de la categoría de positivos se introduce además una subdivisión propia entre consultas *bien escritas* y consultas *con errores tipográficos*, dado que la robustez frente al ruido textual es un criterio diferenciador en el caso de uso real (alumnos escribiendo desde el móvil con teclado predictivo).

12.1.2. Política de exclusión por trabajo futuro

Durante la ejecución de la suite se han identificado varios casos cuya causa raíz no es un defecto puntual, sino una limitación de diseño asumida y documentada. Entre ellas destacan el atajo de coordinador y suplente resuelto exclusivamente por recuperación semántica (decisión D-064), el emparejamiento profesor–asignatura–grupo cuando la tabla intermedia no almacena el grupo, la distinción entre aulas de teoría y de laboratorio cuando la convención por letra inicial del aula falla, y la corrección difusa de erratas severas en nombres propios. Estos casos se han marcado como *trabajo futuro* y se excluyen del denominador del porcentaje de éxito, siguiendo una práctica habitual en evaluación de prototipos de investigación. La tabla detallada de exclusiones figura en el repositorio del proyecto, en `tests/results/testing_general.md`, y se referencia individualmente en cada apartado de este capítulo.

12.1.3. Ejecución y registro

La suite end-to-end se ejecuta mediante el *script* `tests/run_test_plan.py` con el modificador `--manual-review`, que captura el *intent* clasificado y la respuesta textual emitida por el bot pero deja la celda de veredicto vacía para revisión humana. La razón de evitar un veredicto automático es la alta variabilidad léxica de las respuestas generadas por el modelo: una misma consulta puede producir formulaciones distintas que igualmente cumplen el criterio de aceptación, lo que hace inviable una comparación de cadenas. El *runner* introduce una espera de cinco segundos entre casos para no exceder la cuota gratuita de la API de Gemini, y los resultados se vuelcan en formato Markdown y JSON, sobrescribiendo los anteriores; la trazabilidad histórica se conserva mediante el control de versiones del repositorio.

12.2– Ejecución funcional extremo a extremo

12.2.1. Alcance

La capa de aceptación funcional extremo a extremo es la prueba de mayor cobertura del proyecto, ya que ejercita el sistema con el *stack* completo en marcha: clasificador NLU, acciones personalizadas en Python, motor de Rasa, base de datos Supabase con extensión `pgvector`, módulo de recuperación semántica y modelo Gemini para la generación de la respuesta final. Su propósito es comprobar que las piezas funcionan acopladas correctamente bajo condiciones equivalentes a las de producción, no validar cada componente por separado.

La suite se compone de **159 casos** contables distribuidos en diez categorías que reflejan los flujos conversacionales del prototipo. La Tabla 12.1 resume los resultados por categoría tras la última iteración de re-ejecución, fechada el 26 de abril de 2026.

El cómputo arroja un **94,3 %** de éxito, calculado sobre los 150 casos válidos frente a los 159 contables, con 13 casos adicionales excluidos por la política descrita en el Apartado 12.1.2. Las categorías deterministas (conteo, listado, fuera de ámbito, especificación de asignaturas) alcanzan el 100 % gracias a que dependen únicamente de SQL y del clasificador. La categoría `profesor`, en cambio, presenta el porcentaje más bajo (86,0 %) porque concentra los casos con dependencia

Cuadro 12.1: Resultados por categoría de la suite E2E (revisión manual, 26/04/2026).

Categoría	PASS	FAIL	PEND	Vacío	Total	%
cambiar_contexto	12	1	0	0	13	92,3
conteo	10	0	0	0	10	100
cross_dominio	1	0	0	0	1	100
especifica	22	0	0	0	22	100
fuera_ambito	8	0	0	0	8	100
horario	26	0	1	0	27	96,3
horario_asignatura	18	0	0	0	18	100
listado	15	0	0	0	15	100
profesor	37	1	4	1	43	86,0
seguimiento_cross_intent	1	0	0	0	1	100
TOTAL	150	2	5	1	159	94,3

múltiple entre tablas (profesor, asignatura y grupo) y los flujos resueltos por recuperación semántica que no enriquecen su salida con datos estructurados.

12.2.2. Lectura frente a la literatura

El valor obtenido se sitúa por encima del umbral de *production-ready* señalado por Braun et al. [47] (igual o superior al 90 %) y queda dentro de la franja superior del *benchmark* BANKING77 reportado por Casanueva et al. [48] (entre el 87 % y el 93 %). En relación con los chatbots universitarios publicados en la última década, la cifra supera con claridad los resultados comunicados por AdmitBot (82 %) [21], EDUBOT (alrededor del 78 %) [22], Ranoliya (85 %) [23] y el sistema de Dibya y Sahoo (84 %) [24].

12.3– Recuperación aumentada por generación

El módulo RAG es el componente más sensible del sistema y, por tanto, ha sido evaluado en dos capas separadas: una capa *baseline*, que mide la capacidad de recuperación del sistema sobre un banco de preguntas genéricas, y una capa *de profundidad*, que comprueba la fidelidad de la respuesta frente a los planes docentes oficiales.

12.3.1. Capa baseline

La capa baseline está formada por un banco de **50 preguntas genéricas** sobre asignaturas (créditos, curso, cuatrimestre, optatividad, equivalencias y conteos). Cada pregunta se reformula en dos o tres variantes y se ejecuta manualmente con replay del bot para aislar el efecto de la recuperación semántica de las distintas acciones SQL. Sobre esta capa se obtiene un **94,0 %** de éxito (47 de 50), con tres errores aislados: una pregunta cuyo alias en la base de datos no coincidía con el nombre coloquial empleado en la consulta (E-T02), un caso de enrutado erróneo a la acción de cambio de contexto (E-T04) y una confusión entre el flujo de conteo y el de listado (C-P05). Los tres han sido recogidos en el cuaderno de fallos para su posterior corrección.

12.3.2. Capa de profundidad

La capa de profundidad evalúa el rendimiento del sistema sobre **74 preguntas reales** extraídas de los planes docentes oficiales de **19 asignaturas** repartidas en **cinco grupos** de la titulación de Ingeniería del Software. Las preguntas no son genéricas: exigen al RAG recuperar información concreta de secciones específicas del plan (profesorado por grupo, bloques temáticos con

sus horas, criterios y fórmulas de evaluación, composición de tribunales, idioma de impartición y contenidos de bloques). La cobertura por curso se recoge en la Tabla 12.2.

Cuadro 12.2: Cobertura de la capa de profundidad RAG por curso.

Curso / tipo	Asignaturas	Preguntas
1.º (formación básica)	5	25
2.º (obligatoria)	3	15
3.º (obligatoria)	4	16
4.º (obligatoria)	2	6
4.º (optativa)	5	12
Total	19	74

El veredicto se emite con un código de tres niveles: *OK* cuando la respuesta es correcta y completa, *PARCIAL* cuando es correcta pero le falta algún detalle del plan docente, y *FAIL* cuando es incorrecta o vacía. Los resultados son 68 OK, 5 PARCIAL y 1 FAIL, lo que da un **91,9 %** de respuestas correctas y completas en lectura estricta y un 98,6 % si se admite la categoría parcial. La cifra entra en la franja alta del modelo RAGAS de Es et al. [49] (faithfulness igual o superior a 0,85) y supera el rango de exact-match esperado para sistemas RAG de dominio cerrado descrito por Lewis et al. [30] (entre el 75 % y el 88 %).

12.4– Clasificación NLU y política de diálogo

La capa de NLU y política de diálogo evalúa los componentes deterministas de Rasa Core: clasificación de *intent* y selección de la acción siguiente dado el historial de *slots*. La prueba se ejecuta con la orden `rasa test core --fail-on-prediction-errors` sobre 44 *stories* que cubren los caminos felices de las tres épicas, los *stories* de seguimiento conversacional y los *stories* de *fallback*. La Tabla 12.3 recoge las métricas agregadas.

Cuadro 12.3: Resultados de la capa NLU y Policy.

Métrica	Valor
Conversaciones correctas	44 / 44
Acciones predichas correctamente	158 / 158
<i>Accuracy</i> (conversación)	1,000
<i>Accuracy</i> (acción)	1,000
Precisión / <i>recall</i> / F1 (<i>weighted</i>)	1,00 / 1,00 / 1,00
<i>Stories</i> fallidas	0

La cifra se interpreta con la cautela apropiada: el conjunto de *stories* se ha construido a partir del uso real esperado del bot en el prototipo, no a partir de un *benchmark* adversarial, por lo que el 100 % certifica que NLU y política no son cuello de botella en el resto de capas, pero no garantiza idéntico desempeño bajo distribuciones de tráfico distintas. Como prueba complementaria de no regresión se verificó que tras el reentrenamiento con 117 ejemplos NLU adicionales extraídos de la tabla `conversation_log` (modelo `linceus_v4_7_9.tar.gz`, 24 de abril de 2026) la política aprendida mantenía las 158 de 158 acciones correctas que arrojaba la corrida previa, lo que descarta una degradación inducida por la ampliación del corpus.

Frente al ámbito académico, este resultado supera el umbral de despliegue de Rasa establecido en el modelo DIET por Bocklisch et al. [29] (igual o superior al 85 % de F1 de *intent*) y se sitúa por encima del estado del arte reportado en CLINC150 [48].

12.5– Reproducción de tráfico real tras el despliegue

La capa de tráfico real es la única en la que la distribución de las consultas no está controlada por el evaluador. Tras el despliegue del prototipo en la infraestructura de DigitalOcean, el bot se puso a disposición de un grupo de alumnos de la ETSII a través del *widget* integrado en la interfaz web del proyecto. Las conversaciones generadas en ese piloto se almacenan automáticamente en la tabla `conversation_log` de la base de datos, junto con el *intent* clasificado, los *slots* activos y la respuesta del bot turno a turno.

12.5.1. Metodología de revisión

La validación del piloto se ha realizado revisando manualmente cada una de las 59 sesiones desde el panel de administración, replicando turno a turno el intercambio entre el alumno y el bot. Una sesión se marca como válida cuando todas las consultas relevantes se resuelven con datos correctos verificados contra los planes docentes, los horarios oficiales o el directorio de profesores; cuando el bot responde *no encontrado* en aquellos casos en que la información no estaba presente en la base de datos sin inventar contenido; o cuando una pregunta cae claramente fuera del alcance del prototipo y el flujo de *out_of_scope* la redirige correctamente.

12.5.2. Resultados

La Tabla 12.4 resume los datos agregados de la auditoría.

Cuadro 12.4: Resultados de la reproducción de tráfico real.

Métrica	Valor
Sesiones de usuarios reales	59
Sesiones validadas manualmente	58
Sesiones excluidas (trabajo futuro)	1
% de éxito	98,3
Total de turnos auditados	203
Turnos validados	201
Ventana de captura	01/04/2026 – 25/04/2026

La sesión excluida corresponde a una pregunta en la que el alumno describe un tema técnico (electrónica de sistemas embebidos) y solicita saber en qué titulaciones se cursa. El comportamiento esperado consistiría en una recuperación semántica seguida de un cruce con la tabla de titulaciones; el bot, en cambio, responde con un listado de asignaturas afines al tema sin completar el segundo paso. El caso queda documentado como X-P06 dentro del catálogo de trabajo futuro y motiva la incorporación de un nuevo flujo de *consulta_titulacion_por_tema* en la fase siguiente.

La cifra de 98,3 % supera el mínimo aceptable señalado por Casas et al. [50] (igual o superior al 70 %) y la franja *satisfactorio* de Følstad y Brandtzaeg [51] (entre el 80 % y el 85 %). Conviene notar, además, que esta capa es la más realista de las cinco evaluadas, dado que la distribución de las consultas no ha sido diseñada por el equipo de pruebas: refleja el tipo de uso espontáneo que cabe esperar de un alumno consultando el bot desde el móvil entre clases.

12.6– Métricas no funcionales: latencia y coste

Una validación funcional positiva no garantiza por sí sola que el sistema sea desplegable a la población completa de la facultad. Por ello, sobre la misma suite de la Sección 12.2 se han medido dos métricas no funcionales adicionales: la latencia extremo a extremo del bot y el coste por turno asociado a las llamadas al modelo Gemini. La medición se realiza durante una corrida con el modificador `--delay 10` sobre 151 turnos, capturando 216 llamadas al modelo. El consumo de

tokens de entrada se obtiene del campo `usage_metadata` del SDK; el consumo de salida se estima mediante la heurística estándar de cuatro caracteres por *token* en español, dado que el SDK de Google no expone `candidates_token_count` para la familia Gemma 3.

12.6.1. Latencia

Los percentiles de latencia se recogen en la Tabla 12.5. La mediana de 3,4 segundos es comparable a la observada en sistemas comerciales de chat asistido en horas de tráfico moderado. El percentil 95 de 10,5 segundos refleja la cola larga propia de las consultas que requieren recuperación semántica seguida de dos llamadas al modelo (*Text-to-SQL* y renderizado), concentradas mayoritariamente en la categoría `profesor`.

Cuadro 12.5: Latencia extremo a extremo (segundos).

Estadístico	Valor (s)
Mediana (p50)	3,4
Media	4,3
Percentil 90	9,2
Percentil 95	10,5
Máximo observado	14,1

12.6.2. Coste por turno

El coste se calcula aplicando las tarifas públicas equivalentes a Gemini Flash Lite (0,075 USD por millón de *tokens* de entrada y 0,30 USD por millón de *tokens* de salida) y un tipo de cambio de 0,92 EUR/USD. Sobre 216 llamadas medidas, el coste medio por turno se sitúa en **0,012 céntimos de euro**, con un coste mediano de 0,008 céntimos y un percentil 95 de 0,036 céntimos. El turno más caro de la suite alcanzó 0,064 céntimos.

Para extrapolar este coste al despliegue completo del prototipo en la ETSII (con una población estimada de 700 alumnos del Grado en Ingeniería Informática) se han considerado tres escenarios de intensidad de uso, recogidos en la Tabla 12.6. En el escenario más conservador, propio de un periodo lectivo normal, el coste anual queda por debajo de 9 euros; en el escenario pico, propio de los periodos de matrícula y exámenes, ronda los 50 euros anuales. En cualquiera de los tres casos, la cifra queda dos órdenes de magnitud por debajo de cualquier suscripción comercial equivalente, lo que respalda la viabilidad económica del despliegue.

Cuadro 12.6: Extrapolación del coste anual a 700 alumnos según intensidad de uso.

Escenario	Turnos/alumno/mes	Turnos/mes	Coste/mes	Coste/año
Conservador (lectivo)	8	5 600	0,67 EUR	8,06 EUR
Realista (uso medio)	20	14 000	1,68 EUR	20,16 EUR
Pico (exámenes)	50	35 000	4,20 EUR	50,40 EUR

12.7– Visión integrada y discusión

La Tabla 12.7 agrega los resultados de las cinco capas funcionales junto con su umbral académico de referencia. Las cinco superan dicho umbral, lo cual permite afirmar que el sistema no presenta un único punto de fallo concentrado y que los errores residuales son conocidos, aislados y gestionables como trabajo futuro.

La comparación con chatbots universitarios publicados refuerza esta lectura. Los cuatro sistemas tomados como referencia (AdmitBot, EDUBOT, Ranoliya y Dibya–Sahoo) reportan cifras

Cuadro 12.7: Visión integrada de las cinco capas de validación funcional.

Capa	N	PASS	%	Umbral	Veredicto
E2E (§12.2)	159	150	94,3	≥ 90 % [47]	Supera
RAG <i>baseline</i> (§12.3.1)	50	47	94,0	≥ 85 % [30]	Supera
RAG profundidad (§12.3.2)	74	68	91,9	≥ 85 % [49]	Supera
NLU + Policy (§12.4)	158	158	100	≥ 85 % [29]	Supera
Tráfico real (§12.5)	59	58	98,3	≥ 70 % [50]	Supera

de exactitud comprendidas entre el 78 % y el 85 %; Linceus Assistant supera con holgura ese rango en las tres capas comparables (94,3 % en E2E, 91,9 % en RAG de profundidad y 98,3 % en tráfico real). La distancia es, en cualquier caso, prudente: la metodología de evaluación, el corpus utilizado y la naturaleza del dominio difieren entre proyectos y no permiten una comparación estrictamente cuantitativa.

12.7.1. Cumplimiento del criterio de cierre

Se considera que la fase de validación del prototipo queda cerrada en el momento en que se cumplen, de manera simultánea, los siguientes criterios: las cinco capas funcionales superan su umbral académico de referencia, la metodología empleada está documentada y respaldada por bibliografía, los fallos residuales han sido identificados, categorizados y justificados, y se demuestra mediante medición directa que el coste y la latencia son compatibles con un despliegue real. Todos los criterios anteriores se cumplen a fecha de 26 de abril de 2026.

12.7.2. Limitaciones y trabajo futuro

La validación realizada no agota la evaluación posible del sistema. Quedan fuera de su alcance la medición de la experiencia subjetiva del usuario mediante una escala estandarizada como SUS, la evaluación de la disponibilidad sostenida del servicio en producción mediante un acuerdo de nivel de servicio, y la suite automática para los *endpoints* del panel de administración. Estos tres frentes se abordarán en una segunda iteración, junto con un conjunto de palancas técnicas identificadas durante la ejecución de las pruebas: la elección de modelo Gemini en función del tipo de llamada (clasificación, *Text-to-SQL* o renderizado natural), la sustitución de heurísticas por expresiones regulares por clasificadores ligeros basados en LLM, la introducción de una memoria caché de respuestas para las consultas más frecuentes, la instrumentación de métricas de monitorización en producción y la persistencia de la última intención específica como *slot* consultable para resolver el seguimiento conversacional entre sub-*intents*.

12.8– Pruebas automatizadas del panel de administración

El panel de administración se valida mediante una suite de pruebas automatizadas con **pytest** y el cliente de test integrado de Flask, ubicada en `tests/test_admin.py`. La suite se ejecuta con el comando `pytest tests/test_admin.py -v` y no requiere infraestructura adicional.

La estrategia combina tres bloques con objetivos distintos: los *tests de lectura* ejercitan los endpoints GET contra la base de datos de producción sin ningún efecto secundario; los *tests de validación* comprueban que los endpoints de escritura rechazan correctamente los parámetros inválidos antes de llegar a la base de datos; y los *tests con mock* verifican la lógica de extracción de Sevius sustituyendo el scraper por un doble de prueba, lo que permite cubrir los flujos de error y los invariantes de inserción sin modificar los datos de producción.

12.8.1. Resultados

La Tabla 12.8 recoge los 24 casos ejecutados el 26 de abril de 2026, junto con su clasificación y veredicto.

Cuadro 12.8: Resultados de la suite `test_admin.py` (pytest, 26/04/2026).

ID	Bloque	Qué verifica	Res.
A01	GET	Servidor activo y BD accesible (<code>/health</code>)	PASS
A02	GET	<code>/stats</code> devuelve las 10 claves con valores ≥ 0	PASS
A03	GET	<code>/centros</code> lista no vacía con campos requeridos	PASS
A04	GET	<code>/asignaturas</code> devuelve todas las asignaturas activas	PASS
A05	GET	<code>/asignaturas?titulacion_id</code> filtra correctamente	PASS
A06	GET	<code>/profesores</code> lista con campos de contacto	PASS
A07	GET	<code>/profesores/<id></code> devuelve perfil completo	PASS
A08	GET	<code>/profesores/<uuid-falso></code> devuelve 404	PASS
A09	GET	<code>/horarios?titulacion_id</code> devuelve franjas con campos completos	PASS
A10	GET	<code>/conversaciones</code> devuelve total y rows	PASS
A11	GET	<code>/conversaciones/sesiones</code> devuelve <code>session_id</code> y conteo	PASS
A12	GET	<code>/titulaciones?centro_id</code> devuelve titulaciones con conteos	PASS
A13	GET	<code>/horarios/extractores</code> lista centros con extracción soportada	PASS
A14	400	POST <code>/asignaturas/sync</code> sin body devuelve 400	PASS
A15	400	POST <code>/asignaturas/sync</code> sin <code>titulacion_id</code> devuelve 400	PASS
A16	400	POST <code>/asignaturas/sync</code> con UUID inexistente devuelve 404	PASS
A17	400	POST <code>/horarios/generar</code> sin <code>centro_id</code> devuelve 400	PASS
A18	400	POST <code>/horarios/generar</code> con UUID inexistente devuelve 404	PASS
A19	400	GET <code>/horarios/titulaciones</code> sin <code>centro_id</code> devuelve 400	PASS
A20	400	DELETE <code>/horarios</code> sin <code>centro_id</code> devuelve 400	PASS
A21	mock	Catálogo vacío de Sevius produce 404	PASS
A22	mock	Excepción en Sevius produce 502 con mensaje descriptivo	PASS
A23	mock	Asignaturas ya existentes no se reinsertan	PASS
A24	mock	INSERT invocado con <code>titulacion_id</code> correcto	PASS
Total			24/24

Los 24 casos pasan en 16,86 segundos. El bloque GET confirma la integridad de los datos en producción y la ausencia de regresiones en los endpoints de lectura. El bloque 400 verifica que la capa de validación rechaza las solicitudes malformadas antes de ejecutar cualquier operación en la base de datos. El bloque mock demuestra que la lógica de extracción de Sevius maneja correctamente los tres escenarios críticos: error de red, catálogo vacío y catálogo con elementos ya presentes, sin insertar duplicados en ningún caso.

Las operaciones de escritura destructiva (`horarios/generar` con `limpiar=true` y `DELETE /horarios`) quedan fuera de la suite por el riesgo de dejar la base de datos en estado inconsistente al no disponer de un entorno de *staging* aislado, tal como se documenta en la Sección 12.7.2.

Manuales

Un sistema de software no se considera completo si quienes han de utilizarlo, mantenerlo o evolucionarlo no disponen de instrucciones suficientes para hacerlo. Por ese motivo, este capítulo recoge tres manuales complementarios que cubren los tres perfiles de interacción previstos para Linceus Assistant. El **manual de usuario** se dirige al alumnado de la ETSII, principal destinatario del prototipo, y describe cómo formular consultas y aprovechar las funcionalidades disponibles. El **manual de despliegue** se dirige al personal técnico encargado de instalar y mantener el sistema, y detalla los requisitos, procedimientos y comprobaciones necesarias para poner en marcha la plataforma. El **manual de desarrollo**, por último, se dirige a los futuros estudiantes, becarios o investigadores que continúen este trabajo, y describe la organización del repositorio, las convenciones de codificación y el ciclo de incorporación de nuevos cambios.

Los tres manuales se basan en el estado del proyecto a fecha del cierre del prototipo (26 de abril de 2026). Cualquier divergencia con el repositorio actual debe consultarse en los ficheros `README.md` y `docker-compose.yml`, que son las referencias técnicas autoritativas y se mantienen actualizadas con los cambios del proyecto.

13.1– Manual de usuario

El destinatario de este manual es el alumnado de la ETSII que utiliza Linceus Assistant para resolver dudas académicas. El manual describe la interfaz, el modelo conversacional y los flujos de consulta disponibles, sin entrar en detalles técnicos de implementación.

13.1.1. Acceso a la aplicación

Linceus Assistant se ofrece como una aplicación web responsiva, accesible desde cualquier navegador moderno (Chrome, Firefox, Edge, Safari) tanto en ordenador como en dispositivo móvil. La página principal del prototipo muestra una breve descripción del proyecto y un *widget* de chat anclado en la esquina inferior derecha, que se despliega al hacer clic sobre el icono (Figura 13.1). No se requiere autenticación previa: la sesión se identifica internamente mediante un identificador anónimo generado por el navegador, lo que permite mantener el contexto durante la conversación sin recoger datos personales.

Tras desplegar el chat, el alumno encuentra un mensaje de bienvenida y, sobre él, un selector de titulación que sirve para fijar el contexto académico inicial (Figura 13.2). Por defecto, la titulación seleccionada es Grado en Ingeniería Informática – Ingeniería del Software (GII-IS), por ser la titulación con mayor cobertura informativa en el prototipo, pero el alumno puede cambiarla en cualquier momento, ya sea desde el selector o expresando el cambio en lenguaje natural durante la conversación (“cambia a tecnologías informáticas”, “soy de IC”).



Figura 13.1: Página principal del prototipo con el *widget* de chat cerrado.



Figura 13.2: *Widget* desplegado con mensaje de bienvenida y selector de titulación.

13.1.2. Modelo conversacional

El asistente está diseñado para responder a consultas formuladas en español natural. No es necesario emplear comandos ni una sintaxis específica: el sistema interpreta la intención del usuario a partir del texto libre y se apoya en el contexto de los turnos anteriores cuando la consulta es elíptica. Esto significa, por ejemplo, que tras preguntar por una asignatura concreta el alumno puede continuar con preguntas como “¿y cuántos créditos tiene?” sin necesidad de repetir el nombre.

El bot tolera errores tipográficos moderados, abreviaturas habituales (FP, ADDA, IISSI2, PSG2) y reformulaciones de la misma pregunta. Ante una consulta ambigua, el asistente solicita al usuario los datos que necesita, típicamente el curso o el grupo cuando la consulta afecta a horarios. Si la información solicitada no está disponible en la base de datos, el bot responde de forma explícita, sin inventar contenido. Las preguntas que caen fuera del ámbito académico (información meteorológica, peticiones generales no relacionadas con la universidad) se rechazan con un mensaje cortés y se redirige al usuario hacia el conjunto de funcionalidades soportadas.

13.1.3. Funcionalidades disponibles

El prototipo cubre tres dominios funcionales principales. La Tabla 13.1 resume los tipos de consulta soportados con un ejemplo representativo de cada uno.

Cuadro 13.1: Tipos de consulta soportados por el prototipo.

Dominio	Tipo de consulta	Ejemplo de pregunta
Asignaturas	Ficha individual	“¿Qué es ADDA?”
	Listado por criterio	“¿Qué optativas hay en cuarto?”
	Conteo	“¿Cuántas obligatorias tiene tercero?”
	Información del plan docente	“¿Cómo se evalúa PSG2?”
Horarios	Por curso y grupo	“Horario de segundo grupo 3”
	Por asignatura	“¿Cuándo es ADDA?”
	Por día concreto	“¿Qué clases hay el lunes?”
	Por aula o laboratorio	“Laboratorio de Álgebra grupo 1”
Profesorado	Datos de contacto	“Email de Galindo”
	Profesores de una asignatura	“¿Quiénes imparten IISSI2?”
	Coordinación	“¿Quién coordina IA?”
	Tutorías	“Tutorías de Bernárdez”

Más allá de estas funcionalidades primarias, el bot incorpora dos comportamientos transversales que conviene conocer. El primero es el *cambio de contexto académico*, ya mencionado, que permite consultar información de cualquiera de las tres titulaciones de la ETSII (Ingeniería del Software, Ingeniería de Computadores, Tecnologías Informáticas) cambiando dinámicamente la titulación activa. El segundo es el *seguimiento conversacional*, que permite formular consultas elípticas apoyadas en los turnos anteriores: tras preguntar por una asignatura, expresiones del tipo “¿y de qué curso es?”, “¿y los profesores?” ó “¿y el horario?” resuelven correctamente la referencia implícita. La Figura 13.3 muestra una sesión representativa con varias de estas capacidades encadenadas.

13.1.4. Recomendaciones de uso

Aunque el sistema admite formulaciones libres, la calidad de la respuesta mejora cuando el alumno aporta el contexto necesario en la propia consulta. Para horarios resulta útil indicar curso



Figura 13.3: Ejemplo de sesión con seguimiento conversacional y cambio de contexto.

y grupo desde el primer turno; para profesorado, el apellido completo o el nombre exacto reduce la ambigüedad cuando varios docentes comparten apellido. Las consultas demasiado abiertas (“cuéntame cosas de la carrera”) tienden a producir respuestas genéricas, ya que el bot no dispone de un único objetivo informativo al que dirigirse. Por último, el sistema no almacena el contenido de las conversaciones para fines comerciales y permite al usuario reportar respuestas incorrectas mediante el botón de *feedback* integrado en cada turno, lo cual contribuye a la mejora continua del prototipo.

13.1.5. Panel de administración

El sistema incluye un panel de administración accesible en `/admin.html`, protegido por autenticación, que permite gestionar los datos académicos del prototipo sin necesidad de acceder directamente a la base de datos. El panel se organiza en cinco secciones accesibles desde la barra de navegación superior.

La vista de inicio (Figura 13.4) muestra la jerarquía de centros y titulaciones disponibles. Desde ella se puede navegar a las asignaturas de cada titulación y consultar o actualizar su información.

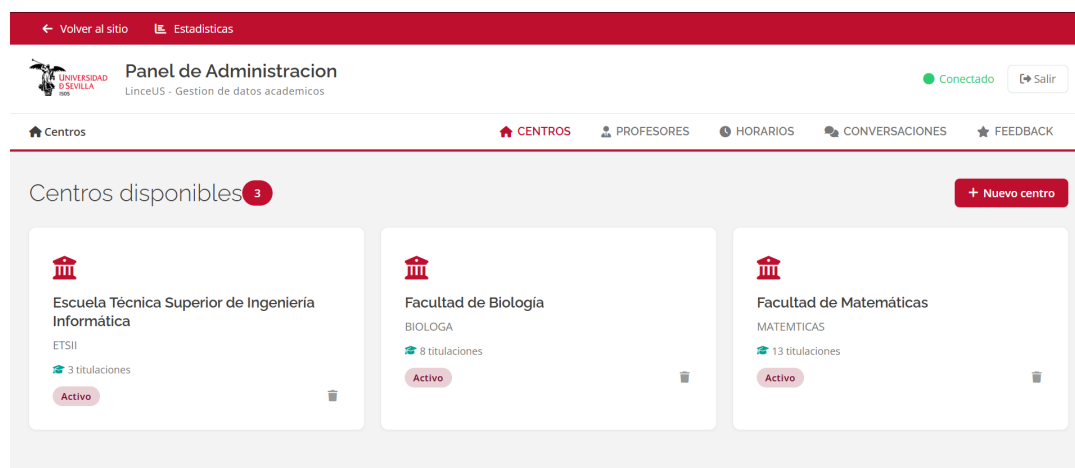
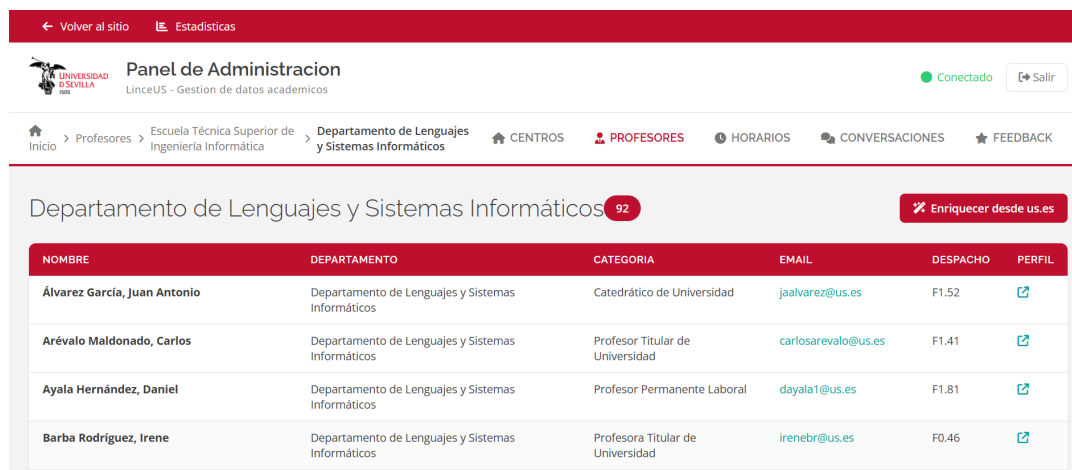


Figura 13.4: Panel de administración: vista de centros y titulaciones.

La sección de profesores (Figura 13.5) lista el profesorado con sus datos de contacto y permite actualizar la información directamente desde el panel, sin necesidad de re-entrenar el modelo.



NOMBRE	DEPARTAMENTO	CATEGORIA	EMAIL	DESPACHO	PERFIL
Álvarez García, Juan Antonio	Departamento de Lenguajes y Sistemas Informáticos	Catedrático de Universidad	jaalvarez@us.es	F1.52	Perfil
Arévalo Maldonado, Carlos	Departamento de Lenguajes y Sistemas Informáticos	Profesor Titular de Universidad	carlosarevalo@us.es	F1.41	Perfil
Ayala Hernández, Daniel	Departamento de Lenguajes y Sistemas Informáticos	Profesor Permanente Laboral	dayala1@us.es	F1.81	Perfil
Barba Rodríguez, Irene	Departamento de Lenguajes y Sistemas Informáticos	Profesora Titular de Universidad	irenebr@us.es	F0.46	Perfil

Figura 13.5: Panel de administración: gestión del profesorado.

La sección de horarios (Figura 13.6) muestra los horarios estructurados por titulación, curso y grupo, y expone los mismos datos que el chatbot consulta en tiempo real.



ASIGNATURA	DÍA	INICIO	FIN	AULA
Circuitos Electrónicos Digitales 2050003	Lunes	08:30:00	10:20:00	G1.32
Estructura de Computadores 2050009	Lunes	08:30:00	10:20:00	A0.12
Estructura de Computadores 2050009	Lunes	08:30:00	10:20:00	G1.32
Estructura de Computadores	Lunes	08:30:00	10:20:00	G1.35

Figura 13.6: Panel de administración: vista de horarios.

La sección de conversaciones (Figura 13.7) registra las sesiones de usuario de forma anonimizada, lo que permite detectar patrones de uso, preguntas frecuentes y casos en los que el bot no proporcionó una respuesta satisfactoria.

Por último, la vista de estadísticas (Figura 13.8) agrega métricas de uso del sistema: número de sesiones, distribución de intenciones detectadas y evolución temporal del tráfico.

13.2– Manual de despliegue

El manual de despliegue describe los pasos necesarios para instalar Linceus Assistant en una infraestructura nueva, ya sea un entorno de desarrollo local o un servidor de producción. La instalación se ha diseñado para ser reproducible mediante Docker Compose, lo que evita la necesidad de instalar manualmente el conjunto de dependencias del *stack* y reduce drásticamente las divergencias entre entornos.

13.2.1. Requisitos previos

La máquina destinataria debe cumplir los requisitos mínimos recogidos en la Tabla 13.2. Estos requisitos se han validado sobre el *droplet* de DigitalOcean utilizado durante el piloto y son suficientes para la carga prevista en la fase de prototipo (un orden de magnitud por debajo de la población total de la ETSII).

Cuadro 13.2: Requisitos mínimos del entorno de despliegue.

Recurso	Valor recomendado
Sistema operativo	Linux (Ubuntu 22.04 LTS o equivalente)
CPU	2 vCPU
Memoria RAM	4 GB
Almacenamiento	20 GB SSD
Docker Engine	24.x o superior
Docker Compose	v2 (incluido en Docker Engine)
Conectividad saliente	HTTPS hacia Supabase y Google AI Studio

Además de los recursos de cómputo, el despliegue requiere disponer de tres credenciales externas: la cadena de conexión a la base de datos Supabase con extensión `pgvector`, la clave de API de Google AI Studio para el modelo Gemini y un dominio o subdominio con certificado TLS si la instalación se realiza en producción. Estas credenciales se configuran en un fichero `.env` ubicado en la raíz del proyecto, cuyo formato se describe en `.env.example`.

13.2.2. Instalación

La instalación parte de la clonación del repositorio del proyecto y se completa en cuatro pasos. El primer paso consiste en obtener una copia local del código y situarse en el directorio resultante. El segundo paso es preparar el fichero `.env` a partir de la plantilla, sustituyendo cada variable por el valor correspondiente al entorno de despliegue. El tercer paso es asegurarse de que el directorio `models/` contiene un modelo Rasa entrenado: si no es así, basta con ejecutar `rasa train` en un entorno con las dependencias instaladas, o bien copiar el último modelo válido desde el repositorio. El cuarto y último paso es levantar la pila completa con Docker Compose:

```
1 git clone <url-repositorio> && cd linceus-assistant
2 cp .env.example .env # completar variables
3 docker compose -f docker/docker-compose.yml up --build -d
```

Código 13.1: Levantar la pila completa con Docker Compose.

La opción `-d` ejecuta los contenedores en segundo plano. El primer arranque construye las imágenes correspondientes, lo que puede tardar varios minutos en función del ancho de banda disponible para descargar las dependencias. A partir de ese momento, los rearranques posteriores son cuestión de pocos segundos, salvo que se hayan modificado las dependencias de Python.

13.2.3. Arquitectura del despliegue

La pila se compone de cuatro contenedores que se comunican entre sí a través de la red interna de Docker, recogidos en la Tabla 13.3. Únicamente el contenedor de *frontend* expone el puerto 80 al exterior; el resto de servicios son accesibles solo desde el interior de la red, salvo que el operador decida exponerlos explícitamente para fines de depuración.

El contenedor de *actions* monta los directorios `actions/` y `rag/` como volúmenes, lo que permite recargar los cambios en caliente sin reconstruir la imagen. El contenedor de Rasa monta los ficheros de configuración (`config.yml`, `domain.yml`, `credentials.yml`) y un fichero específico de *endpoints* que enruta el servidor hacia el contenedor de *actions* a través del nombre

Cuadro 13.3: Contenedores del despliegue y responsabilidad de cada uno.

Contenedor	Imagen base	Versión	Responsabilidad
frontend	nginx:alpine	alpine	Sirve los ficheros estáticos y actúa como proxy
rasa	rasa/rasa	3.6.21 + spaCy	Gestor del diálogo y NLU (puerto 5005)
actions	python:3.10.14-slim-bookworm	3.10.14	Acciones personalizadas y módulo RAG
admin	python:3.10-slim	3.10	API Flask del panel de administración

de servicio interno. El contenedor de *frontend* inyecta la variable `RASA_SERVER` en los ficheros estáticos en el momento del arranque mediante un *script* de *entrypoint*, lo que evita tener que recompilar el *frontend* para distintos entornos.

13.2.4. Verificación del despliegue

Una vez levantada la pila, la verificación del despliegue se realiza siguiendo cuatro comprobaciones en orden. En primer lugar, la página principal debe estar accesible en `http://<host>/`, donde `<host>` es la dirección o dominio del servidor. En segundo lugar, el panel de administración debe estar accesible en `http://<host>/admin.html` y permitir consultar la lista de centros y titulaciones. En tercer lugar, el *widget* de chat debe responder a un mensaje de prueba simple (“hola”) con un saludo personalizado, lo cual confirma que el camino completo entre *frontend*, Rasa, *actions* y modelo Gemini está operativo. Por último, una consulta sencilla con dependencia de base de datos (“¿cuántas asignaturas tiene primero?”) debe devolver una cifra coherente con los datos cargados, lo cual confirma la conectividad con Supabase.

Si alguna de estas comprobaciones falla, los registros de cada contenedor se consultan con la orden recogida en el Código 13.2, donde `<servicio>` es uno de los cuatro nombres recogidos en la Tabla 13.3. El error más habitual durante la primera puesta en marcha es la falta de un modelo entrenado en `models/`, lo que provoca que el contenedor de Rasa se reinicie en bucle.

```
1 docker compose -f docker/docker-compose.yml logs -f <servicio>
2 # Ejemplo: ver logs del servidor Rasa en tiempo real
3 docker compose -f docker/docker-compose.yml logs -f rasa
```

Código 13.2: Consultar registros de un contenedor.

13.2.5. Operaciones habituales

El día a día del operador se concentra en un puñado de tareas rutinarias. Los cambios en el código de *actions* o del módulo RAG no requieren reiniciar nada, ya que el *action server* se ejecuta con la opción `--auto-reload`. Los cambios en el modelo Rasa (re-entrenamiento) requieren detener y volver a arrancar el contenedor *rasa* para que recargue el modelo desde el volumen `models/`. Las actualizaciones de las dependencias de Python requieren reconstruir la imagen afectada antes de volver a levantarla. Las copias de seguridad de la base de datos se gestionan desde el panel de Supabase, que dispone de *snapshots* automáticos diarios; el código del proyecto no almacena estado mutable en disco fuera de la base de datos.

```
1 # Recargar modelo Rasa tras reentrenamiento
2 docker compose -f docker/docker-compose.yml restart rasa
3
4 # Reconstruir imagen tras cambio de dependencias Python
5 docker compose -f docker/docker-compose.yml build actions
6 docker compose -f docker/docker-compose.yml up -d actions
7
8 # Detener toda la pila
9 docker compose -f docker/docker-compose.yml down
```

Código 13.3: Operaciones de mantenimiento habituales.

13.3– Manual de desarrollo

El manual de desarrollo se dirige a quien deba modificar, ampliar o mantener el código de Linceus Assistant tras el cierre de este TFG. Su objetivo es minimizar la curva de aprendizaje proporcionando una visión panorámica del repositorio, las convenciones empleadas y el ciclo de trabajo recomendado.

13.3.1. Estructura del repositorio

El repositorio sigue la disposición habitual de un proyecto Rasa con extensiones específicas para el módulo RAG y el panel de administración. La Tabla 13.4 recoge los directorios de primer nivel y su responsabilidad.

Cuadro 13.4: Estructura del repositorio Linceus Assistant.

Directorio	Contenido
actions/	Acciones personalizadas (Python) que ejecuta el <i>action server</i>
admin/	Aplicación Flask del panel de administración
data/	Conjuntos de datos NLU y <i>stories</i> para Rasa
docker/	Ficheros de Compose y <i>Dockerfile</i> de cada servicio
frontend/	<i>Widget</i> de chat y panel HTML/CSS/JS
models/	Modelos Rasa entrenados (no versionados)
rag/	Módulo de recuperación aumentada por generación
tests/	Suite de aceptación funcional y planes de prueba

Los ficheros de configuración principales (`config.yml`, `domain.yml`, `credentials.yml`, `endpoints.yml` y `endpoints.docker.yml`) residen en la raíz, junto con los ficheros de dependencias `requirements-rasa.txt` y `requirements-actions.txt`. La documentación específica de cada componente se encuentra en su propio fichero `README.md`, mientras que el `README.md` de la raíz centraliza las instrucciones de arranque local y con Docker.

13.3.2. Entorno de desarrollo

El entorno de desarrollo recomendado se basa en Python 3.10 dentro de un entorno virtual. Tras clonar el repositorio, los pasos habituales son los del Código 13.4.

```
1 python3.10 -m venv .venv && source .venv/bin/activate
2 pip install -r requirements-rasa.txt -r requirements-actions.txt
3 cp .env.example .env # completar con claves reales
4 rasa train           # genera models/linceus_vX.Y.Z.tar.gz
```

Código 13.4: Preparación del entorno de desarrollo local.

A partir de ese momento, los cuatro componentes (Rasa, *action server*, panel y *frontend*) pueden levantarse en terminales independientes según se documenta en el `README.md` principal. Los comandos del Código 13.5 arrancan el flujo completo sin Docker.

```
1 # Terminal 1: servidor Rasa
2 rasa run --enable-api --cors "*" --port 5005 --endpoints endpoints.yml
3
4 # Terminal 2: action server (recarga automática al guardar)
5 rasa run actions --port 5055 --auto-reload
```

```

6
7 # Terminal 3: panel de administracion Flask
8 python -m admin.app
9
10 # Terminal 4: frontend (servir estaticos)
11 cd frontend && python -m http.server 8080

```

Código 13.5: Arranque de los componentes en modo desarrollo.

Durante el desarrollo, la opción `--auto-reload` del *action server* y la naturaleza estática del *frontend* permiten iterar rápidamente sobre la mayoría de los cambios. Los cambios en el modelo NLU o en el dominio de Rasa requieren un nuevo entrenamiento, que en una máquina convencional toma alrededor de uno o dos minutos. Los cambios en la estructura de la base de datos se gestionan a través del editor de tablas de Supabase y deben propagarse manualmente al fichero `schema.sql` de referencia.

13.3.3. Ciclo de incorporación de cambios

El proyecto sigue un flujo de trabajo basado en *feature branches* sobre la rama principal `main`. Cada cambio se desarrolla en una rama propia, se valida localmente mediante la suite de pruebas pertinente y se integra a través de una solicitud de incorporación que se revisa antes de fusionar. El ciclo se compone de cinco etapas, descritas en la Tabla 13.5.

Cuadro 13.5: Etapas del ciclo de incorporación de cambios.

Etapas	Descripción
1. Diseño	Definición del cambio en términos de requisito o defecto. Si el cambio afecta al NLU, se preparan ejemplos adicionales para <code>nlu.yml</code> .
2. Implementación	Modificación del código en la rama de trabajo, siguiendo las convenciones de estilo de Python (PEP 8) y de Rasa.
3. Pruebas locales	Ejecución de <code>rasa test core</code> para la política, <code>run-test-plan.py</code> para la suite E2E pertinente y revisión manual de los casos afectados.
4. Integración	Solicitud de incorporación a <code>main</code> con descripción de los cambios y de los casos de prueba afectados.
5. Despliegue	Reconstrucción de los contenedores afectados y verificación en el entorno de pruebas antes de promocionar a producción.

13.3.4. Cómo añadir una nueva intención

La incorporación de una nueva intención conversacional es la operación más frecuente durante la evolución del bot, y el procedimiento es suficientemente representativo como para servir de ilustración del ciclo completo.

El primer paso es declarar la intención en `domain.yml` y añadir entre quince y veinte ejemplos representativos en `data/nlu.yml`, procurando cubrir tanto formulaciones limpias como con errores tipográficos habituales. El Código 13.6 muestra la estructura de un bloque de ejemplos típico:

```

1 - intent: consulta_creditos_asignatura
2   examples: |
3     - cuantos creditos tiene [ADDA] (nombre_asignatura)
4     - cuantos creditos son de [Redes] (nombre_asignatura)

```

```

5 - creditos de [PSG2] (nombre_asignatura)
6 - de cuantos creditos es [IISSI2] (nombre_asignatura)
7 - [FP] (nombre_asignatura) cuantos creditos

```

Código 13.6: Declaracion de una intencion con ejemplos NLU en `data/nlu.yml`.

El segundo paso es definir, si procede, los *slots* y entidades que la intención necesita extraer; estos también se declaran en `domain.yml` y se entrenan automáticamente con el modelo. El tercer paso es escribir la acción personalizada en `actions/` si la intención requiere consultar la base de datos o el módulo RAG, registrándola tanto en el dominio como en `actions/__init__.py`. El Código 13.7 muestra la estructura mínima de una acción:

```

1 from rasa_sdk import Action, Tracker
2 from rasa_sdk.executor import CollectingDispatcher
3 from rasa_sdk.events import SlotSet
4
5 class ActionConsultaCreditos(Action):
6
7     def name(self) -> str:
8         return "action_consulta_creditos"
9
10    def run(self, dispatcher: CollectingDispatcher,
11            tracker: Tracker,
12            domain: dict) -> list:
13        nombre = tracker.get_slot("nombre_asignatura")
14        # ... consultar BD y generar respuesta
15        dispatcher.utter_message(text=f"{nombre} tiene 6 creditos ECTS.")
16        return [SlotSet("ultimo_nombre_asignatura", nombre)]

```

Código 13.7: Estructura minima de una accion personalizada en Rasa.

El cuarto paso es añadir las *stories* correspondientes en `data/stories.yml` para que la política aprenda a enrutar correctamente la nueva intención. El quinto paso es entrenar el modelo, validar la intención con casos manuales y, si la cobertura es razonable, añadir uno o dos casos de aceptación a `tests/run_test_plan.py` para garantizar que la nueva funcionalidad no se regresa en futuros cambios.

13.3.5. Buenas prácticas y convenciones

A lo largo del desarrollo se han adoptado un conjunto de convenciones que conviene mantener en futuras iteraciones. Las acciones personalizadas se nombran con el prefijo `action_` seguido del verbo principal del flujo (`action_consulta_profesor`, `action_cambiar_contexto`). Los *slots* cuya función es persistir la última entidad consultada llevan el prefijo `ultimo_` (`ultimo_nombre_asignatura`, `ultimo_profesor_consultado`). Los identificadores de los casos de prueba siguen un patrón de letra mayúscula seguido de tipo y número, donde la letra inicial alude a la categoría (E para asignaturas específicas, H para horarios, P para profesores, C para conteo, L para listado, X para cross-dominio, F y R para casos fuera de ámbito y *jailbreak*). Cada decisión de diseño no trivial se documenta como una entrada D-XXX en el fichero correspondiente del repositorio, lo que facilita reconstruir el porqué de un comportamiento al cabo del tiempo.

El Código 13.8 ilustra un patrón de diseño transversal al proyecto: la separación entre generación y validación. El módulo `text_to_sql` delega en Gemini la generación de SQL en lenguaje natural, pero antes de ejecutar cualquier *query* la pasa por una función de validación que rechaza instrucciones de escritura, tablas no autorizadas y columnas fuera del esquema permitido. Este patrón evita que un fallo en el LLM pueda comprometer la integridad de la base de datos.

```

1 # actions/asignaturas/text_to_sql.py
2 PALABRAS_PROHIBIDAS = [

```



```
3     'INSERT', 'UPDATE', 'DELETE', 'DROP', 'ALTER', 'CREATE',
4     'TRUNCATE', 'EXEC', 'UNION SELECT', 'OR 1=1', 'SLEEP(',
5 ]
6 TABLAS_PERMITIDAS = {'ASIGNATURAS', 'TITULACIONES'}
7
8 def validar_sql(sql: str, tipo: str = 'select') -> Optional[str]:
9     sql_upper = sql.upper().strip()
10    for palabra in PALABRAS_PROHIBIDAS:
11        if palabra in sql_upper:
12            return None      # rechazar silenciosamente
13    if not sql_upper.startswith('SELECT'):
14        return None
15    tablas = re.findall(r'FROM\s+(\w+)|JOIN\s+(\w+)', sql_upper)
16    tablas_flat = {t for grupo in tablas for t in grupo if t}
17    if tablas_flat - TABLAS_PERMITIDAS:
18        return None
19    return sql
```

Código 13.8: Fragmento de `validar_sql`: lista de palabras prohibidas y verificación de tabla.

Por último, conviene recordar que toda modificación que afecte al comportamiento del bot debe acompañarse de la actualización de los ejemplos NLU correspondientes y de un nuevo caso de aceptación. La experiencia acumulada durante el TFG demuestra que las regresiones más costosas de detectar son las que se producen en flujos colaterales tras un cambio aparentemente local; mantener la suite de aceptación actualizada es la principal salvaguarda frente a este tipo de error.

Conclusiones y desarrollos futuros

Este capítulo cierra la memoria del trabajo. Tras un proyecto extenso conviene levantar la vista del detalle técnico y revisar qué se planteó al inicio, qué ha quedado efectivamente construido y qué queda por hacer. La estructura del capítulo reproduce ese orden: se contrastan los objetivos enunciados en el Capítulo 1 con la evidencia recogida durante la validación, se recopilan las lecciones aprendidas a lo largo del desarrollo, se analiza la desviación de la planificación inicial, se enumeran las líneas de evolución previstas para una segunda iteración del sistema y se cierra con una reflexión personal sobre la experiencia.

14.1– Cumplimiento de objetivos

El Capítulo 1 planteaba un objetivo general y un conjunto de objetivos técnicos y académicos. Resulta razonable revisar uno por uno hasta qué punto se han alcanzado, apoyándose en la evidencia recopilada en el Capítulo 12.

14.1.1. Objetivo general

El objetivo general consistía en diseñar y desarrollar un asistente conversacional que ofreciera al alumnado de la Universidad de Sevilla un punto de acceso centralizado, accesible y eficiente a la información académica y administrativa. Tras el cierre del prototipo, este objetivo puede considerarse alcanzado en su versión correspondiente a la fase 1: el sistema cubre las tres épicas funcionales previstas (asignaturas, profesorado y horarios), responde correctamente al 94,3 % de los casos de la suite de aceptación funcional y al 98,3 % de las sesiones generadas por usuarios reales durante el piloto. La pieza queda desplegada en infraestructura de DigitalOcean y accesible mediante navegador, sin necesidad de credenciales, lo que satisface la condición de accesibilidad. Los grupos a los que la propuesta atendía con mayor prioridad (alumnado de nuevo ingreso e internacional) figuran entre los principales usuarios del piloto, lo que respalda la utilidad práctica del sistema.

14.1.2. Objetivos técnicos

La Tabla 14.1 cruza cada objetivo técnico definido en el Capítulo 1 con su evidencia en el Capítulo 12. Todos ellos se han cumplido en la fase 1, con dos matices: la interfaz web final no se ha desarrollado en React como contemplaba la propuesta inicial sino en JavaScript *vanilla* sobre HTML estático, decisión justificada por el menor coste de mantenimiento del prototipo, y el cumplimiento del RGPD se materializa por la naturaleza anónima de las sesiones más que por un despliegue *on-premise*, dado que la infraestructura final reside en DigitalOcean y la base de datos en Supabase.

Cuadro 14.1: Cruce entre objetivos técnicos y evidencia obtenida.

Objetivo técnico	Evidencia
Sistema conversacional Rasa con flujos contextuales	Modelo Rasa 3.6 entrenado, 158/158 acciones correctas en la prueba de política (§12.4).
Integración con Gemini	216 llamadas medidas en la suite, coste medio de 0,012 cts/turno (§12.6.2).
Base de datos híbrida en Supabase con pgvector	Esquema relacional + tabla de <i>embeddings</i> , capa RAG validada al 91,9 % de profundidad (§12.3.2).
Acciones personalizadas en Python	Tres épicas implementadas; 150/159 PASS en la suite E2E (§12.2).
Frontend accesible	Widget desplegado, validado mediante 59 sesiones reales (§12.5).
Cumplimiento RGPD	Sesiones anónimas, sin almacenamiento de datos personales identificables.

14.1.3. Objetivos académicos y personales

Desde el punto de vista formativo, el proyecto ha permitido aplicar de forma articulada conocimientos procedentes de varias asignaturas del Grado en Ingeniería del Software (bases de datos, ingeniería del software, sistemas inteligentes, procesamiento del lenguaje natural y despliegue de aplicaciones), y ha exigido autoaprendizaje sobre tecnologías que no formaban parte del currículo oficial, como el módulo RAG y la integración de modelos de lenguaje. La metodología SCRUM adaptada a un proyecto individual ha sido también una novedad relevante, dado que el grado expone fundamentalmente a equipos de cinco o seis personas. El trabajo ha quedado documentado, validado mediante una suite de pruebas reproducible y desplegado en producción, lo que cumple los tres criterios formativos planteados al inicio: aplicar, integrar y dejar trazabilidad del proceso.

14.2– Lecciones aprendidas

La experiencia acumulada durante el desarrollo deja un conjunto de lecciones que resulta razonable consignar. Las primeras son de naturaleza técnica y se han ido formulando como decisiones documentadas a lo largo del repositorio (entradas D-XXX). Las segundas son de naturaleza metodológica y afectan al modo en que el proyecto se ha organizado en el tiempo. Las terceras son de naturaleza personal.

En el plano técnico, la lección más relevante es que la combinación de *Text-to-SQL* y recuperación aumentada por generación cubre dominios distintos y conviene mantenerlas como caminos separados. *Text-to-SQL* resuelve con eficacia las consultas estructuradas (créditos, listados, conteos), mientras que el RAG es indispensable cuando la pregunta requiere fragmentos textuales no normalizados (criterios de evaluación, composición de tribunales, contenidos de bloques). El intento inicial de unificar ambas vías mediante un único *prompt* produjo respuestas confusas y obligó a separar acciones específicas para cada caso. Una segunda lección es que el clasificador NLU debe re-entrenarse con ejemplos extraídos del tráfico real una vez disponible, dado que las formulaciones espontáneas de los usuarios reales cubren un espacio léxico que el conjunto de *stories* sintéticas no anticipa.

En el plano metodológico, la introducción de la suite de pruebas en una etapa temprana del desarrollo ha resultado decisiva. La primera versión de la suite contaba con 51 casos sobre la épica de asignaturas y arrojaba un 70,6 % de éxito; el sistema actual evalúa 159 casos con un 94,3 % de éxito. Sin la suite, las regresiones introducidas por cada cambio habrían sido muy difíciles de detectar a ojo. La lección complementaria es que cada decisión de diseño no trivial conviene registrarla en el momento, no al final: cuando se trata de reconstruir el porqué de un comportamiento al cabo de un mes, la memoria del autor ya no basta.

En el plano personal, gestionar un proyecto individual de la magnitud de un TFG exige imponerse una cadencia regular de trabajo, especialmente durante los meses en los que coexiste con asignaturas. La adopción de *sprints* de tres o cuatro semanas ha funcionado bien como mecanismo de auto-disciplina, y el cierre de cada *sprint* con un incremento desplegable ha sostenido la motivación a lo largo del curso.

14.3– Análisis de desviación

La planificación inicial recogida en el Capítulo 5 estimaba el esfuerzo total del proyecto en torno a 300 horas, repartidas entre tres fases (Inicio, Desarrollo y Cierre) y un conjunto de paquetes de trabajo. La ejecución real ha respetado el orden y la estructura previstos, pero ha presentado desviaciones puntuales que conviene comentar.

La fase de *Inicio* ha consumido aproximadamente el esfuerzo previsto. La investigación tecnológica resultó algo más extensa de lo planeado por el tiempo dedicado a comparar modelos de lenguaje y a evaluar alternativas a Rasa, pero el diseño arquitectónico se completó dentro del margen estimado.

La fase de *Desarrollo* ha sido la más larga y la que más ha sufrido desviaciones positivas, en el sentido de que se ha invertido más esfuerzo del estimado en algunos paquetes. La época de profesorado, en concreto, requirió un ciclo adicional para resolver la combinatoria entre profesor, asignatura y grupo y para ajustar el atajo de coordinador y suplente, ambos bloqueos identificados durante el testing de aceptación. El *pipeline* RAG también consumió más tiempo del previsto, principalmente por la fase de afinado de los *prompts* de respuesta, que pasó por varias iteraciones hasta alcanzar la cifra del 91,9 % de profundidad.

La fase de *Cierre*, por contra, se ha mantenido dentro del margen previsto. El piloto con usuarios reales y la auditoría manual de las 59 sesiones se ejecutaron en la ventana planificada (1 al 25 de abril de 2026), y la redacción de la memoria comenzó tan pronto como las épicas estuvieron estabilizadas, lo que ha evitado una concentración excesiva del esfuerzo en las últimas semanas.

Como recomendación de la guía de referencia [42], las desviaciones se han mantenido siempre en sentido positivo: en ningún momento se ha invertido un volumen de horas inferior al estimado, lo que descartaría un cumplimiento meramente nominal del proyecto. La lección aprendida en este apartado es que la estimación inicial subestimaba sistemáticamente el coste de las tareas con dependencia de modelo de lenguaje (afinado de *prompts*, evaluación manual de respuestas), debido a la naturaleza no determinista de ese tipo de tareas.

14.4– Desarrollos futuros

El estado actual de Linceus Assistant corresponde a una fase 1, equivalente a un prototipo plenamente funcional pero con un alcance acotado a tres épicas y a la titulación GII-IS. El trabajo identificado para iteraciones posteriores se organiza en tres bloques: optimización del sistema actual, ampliación funcional y validación reforzada.

14.4.1. Optimización del sistema actual

A lo largo del Capítulo 12 se han identificado cinco palancas técnicas con potencial para reducir la latencia, abaratar el coste y mejorar la robustez del sistema sin alterar su funcionalidad esencial. La primera es la **elección de modelo Gemini en función del tipo de llamada**: las tareas de *Text-to-SQL* y de clasificación binaria pueden resolverse con modelos más pequeños y rápidos (familia Gemini Flash Lite o Gemma 3 de 4B parámetros), reservando modelos mayores para el renderizado natural de la respuesta. La estimación es que esta diferenciación reduciría a la mitad el coste anual y la cola larga de latencia.

La segunda palanca es la **sustitución de heurísticas por expresiones regulares por clasificadores ligeros basados en LLM**. Varias funciones del bot detectan curso, grupo, filtro de aula

o intención de consulta de tutorías mediante patrones rígidos, lo que las hace robustas pero frágiles ante fraseos atípicos. Un clasificador LLM ligero por turno tendría un coste despreciable y convertiría varios de los casos actualmente excluidos como trabajo futuro en casos PASS.

La tercera palanca es una **caché de respuestas a las consultas más frecuentes**. La distribución del tráfico recogida en la tabla `conversation_log` muestra que un puñado de preguntas (“¿qué es ADDA?”, “horario de IS 2 grupo 1”, “email de Galindo”) concentran un porcentaje significativo de los turnos. Una caché por *hash* de pregunta y titulación reduciría a la mitad el coste anual y bajaría la latencia mediana al rango de uno o dos segundos.

La cuarta palanca es la **instrumentación de métricas en producción**: latencia por percentil, tasa de *fallback*, tasa de *out_of_scope* y *feedback* explícito por turno. La tabla de *feedback* ya está soportada parcialmente en la base de datos pero no se está explotando aún. Esta instrumentación permitiría detectar regresiones en producción mucho antes de que un usuario las reporte.

La quinta palanca es la **persistencia de la última intención específica como slot consultable**, dirigida a resolver los casos de seguimiento conversacional entre sub-*intents* (P-S01 y H-S01 del Capítulo 12). La solución consiste en mantener *slots* con tiempo de vida acotado que permitan a las acciones recuperar la intención del turno previo sin pedir al clasificador NLU que la infiera del historial.

14.4.2. Ampliación funcional

El alcance funcional del prototipo puede extenderse en varias direcciones. La más inmediatea es la incorporación de las titulaciones del centro distintas a las tres de Ingeniería Informática actualmente cargadas, lo cual requiere principalmente trabajo de carga de datos en el panel de administración. A medio plazo, podría considerarse la incorporación de nuevos dominios al bot, como información sobre tutorías estructuradas, normativa académica o convocatorias de movilidad, siempre que se disponga de una fuente oficial estable y vectorizable. Por último, la ampliación natural a otros centros de la Universidad de Sevilla pasa por validar el grado en que los *prompts* y los flujos diseñados son transferibles cuando cambian los datos de origen.

14.4.3. Validación reforzada

La validación realizada en la fase 1 cubre cinco capas funcionales y dos métricas no funcionales, pero deja fuera tres aspectos que conviene atender en una segunda iteración. El primero es la medición de la experiencia subjetiva del usuario mediante una escala estandarizada como SUS, que permitiría situar el sistema frente a otras herramientas universitarias. El segundo es la garantía de disponibilidad sostenida del servicio mediante un acuerdo de nivel de servicio y la correspondiente monitorización de uptime. El tercero es la implementación de la suite automática de pruebas para los *endpoints* del panel de administración, cuyo diseño ya consta en el plan de aceptación pero cuyo desarrollo se ha pospuesto.

14.5– Reflexión final

Mirar atrás al proyecto completo deja una impresión clara: la mayor dificultad de un sistema conversacional basado en modelos de lenguaje no reside en hacerlo funcionar, sino en hacerlo funcionar de forma fiable y trazable. Construir una primera versión que respondiera correctamente a un puñado de consultas demostrativas resultó relativamente rápido; alcanzar el punto en que cinco capas independientes de validación superan su umbral académico de referencia ha exigido un trabajo metódico de evaluación, corrección de defectos y registro de decisiones que ha constituido la mitad del esfuerzo del proyecto.

Esta experiencia sugiere que la incorporación de modelos de lenguaje a aplicaciones reales no se resuelve únicamente con buenas elecciones tecnológicas. Exige adoptar prácticas de ingeniería del software clásicas, en particular pruebas de aceptación, registro de decisiones y validación con

usuarios, adaptadas al carácter no determinista del componente generativo. Linceus Assistant, en su versión actual, es un primer paso en esa dirección dentro del contexto universitario; las cinco palancas y las tres ampliaciones identificadas para la fase 2 deberían permitir convertirlo en una herramienta utilizable de forma sostenida por la comunidad de la ETSII.

CAPÍTULO 15

Uso de Inteligencia Artificial

15.1– Herramientas utilizadas

15.2– Tareas asistidas y verificación

15.3– Limitaciones y responsabilidad del autor

CAPÍTULO 16

Vídeo demostración y presentación

16.1– Vídeo demostración

16.2– Estructura de la presentación

Apéndices

.1– Actas de reuniones con el tutor

Las reuniones de seguimiento con el tutor se celebraron al inicio de cada sprint. A continuación se recogen las actas en orden cronológico inverso.

Datos de contacto

Tutor: José Antonio Parejo Maestre — japarejo@us.es
Alumno: Santia Bregu — sanbre@alum.us.es

Sprint 7 — 29/04/2026

Desarrollo de la reunión:

- Ajustar enfoque del resumen y añadir resumen en inglés.
- Incluir apéndice con actas de reuniones.
- Mejoras en manuales: capturas de pantalla, versiones de imágenes Docker y ejemplos de código.
- Revisión de diagramas y EDT.
- Pendiente: sección de metodología y revisión del capítulo de pruebas.
- Explicación de entregables y siguientes pasos hacia la presentación.

Objetivos acordados:

- Reescribir el resumen y elaborar la sección de metodología.
- Añadir apéndice de actas y mejorar los manuales.
- Revisar el capítulo de pruebas.
- Enviar nueva versión de la memoria al tutor.
- Preparar presentación y vídeo de defensa.

Sprint 6 — 15/04/2026

Desarrollo de la reunión:

- Enfoque en validación del sistema y escritura de la memoria.
- Definición de pruebas de aceptación (conversaciones e importaciones).
- Dos fases de validación: prototipo inicial y validación final.
- Uso de la guía de referencia para estructurar la memoria; secciones clave: introducción, conclusiones, manuales y validación.
- Tipos de manuales a redactar: usuario, despliegue y desarrollo.

- Envío progresivo de la memoria para obtener feedback continuo.
- Feedback positivo del estado actual del sistema.
- Planificación de sprint corto y fecha de próxima reunión.

Objetivos acordados:

- Comenzar la redacción de la memoria con las secciones prioritarias.
- Definir y ejecutar el plan de pruebas de aceptación.
- Cerrar el desarrollo progresivamente y estabilizar el sistema.

Sprint 5 — 25/03/2026

Desarrollo de la reunión:

- Revisión de objetivos del sprint anterior (testing, memoria, sistema).
- Presentación del testing automatizado y manual; corrección de errores detectados.
- Mejoras en el sistema: mapeo de abreviaturas, *fuzzy matching* y fallback con IA.
- Gestión de contexto con *slots* para consultas encadenadas.
- Revisión del frontend: añadir enlaces a la web oficial de la US.
- Revisión de la memoria: EDT, arquitectura y decisiones de diseño.
- Correcciones en diagramas: uso de UML estándar.
- Revisión de horas dedicadas y mejora del tracking.

Objetivos acordados:

- Desplegar el sistema y probarlo con usuarios reales.
- Dockerizar la aplicación.
- Implementar la épica de horarios y aulas.
- Mejorar la memoria: EDT y diagramas UML.
- Añadir enlaces en el frontend y aumentar horas dedicadas.

Sprint 4 — 27/02/2026

Desarrollo de la reunión:

- Revisión del estado actual del proyecto e identificación de tareas pendientes.
- Decisión de centrar el sprint en testing y consolidación del sistema.
- Definición del enfoque de pruebas: manual y automatizado.
- Propuesta de tareas opcionales: frontend y despliegue.
- Planificación de la próxima reunión.

Objetivos acordados:

- Realizar testing exhaustivo: definir plan, ejecutar iteraciones y evaluar resultados.
- Identificar y corregir errores del sistema actual.
- Avanzar la memoria: arquitectura, decisiones de diseño y diagramas.
- Elaborar la EDT a nivel abstracto.
- Opcional: despliegue del sistema y versión inicial del frontend.

Sprint 3 — 03/02/2026

Desarrollo de la reunión:

- Revisión del avance de la épica de asignaturas.
- Planificación de la épica 7.3 (Asignaturas avanzado).
- Revisión de la estructura de la BD y del marco tecnológico.

Objetivos acordados:

- Comenzar el desarrollo de la épica 7.3 (Asignaturas avanzado).
- Documentar en la plantilla TFG lo realizado: estructura de la BD, marco tecnológico y metodología.
- Mejorar las consultas actuales ampliando el contexto conversacional con el framework.
- Ampliar el modelo de información para dar soporte a múltiples titulaciones.
- Ampliar el conjunto de datos del módulo RAG y realizar pruebas.

Sprint 2 — 08/01/2026

Desarrollo de la reunión:

- Resumen del estado actual: datos de una titulación cargados (IS, ETSII) y frontend integrado con el backend Rasa.

Objetivos acordados:

- Definir e implementar las historias de asignaturas: listado por titulación, créditos, cuatrimestre de pertenencia e impartición.
- Implementar pruebas de las historias anteriores.
- Comenzar la épica 7.3 (Asignaturas avanzado).
- Visualizar los vídeos del tutorial de integración de IA con Rasa.
- Documentar en la plantilla TFG: estructura de la BD, marco tecnológico y metodología.
- Comprobar infraestructura para ejecutar Ollama (modelo ligero tipo llama3.2).

Sprint 1 — 22/09/2025

Desarrollo de la reunión:

- Revisión del anteproyecto.
- Definición del alcance del proyecto.
- Fijación de objetivos para el sprint 1.
- Decisión: añadir un diccionario global de términos relevantes durante el desarrollo.

Objetivos acordados:

- Finalizar el anteproyecto.
- Comenzar con el ítem del backlog “Asignaturas”.

Sprint 0 — 28/07/2025

Desarrollo de la reunión:

- Revisión del anteproyecto.
- Creación de la infraestructura básica de trabajo.
- Adjudicación oficial del TFG con título provisional: *“Diseño y desarrollo de un chat inteligente para consulta de información académica”*.
- Decisiones: usar GitHub como repositorio principal y registrar todas las horas desde el inicio, incluidas reuniones.

Objetivos acordados:

- Justificar la elección tecnológica con estudio de alternativas y tabla comparativa.
- Montar la infraestructura: repositorio en GitHub y producto mínimo viable funcionando.
- Definir metodología y estándares de desarrollo.
- Elaborar la planificación inicial con horas aproximadas y objetivos por sprint.

.2– Listado completo de casos de prueba

.3– Esquema SQL de la base de datos

.4– Configuración de Rasa

.5– Informe de tiempo invertido

Bibliografía

- [1] Universidad de Sevilla, “Universidad de Sevilla – portal institucional.”<https://www.us.es>. Consultado en abril de 2026.
- [2] Universidad de Sevilla, “Escuela Técnica Superior de Ingeniería Informática.”<https://www.informatica.us.es>. Consultado en abril de 2026.
- [3] Universidad de Sevilla, “Sevius – portal del personal de la Universidad de Sevilla.”<https://sevius.us.es>. Consultado en abril de 2026.
- [4] Universidad de Sevilla, “Sevius4 – portal de docencia y planes de estudio.”<https://sevius4.us.es>. Consultado en abril de 2026.
- [5] Universidad de Sevilla, “Secretaría Virtual de la Universidad de Sevilla.”<https://sevius.us.es/secretariavirtual>. Consultado en abril de 2026.
- [6] Universidad de Sevilla, “Portal de matrícula y automátrula de la Universidad de Sevilla.”<https://www.us.es/estudiar/matricula>. Consultado en abril de 2026.
- [7] Universidad de Sevilla, “Plataforma de pago de tasas académicas.”<https://www.us.es/estudiar/matricula/precios-publicos-y-becas>. Consultado en abril de 2026.
- [8] 1MillionBot, “1MillionBot: AI for Universities.”<https://1millionbot.com/en/universidades/>, 2026. Consultado en abril de 2026.
- [9] Universidad de Sevilla, “CATi te ayuda y acompaña en tu vida universitaria.”<https://www.us.es/actualidad-de-la-us/cati-te-ayuda-y-acompana-en-tu-vida-universitaria>, 2022. Consultado en abril de 2026.
- [10] Torre Juana OST, “CATi, el asistente inteligente de la Universidad de Sevilla: 98 % tasa de acierto.”<https://ost.torrejuana.es/cati-asistente-universidad-sevilla-98-tasa-acierto/>, 2022. Consultado en abril de 2026.
- [11] Universidad de Murcia, “La Universidad de Murcia presenta a LOLA, un asistente de inteligencia artificial para ayudar a los nuevos alumnos.”<https://www.um.es/web/sala-prensa/-/la-universidad-de-murcia-presenta-a-lola-un-asistente-de-inteligencia-artificial>, 2018. Consultado en abril de 2026.
- [12] 1MillionBot, “Chatbot Lola – Universidad de Murcia.”<https://1millionbot.com/en/chatbot-lola-umu/>, 2018. Consultado en abril de 2026.
- [13] Universidad de Granada, “Nuevo asistente virtual llamado Alhe, dirigido al estudiantado y futuro estudiantado.”<https://www.ugr.es/universidad/noticias/asistente-virtual-llamado-ahle-dirigidoal-estudiantado-futuro-estudiantado>, 2023. Consultado en abril de 2026.

- [14] 1MillionBot, “La IA de 1MillionBot se implementa en la Universidad de Granada.”<https://1millionbot.com/en/la-ia-de-1millionbot-se-implementa-en-la-universidad-de-granada/>, 2023. Consultado en abril de 2026.
- [15] Universidad de Cadiz, “LUCA, nuevo chatbot inteligente de la Universidad de Cadiz para asistencia al estudiante.”<https://sistinfo.uca.es/noticia/luca-nuevo-chatbot-inteligente-de-la-universidad-de-cadiz-para-asistencia-> 2023. Consultado en abril de 2026.
- [16] Universidad de Alcala, “La Universidad de Alcala presenta un nuevo asistente virtual para su pagina web.”<https://portalcomunicacion.uah.es/sala-prensa/notas-prensa/la-universidad-de-alcala-presenta-un-nuevo-asistente-virtual-para-su-pagina-web/>, 2024. Consultado en abril de 2026.
- [17] El Complutense, “Alex, el asistente virtual creado por estudiantes de la UAH que atiende consultas academicas en AULA 2026.”<https://elcomplutense.com/alex-chatbot-estudiantes-uah/>, 2026. Consultado en abril de 2026.
- [18] Unibuddy Ltd., “Unibuddy: Student-to-student recruitment at scale.”<https://unibuddy.com/>, 2026. Consultado en abril de 2026.
- [19] Unibuddy Ltd., “Introducing a Unibuddy Assistant.”<https://unibuddy.com/a-unibuddy-assistant/>, 2024. Consultado en abril de 2026.
- [20] Hubert AI AB, “Hubert.ai – AI chatbot for course evaluation.”<https://www.hubert.ai/>, 2026. Consultado en abril de 2026.
- [21] R. Gupta *et al.*, “AdmitBot: A system for automated admission counseling,”*in IJCAI Workshop*, 2019.
- [22] F. Colace, M. De Santo, M. Lombardi, F. Pascale, A. Pietrosanto, and S. Lemma, “Chatbot for E-Learning: A case of study,”*International Journal of Mechanical Engineering and Robotics Research (IJMERR)*, 2018.
- [23] B. R. Ranoliya, N. Raghuwanshi, and S. Singh, “Chatbot for university related FAQs,”*in IEEE ICACCI*, 2017.
- [24] D. K. Dibya and S. Sahoo, “Implementation of a chatbot system using AI and NLP,”*in IEEE ICCNT*, 2021.
- [25] T. Bocklisch, J. Faulkner, N. Pawlowski, and A. Nichol, “Rasa: Open source language understanding and dialogue management,”*in NIPS 2017 Conversational AI Workshop*, 2017.
- [26] Botpress Inc., “Botpress documentation,” 2024. Consultado: noviembre 2025.
- [27] Neural Networks and Deep Learning Lab, MIPT, “DeepPavlov: An open-source library for deep learning end-to-end dialog systems,” 2023. Consultado: noviembre 2025.
- [28] Rasa Technologies, “Rasa open source documentation,” 2024. Consultado: abril 2026.
- [29] T. Bocklisch, J. Faulkner, N. Pawlowski, A. Nichol, and V. Vlasov, “DIET: Lightweight language understanding for dialogue systems.”*arXiv:2004.09936*, 2020.
- [30] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,”*in NeurIPS*, 2020.

- [31] Supabase Inc., “Supabase documentation,”2024. Consultado: diciembre 2025.
- [32] A. Kane, “pgvector: Open-source vector similarity search for Postgres,”2024. Consultado: diciembre 2025.
- [33] Qdrant Solutions GmbH, “Qdrant documentation,”2024. Consultado: noviembre 2025.
- [34] Weaviate B.V., “Weaviate documentation,”2024. Consultado: noviembre 2025.
- [35] Google DeepMind, “Gemini API documentation,”2024. Consultado: enero 2026.
- [36] K. Schwaber and J. Sutherland, “The Scrum guide: The definitive guide to Scrum: The rules of the game.”<https://scrumguides.org/scrum-guide.html>, 2020. Consultado en abril de 2026.
- [37] Conventional Commits, “Conventional commits 1.0.0 – a specification for adding human and machine readable meaning to commit messages.”<https://www.conventionalcommits.org/en/v1.0.0/>, 2019. Consultado en abril de 2026.
- [38] G. van Rossum, B. Warsaw, and N. Coghlan, “PEP 8 – style guide for Python code.”<https://peps.python.org/pep-0008/>, 2001. Consultado en abril de 2026.
- [39] PyCQA, “Pylint – a static code analyser for Python.”<https://pylint.readthedocs.io/>, 2001. Consultado en abril de 2026.
- [40] PyCQA, “Flake8: Your tool for style guide enforcement.”<https://flake8.pycqa.org/>, 2010. Consultado en mayo de 2026.
- [41] ESLint, “ESLint: Find and fix problems in your JavaScript code.”<https://eslint.org/>, 2013. Consultado en abril de 2026.
- [42] S. Segura and A. Ruiz, “Guia para proyectos fin de grado sobre desarrollo de software.”https://github.com/ssegura/Guia_TFG, 2024. Departamento de Lenguajes y Sistemas Informaticos, ETSII, Universidad de Sevilla.
- [43] International Organization for Standardization, “ISO/IEC 25010:2023 – systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – product quality model,”2023.
- [44] D. North, “Introducing BDD.”Better Software Magazine, 2006.
- [45] Xoriant, “A comprehensive guide to chatbot testing.”White paper, 2020.
- [46] Test IO Academy, “Chatbot testing: Approaches, challenges and best practices,”2021.
- [47] D. Braun, A. Hernandez-Mendez, F. Matthes, and M. Langen, “Evaluating natural language understanding services for conversational question answering systems,”in SIGDIAL, ACL, 2017.
- [48] I. Casanueva, T. Temcinas, D. Gerz, M. Henderson, and I. Vulic, “Efficient intent detection with dual sentence encoders,”in ACL NLP4ConvAI Workshop, 2020.
- [49] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,”in EACL, 2024.
- [50] J. Casas, M.-O. Tricot, O. Abou Khaled, E. Mugellini, and P. Cudre-Mauroux, “Trends & methods in chatbot evaluation,”in CHIIR Workshop, ACM, 2020.
- [51] A. Folstad and P. B. Brandtzaeg, “Users’ experiences with chatbots: findings from a questionnaire study,”Quality and User Experience, Springer, 2020.